MULTI-ENVIRONMENT CONFIGURATION MANAGEMENT WITH HELM

A PROJECT REPORT

Submitted by

Arun Kumar T (Reg.No.953724243007)

Priyan M (Reg.No.953724243040)

In partial fulfillment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

AAA COLLEGE OF ENGINEERING AND TECHNOLOGY, SIVAKASI

(AN AUTONOMOUS INSTITUTION)

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**Multi-Environment Configuration Management with Helm**" is the bonafide work of **Arun Kumar T (Reg.No.953724243007), Priyan M (Reg.No.953724243040)**, who carried out this project under my supervision.

SIGNATURE

SUPERVISOR NAME, QUALIFICATION
SUPERVISOR &
DESIGNATION
Department of Artificial Intelligence and Data Science
AAA College of Engineering and Technology
Sivakasi

SIGNATURE

Dr.M.KARUPPASAMY, M.Tech., Ph.D.,
ASSOCIATE PROFESSOR &
HEAD OF THE DEPARTMENT
Department of Artificial Intelligence and Data Science
AAA College of Engineering and Technology
Sivakasi

Submitted for viva - voce examination held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We would first like to thank Almighty for the blessings that made this work possible. We thank our Management, AAA College of Engineering and Technology, Sivakasi, for providing the facilities and infrastructure needed for the project.

We express our sincere thanks to **Dr. M. SEKAR M.E., Ph.D., (S.Korea)**, Principal, AAA College of Engineering and Technology, Sivakasi, for the necessary facilities and for permitting us to carry out this work.

We express our gratitude to **Dr.M.KARUPPASAMY M.Tech., Ph.D.**, Associate Professor and Head of the Department, Department of Artificial Intelligence and Data Science, for his constant encouragement, which helped us to complete the project.

We wish to express our deep sense of gratitude to our Project Guide **SUPERVISOR NAME, DESIGNATION**, for the guidance, supervision and information provided throughout the project.

We express our gratitude to our Project Coordinator **PROJECT COORDINATOR NAME, DESIGNATION**, Department of Artificial Intelligence and Data Science, for valuable advice and constant encouragement.

It is our privilege to record our gratitude to all the faculty members and supporting staff of the Department of Artificial Intelligence and Data Science, whose encouragement and suggestions have been a source of inspiration to us.

ABSTRACT

Teams that deploy one application to development, test, staging and production often copy a set of Kubernetes manifests for every environment. The copies drift apart, a change must be repeated several times, and a wrong value is found only after the deployment. This project describes the application once, as the Helm chart `multi-env-app`, and expresses every difference between the environments in a values file of 28 to 43 lines. A JSON Schema rejects unknown environments, wrong types, out-of-range numbers and secrets in the configuration before anything reaches a cluster, and a checksum annotation restarts the Pods when a setting changes.

The application is a Flask dashboard with Bootstrap that shows its name, environment, version and configuration. It is packaged as one image of 126 MB that is promoted unchanged through the four environments. Nine scripts install, validate, test, upgrade and roll back the releases, 62 pytest cases check the application and the chart, and a GitHub Actions workflow lints, renders and validates the chart, builds the image and deploys all four environments to a kind cluster.

On the cluster, all four releases were deployed in 40 s, 56 of 56 compared settings of the running Pods equalled the values files, all 26 live checks passed, a rollback restored an earlier revision, and `--atomic` restored a release after an upgrade to a non-existent image. A measurement of 13 rolling updates showed that `maxUnavailable: 0` keeps full capacity but still lost 1.2 to 1.5 % of the requests in short bursts, which a `preStop` delay removed; “zero downtime” is therefore not claimed. The report states for every result whether it was run on a cluster, run locally or derived. Minikube, which the scripts support, was not executed.

Keywords: Helm, Kubernetes, Configuration Management, Multi-Environment Deployment, Values Files, JSON Schema, Rolling Update, GitHub Actions, Release Rollback

TABLE OF CONTENTS

CHAPTER NO.	DESCRIPTION	PAGE NO.
	ABSTRACT	**iv**
	LIST OF FIGURES	**vii**
	LIST OF TABLES	**ix**
	LIST OF ABBREVIATIONS	**x**
	GLOSSARY OF TECHNICAL TERMS	**xi**
I	**INTRODUCTION**	**1**
	1.1 Foundation and Context of Configuration Management in Kubernetes	1
	1.2 DevOps, Containers and the Emergence of Kubernetes	1
	1.3 Helm as the Package Manager for Kubernetes	2
	1.4 The Multi-Environment Configuration Problem	2
	1.5 Problem Statement	3
	1.6 Objectives of the Project	3
	1.7 Scope and Limitations of the Work	4
	1.8 Project Contribution and Significance	4
	1.9 Overview of the Multi-Environment Helm Workflow	5
	1.10 Organisation of the Report	5
II	**LITERATURE SURVEY**	**6**
	2.1 Research Papers	6
	2.2 Existing Approaches to Configuration Management	9
	2.3 Objective	9
	2.4 Research Gaps	10
III	**PROPOSED METHODOLOGY**	**11**
	3.1 Existing System and Its Limitations	11
	3.2 Proposed System	11
	3.3 System Requirements	12
	3.4 Block Diagram and Explanation	13
	3.5 Application Design	14
	3.5.1 Routes and Endpoints	14
	3.5.2 Configuration Loading	15
	3.5.3 Dashboard: HTML, CSS and Bootstrap	17
	3.5.4 Health Probes and Logging	19
	3.6 Containerisation	19
	3.7 Helm Chart Design	20
	3.7.1 Chart Structure and Metadata	20
	3.7.2 Helper Templates and Naming	21
	3.8 Values and Environment-Specific Configuration	21
	3.8.1 Layering of Values	21
	3.8.2 Configuration of the Four Environments	23
	3.8.3 Schema Validation of Values	23
	3.9 Helm Templates	24
	3.9.1 ConfigMap Template	25
	3.9.2 Deployment Template	25
	3.9.3 Rolling-Update Strategy	27
	3.9.4 Service Template	27
	3.9.5 Helm Test and Release Notes	28
	3.9.6 Rendered Objects of the Four Environments	29

	3.10	Kubernetes Topology and Objects	29
	3.11	Release Management with Helm	31
	3.12	Automation Scripts	34
	3.13	Testing Strategy	35
	3.14	Continuous Integration with GitHub Actions	37
	3.15	Security and Secret Handling	38
	3.16	Resource Requirements of the Four Environments	39
IV	**RESULTS AND DISCUSSION**		**40**
	4.1	Basis of the Results	40
	4.2	Deployment of the Four Environments	41
	4.3	Dashboards of the Four Environments	43
	4.4	Test Cases and Results	47
	4.5	Reduction of Duplicated Configuration	51
	4.6	Schema Validation Results	53
	4.7	Upgrade and Rollback	54
	4.8	Forced Failures	56
		4.8.1 Upgrade to a non-existent image with --atomic	56
		4.8.2 Production installed without its Secret	57
	4.9	Availability during Rolling Updates	58
		4.9.1 Question and Design of the Experiment	58
		4.9.2 Results	59
		4.9.3 Interpretation	60
		4.9.4 Limits of the Experiment	61
	4.10	Image and Pipeline Results	61
	4.11	Discussion	63
		4.11.1 Objectives and Results	63
		4.11.2 Findings	64
		4.11.3 Threats to Validity	64
V	**CONCLUSION AND FUTURE ENHANCEMENTS**		**66**
	5.1	Conclusion	66
	5.2	Summary of Contributions	66
	5.3	Comparison with the Literature	67
	5.4	Practical Deployment Considerations	67
	5.5	Limitations	68
	5.6	Future Enhancements	68
VI	**APPENDICES**		**70**
		Appendix A – Directory structure of the project	70
		Appendix B – Application source code	71
		Appendix C – Container files	75
		Appendix D – Helm chart	76
		Appendix E – Scripts	84
		Appendix F – Tests	94
		Appendix G – GitHub Actions workflows	98
		Appendix H – Evidence files	101
		Appendix I – Reproducing the results	101
VII	**REFERENCES**		**103**

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Copied manifests per environment compared with one chart and four values files	3
1.2	Multi-environment Helm workflow overview	5
3.1	Block diagram of the multi-environment Helm solution	13
3.2	Application inside a Pod: sources of configuration, request flow and probes	14
3.3	Parts of the dashboard (production release)	18
3.4	Layered structure of the application image	19
3.5	Values layering and schema validation	22
3.6	Cluster topology: one namespace and one Helm release per environment	30
3.7	Objects of one release and the references between them	30
3.8	Lifecycle of a Helm release	31
3.9	Sequence of a deployment with `helm upgrade --install --atomic --wait`	33
3.10	Distribution of the 62 pytest cases over the three automated levels	36
3.11	Jobs of the validation workflow	37
4.1	Helm releases after `scripts/deploy.sh all` (`helm list -A`, update time omitted)	41
4.2	Objects of the production environment (`kubectl get deploy,svc,cm,pods -n prod`; address, selector and node columns omitted)	42
4.3	Data section of the production ConfigMap (`kubectl get configmap ... -o yaml`, metadata omitted)	42
4.4	Deployment of the production environment (`kubectl describe deployment`, selected lines)	43
4.5	Secrets of the production namespace (`kubectl get secret -n prod`)	43
4.6	Dashboard of the development environment, served by the Pod myapp-dev-multi-env-app-576769ffc9-gr8ld	44
4.7	Dashboard of the testing environment, served by the Pod myapp-test-multi-env-app-df8d675b7-25z2b	44
4.8	Dashboard of the staging environment, served by the Pod myapp-staging-multi-env-app-598477f467-2t8sc	45
4.9	Dashboard of the production environment, served by the Pod myapp-prod-multi-env-app-78c6bddb45-4xx97	45
4.10	Environment-specific settings read from the values files	47
4.11	Logs of a development Pod and a production Pod (`kubectl logs`, probe requests omitted)	47
4.12	Result of the pytest run on the development machine	48
4.13	Output of `scripts/validate-helm.sh` (lines of the three stages of each environment)	49
4.14	Live checks `scripts/test-environments.sh all` on the kind cluster	50

4.15	`helm test` for the production release	51
4.16	Lines to maintain: copied manifests compared with the chart and values files (level C)	52
4.17	Four invalid values rejected by `helm lint --strict` (error messages shortened to the first line)	54
4.18	Upgrade, history, rollback and history of the development release (update times omitted)	55
4.19	Dashboard of the development release after the upgrade (revision 2) and after the rollback (revision 3); upper part of the page	56
4.20	Failed upgrade with `--atomic`: the old Pod keeps running and Helm restores revision 1 as revision 3	57
4.21	Production installed without its Secret (`helm install`, `kubectl get pods`, events)	58
4.22	Failed requests per rollout in the five scenarios (bars: mean; dots: individual runs)	60
4.23	Size of the layers of the application image (from `docker history`)	62
4.24	Timeline of the four jobs of the validation workflow (run 6, commit 9f0e6bc)	62
5.1	Roadmap of future enhancements	69

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Project objectives and where they are addressed	3
1.2	Separation of responsibilities in the workflow	5
2.1	Mapping of surveyed literature to design decisions of this project	8
2.2	Comparison of configuration-management approaches	9
3.1	Existing and proposed systems compared	11
3.2	Software and hardware environment	12
3.3	Application routes and their roles	15
3.4	Environment variables read by the application	16
3.5	Instructions of the Dockerfile and their purpose	20
3.6	Files of the Helm chart and their roles	20
3.7	Worked example: effective values of a staging release with two command-line overrides	22
3.8	Environment-specific configuration (values that differ)	23
3.9	Rules of values.schema.json	24
3.10	Values consumed by the Deployment template	25
3.11	Probe settings of the chart	27
3.12	Rendered manifests and size of the environment files	29
3.13	Kubernetes objects of one release	31
3.14	Helm commands used in the project	32
3.15	Scripts of the project	34
3.16	Test levels	35
3.17	Jobs of the workflow `helm-validation.yml`	37
3.18	Security measures	38
3.19	Resources requested by the four environments (m = millicores of a CPU, Mi = mebibytes)	39
4.1	Evidence levels used in this report	40
4.2	System of the cluster run (level A)	40
4.3	Releases of the four environments in the cluster (level A)	41
4.4	Settings reported by the running Pods through /api/config (level A)	46
4.5	Test cases and results	48
4.6	Lines to maintain for E environments	52
4.7	Invalid values rejected by values.schema.json	53
4.8	Revisions of the development release	56
4.9	Scenarios of the availability experiment	59
4.10	Failed requests during rolling updates (level A; totals over the runs of a scenario)	59
4.11	Runs of the validation workflow up to commit 9f0e6bc	63
4.12	Objectives compared with the results	63
5.1	Observations of this project compared with the literature	67
5.2	Future enhancements and their reasons	69
A.1	Evidence files	101

LIST OF ABBREVIATIONS

API	Application Programming Interface
CI	Continuous Integration
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DevOps	Development and Operations
GitOps	Operating infrastructure from a Git repository
HPA	Horizontal Pod Autoscaler
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
IaC	Infrastructure as Code
JSON	JavaScript Object Notation
kind	Kubernetes in Docker
OCI	Open Container Initiative
PDB	Pod Disruption Budget
SHA	Secure Hash Algorithm
TLS	Transport Layer Security
UID	User Identifier
URL	Uniform Resource Locator
WSGI	Web Server Gateway Interface
YAML	YAML Ain't Markup Language

GLOSSARY OF TECHNICAL TERMS

Terms	Definition
Atomic upgrade	An upgrade started with `--atomic`: if the release does not become ready within the timeout, Helm rolls it back automatically.
Chart	A Helm package: templates, a file of default values and metadata that describe an application.
Checksum annotation	An annotation of the Pod template that contains the hash of the ConfigMap, so that a changed configuration restarts the Pods.
ConfigMap	A Kubernetes object that holds non-confidential settings as key-value pairs.
Deployment	A Kubernetes object that keeps a desired number of identical Pods running and performs rolling updates.
Downward API	A way to give a container information about its own Pod, such as its name and namespace, as environment variables.
Evidence level	The label A to D of this report that states where and how a result was obtained.
Helm test	A Pod defined in the chart with the annotation `helm.sh/hook: test` that checks a release when `helm test` is run.
Liveness probe	A periodic check; if it fails, Kubernetes restarts the container.
Namespace	A Kubernetes mechanism that separates groups of objects in one cluster; one per environment here.
NodePort	A Service type that publishes a port on every node of the cluster.
Pod	The smallest deployable unit of Kubernetes; one or more containers that share a network address.
preStop hook	A command that Kubernetes runs in a container before it sends the termination signal.
Readiness probe	A periodic check; while it fails, the Pod receives no traffic from its Service.
Release	One installation of a chart with one set of values, identified by a name.
Revision	A numbered version of a release; every install, upgrade and rollback creates one.
ReplicaSet	A Kubernetes object that owns the Pods of one version of a Deployment.
Rolling update	Replacement of the Pods of a Deployment step by step, controlled by `maxSurge` and `maxUnavailable`.
Schema	A JSON Schema file (`values.schema.json`) that defines the permitted values of a chart.
Secret	A Kubernetes object for confidential data; stored base64-encoded unless encryption at rest is configured.
Service	A Kubernetes object that gives a set of Pods a stable address and balances requests between them.
Values file	A YAML file whose entries override the defaults of a chart; one per environment here.

CHAPTER 1
INTRODUCTION

1.1 Foundation and Context of Configuration Management in Kubernetes

Every application that reaches users passes through several environments. Developers run it on a small development cluster, testers exercise it in a test environment, a staging environment imitates production, and production serves the real users. The code is the same in all of them; the configuration is not. Replica counts, resource limits, log levels, host names, image versions and feature flags all change from one environment to the next.

The Twelve-Factor App method states that configuration, which varies between deploys, must be strictly separated from code, which does not [1]. Empirical studies of real configuration errors in commercial and open-source systems [2] and surveys of the techniques developed against them [3] show why this separation deserves engineering discipline of its own: a wrong value can stop a perfectly correct program. Kubernetes describes deployments as YAML manifests, and the question this project addresses is how those manifests should be organised when one application is deployed to four environments.

1.2 DevOps, Containers and the Emergence of Kubernetes

DevOps combines software development and IT operations practices and automates repetitive work such as building, testing, deploying and releasing. The deployment pipeline idea in *Continuous Delivery* makes every change travel through the same automated steps to every environment [4]. Research on software delivery performance associates such automation, together with version-controlled configuration, with better delivery outcomes [5].

Containers package an application with its dependencies so that it behaves identically wherever it runs [6]. Kubernetes, an open-source container orchestrator whose design draws on Google's experience with its Borg cluster manager [7, 8], keeps the desired number of containers running, connects them with Services and replaces them gradually during updates. Its Deployment controller performs rolling updates and records revisions that can be reverted [9]. What Kubernetes does not provide by itself is a convenient way to describe the same application several times with controlled differences. This is the gap that Helm fills.

1.3 Helm as the Package Manager for Kubernetes

Helm packages a Kubernetes application as a *chart*: a directory that contains templates, a file of default values and metadata [10]. When a chart is installed, Helm combines the templates with the supplied values, produces ordinary Kubernetes manifests and stores the result as a numbered *revision* of a named *release*. Because every revision is kept, an earlier one can be restored with a single command [11].

Values can be layered: the chart's own `values.yaml` is overridden by files passed with `-f`, which are in turn overridden by `--set` on the command line [12]. This layering is the mechanism that the project uses to separate what all environments share from what each environment changes. The Helm ecosystem is large and widely used, but studies of public charts also report outdated components and security risks [13], which is one reason why this project fixes the versions of its image, its tools and its dependencies explicitly and applies restrictive security settings by default.

1.4 The Multi-Environment Configuration Problem

The simplest way to support four environments is to keep four folders of manifests. At first this works. Over time a fix applied to the production Deployment is forgotten in staging, a new environment variable is added to development only, and nobody can say which differences between the folders are intentional. Studies of defective configuration scripts show that such code is itself defect-prone and needs the same review and testing as application code [14, 15], and security studies of public Kubernetes manifests find misconfigurations in them as well [16].

Most of the content of the four folders is identical. Only a small set of values differs, yet a copy-based approach forces the maintainer to edit, review and test all four complete copies. Figure 1.1 contrasts the two organisations. The project therefore treats the manifests as a single *template* and the differences as *data*.

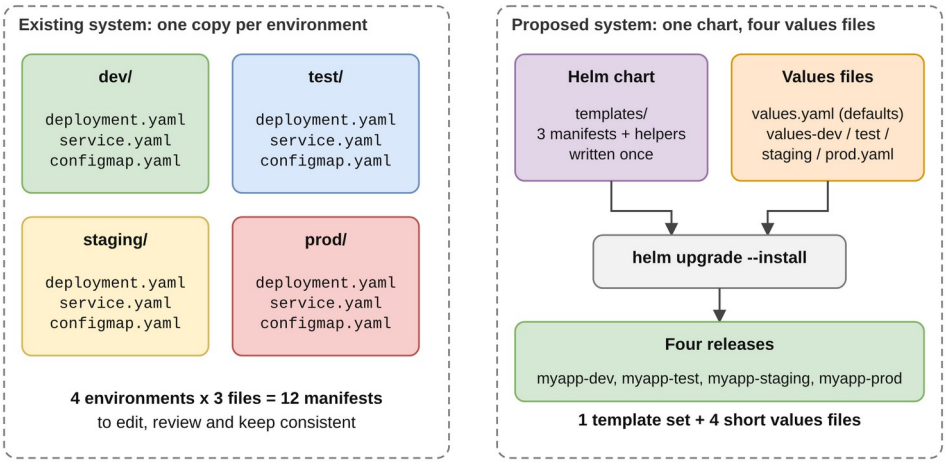


Figure 1.1 – Copied manifests per environment compared with one chart and four values files

1.5 Problem Statement

Design and implement a configuration-management solution in which one application is described once, can be deployed to development, test, staging and production with environment-specific settings, validates those settings before deployment, can be upgraded and rolled back safely, and is checked automatically on every change. The solution must be reproducible on a laptop cluster, must not contain hard-coded secrets and must not require copying manifests between environments.

1.6 Objectives of the Project

The project pursues ten objectives. They follow the requirements of the project brief. Table 1.1 lists them together with the artefact that realises each one and the place in the report where it is discussed.

Table 1.1 – Project objectives and where they are addressed

No.	Objective	Realised by	Discussed in
O1	Simple web application with Flask, HTML, CSS and Bootstrap	`app/app.py`, `app/config.py`, `app/templates/`	Section 3.5
O2	Dashboard showing application name, environment, version and configuration	`index.html`, `/api/config`	Section 3.5.3, Section 4.3
O3	Four environments: Development, Testing, Staging, Production	`values-dev.yaml` ... `values-prod.yaml`	Section 3.8.2, Section 4.2
O4	Reusable Helm chart with the required files	`helm/multi-env-app/`	Section 3.7, Section 3.9
O5	Docker configuration to containerise the application	`Dockerfile`	Section 3.6
O6	Kubernetes deployment on Minikube	`scripts/minikube-setup.sh`, `scripts/deploy.sh`	Section 3.10, Section 4.1

No.	Objective	Realised by	Discussed in
O7	Helm commands for install, validation, upgrade and rollback	`helm upgrade --install`, `helm lint`, `helm rollback`	Section 3.11, Section 4.7
O8	GitHub Actions integration for automatic validation	`.github/workflows/helm-validation.yml`	Section 3.14
O9	Testing scripts for environment-specific configuration	`tests/`, `scripts/test-environments.sh`	Section 3.13, Section 4.4
O10	No hard-coded secrets; consistent configuration	`values.schema.json`, Kubernetes Secret	Section 3.15, Section 4.6

1.7 Scope and Limitations of the Work

The deployed workload is a small stateless web application written with Flask. It was chosen because its behaviour depends entirely on configuration, which makes the effect of every value visible on its dashboard. The four environments are separated by namespaces in one cluster. The chart contains no Secret: the production `SECRET_KEY` is created outside the chart, by the deployment script or with `kubectl`, and is never stored in Git. Ingress, autoscaling and a GitOps controller are not used; Chapter V lists them as extensions.

The chart, the scripts and the tests were executed in two places: the development machine and GitHub Actions, where a temporary *kind* cluster [17] runs all four environments. Minikube [18] is the documented local platform and the scripts support it, but it was not available in the environment in which the evidence for this report was recorded. The commands were run on Linux only. Section 4.1 states for every result where and how it was obtained.

1.8 Project Contribution and Significance

The main contribution is a complete, small and reproducible example of configuration management with Helm that a student team can read in one sitting and run on a laptop. It combines a reusable chart, a schema-validated values interface, an environment promotion model, release management with rollback and a continuous-integration workflow that deploys every environment to a real cluster. The report states which parts were verified, where and how, and it includes experiments that go beyond a demonstration: two deliberate failures (a non-existent image and a missing Secret) and a measurement of the requests that fail while a release is being updated.

1.9 Overview of the Multi-Environment Helm Workflow

Figure 1.2 shows the workflow in four stages. The configuration is stored in Git; Helm merges the values, checks them against the schema and renders the templates; the rendered objects are applied to the namespace of the chosen environment; and the release is then verified, or reverted if the result is wrong. Because the templates never change between environments, only the values file selects the behaviour.

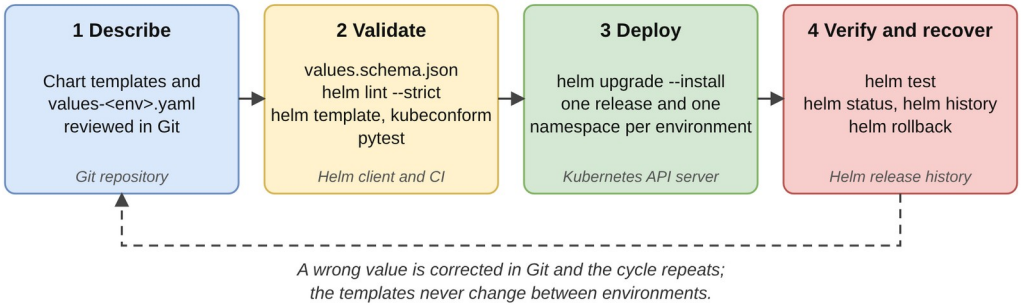


Figure 1.2 – Multi-environment Helm workflow overview

Table 1.2 states which tool performs each stage and where it is defined in the delivered project.

Table 1.2 – Separation of responsibilities in the workflow

Stage	What happens	Tool	Where defined
1. Describe	Templates and values are written and reviewed in Git	Helm chart	`helm/multi-env-app/`
2. Validate	Schema check, lint, render, manifest validation and tests	`helm lint`, `helm template`, kubeconform, pytest	`values.schema.json`, `scripts/validate-helm.sh`, `tests/`
3. Deploy	One release per environment, each in its own namespace	`helm upgrade --install`	`scripts/deploy.sh`
4. Verify	Readiness, in-cluster HTTP test and live checks of the configuration	`helm test`, kubectl, curl	`templates/tests/`, `scripts/test-environments.sh`
5. Recover	Return to an earlier revision, or automatically after a failed upgrade	`helm rollback`, `--atomic`	`scripts/upgrade-rollback-demo.sh`

1.10 Organisation of the Report

Chapter II surveys the literature and compares the main approaches. Chapter III presents the methodology: requirements, architecture, application, container, chart, values, release management, scripts, tests and the CI workflow. Chapter IV states the evidence levels and presents the results of the cluster runs, the tests and the experiments. Chapter V concludes and lists future enhancements.

CHAPTER 2
LITERATURE SURVEY

2.1 Research Papers

The works below were reviewed for their relevance to configuration errors, configuration and infrastructure code, container platforms and automated delivery. Full bibliographic entries are given in the References chapter. The descriptions state only what the cited works report; where a statement concerns this project it is marked as such.

1. Wiggins, A. The Twelve-Factor App, factor III: Config.

The method states that an application's configuration, everything that is likely to vary between deploys, must be stored in the environment and strictly separated from code [1]. This is the principle on which the project is built: the Flask application reads every setting from environment variables, and Helm supplies them from a ConfigMap and, in production, from a Secret.

2. Yin, Z., Ma, X., Zheng, J., Zhou, Y., Bairavasundaram, L. N., & Pasupathy, S. (2011). An empirical study on configuration errors in commercial and open source systems. SOSP '11.

The authors analysed 546 real-world misconfigurations from a commercial storage system and four open-source systems. They found that most of them are mistakes in setting parameters, and that a large share of those mistakes are values that violate the format or the rules of the parameter [2]. The result motivates automatic checking of values, which this project implements with a JSON Schema for the Helm values.

3. Xu, T., & Zhou, Y. (2015). Systems approaches to tackling configuration errors: A survey. ACM Computing Surveys.

The survey classifies the techniques that prevent, detect and diagnose configuration errors [3]. The project uses the first two classes: prevention by restricting what a values file may contain, and detection by linting, rendering and testing every change before it reaches a cluster.

4. Rahman, A., & Williams, L. (2018). Characterizing defective configuration scripts used for continuous deployment. ICST.

The paper characterises defective configuration scripts and identifies properties that are associated with defects [14]. It supports the decision to treat the chart and the values files as code that is reviewed, versioned and tested.

5. Rahman, A., Mahdavi-Hezaveh, R., & Williams, L. (2019). A systematic mapping study of infrastructure as code research. Information and Software Technology.

The mapping study of 31 publications groups infrastructure-as-code research into frameworks and tools, use of IaC, empirical studies and testing, and points out that defects and security problems in such code were little studied [15]. It places this project in the area of IaC practice with an emphasis on testing.

6. Rahman, A., Shamim, S. I., Bose, D. B., & Pandita, R. (2023). Security misconfigurations in open source Kubernetes manifests: An empirical study. ACM TOSEM.

The authors examined 2,039 Kubernetes manifests from 92 open-source repositories and identified 11 categories of security misconfiguration, among them missing resource limits and a missing security context [16]. The chart therefore sets resource requests and limits for every environment, runs the container as a non-root user and drops all Linux capabilities.

7. Zerouali, A., Opdebeeck, R., & De Roover, C. (2023). Helm charts for Kubernetes applications: Evolution, outdatedness and security risks. MSR.

The study of 9,482 charts from Artifact Hub reports that the chart ecosystem is growing, that most charts are unofficial and that many container images carry known vulnerabilities [13]. The project keeps its dependencies few, pins the versions of Flask, gunicorn and the CI tools, and uses a slim base image.

8. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., & Wilkes, J. (2015). Large-scale cluster management at Google with Borg. EuroSys '15; and Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). Borg, Omega, and Kubernetes. ACM Queue.

These papers describe the cluster manager on whose design Kubernetes draws, including declarative job specifications, and the lessons learned from three generations of container-management systems [7, 8]. They explain why the project

describes the desired state declaratively and leaves restarts, rolling updates and health checks to the platform.

9. Limoncelli, T. A. (2018). GitOps: A path to more self-service IT. ACM Queue.

GitOps keeps the desired state of the infrastructure in version control and changes it through reviewed commits [19]. All files of the project, including the values of every environment, are kept in Git and are validated by the same workflow on every change.

10. Humble, J., & Farley, D. (2010). Continuous Delivery; and Forsgren, N., Humble, J., & Kim, G. (2018). Accelerate.

The first book describes the deployment pipeline, an automated and repeatable path of a change to every environment [4]; the second reports research that associates version control, continuous delivery and automation with better delivery performance [5]. The GitHub Actions workflow of the project is a small deployment pipeline of this kind.

11. Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal.

The article introduces Docker and explains how packaging an application with its dependencies gives a consistent environment from development to deployment [6]. It supports the decision to build one image and promote it unchanged through all four environments.

The research literature describes problems and principles; the behaviour of the tools is described in their documentation, the second class of sources used in this report. The Helm documentation defines the layering of values [12], the chart tests [20], the linter [21] and the upgrade and rollback commands [11, 22]. The Kubernetes documentation describes Deployments [9], ConfigMaps [23], Secrets [24], probes [25] and Services [26], and the GitHub documentation defines the workflow syntax [27]. Table 2.1 maps each source to a decision in this project.

Table 2.1 – Mapping of surveyed literature to design decisions of this project

Source	Relevant observation	Design decision in this project
[1]	Configuration must be strictly separated from code	Templates hold the structure, values files hold all environment differences; the app reads environment variables
[2]	Most misconfigurations are wrong parameter values, many of them illegal values	Typed and bounded values checked by `values.schema.json`

Source	Relevant observation	Design decision in this project
[3]	Techniques exist to prevent, detect and diagnose misconfiguration	Prevention (enums, ranges) and detection (`helm lint --strict`, tests, CI)
[14, 15]	Configuration code is defect-prone and is itself a subject of engineering	Chart and values are reviewed, versioned and tested like code
[16]	Security misconfigurations such as missing limits are common in Kubernetes manifests	Resource limits, non-root user, dropped capabilities, masked secret
[13]	Public charts show outdated and vulnerable components	Pinned versions, slim base image, few dependencies
[19]	Desired state in version control, changed by reviewed commits	All configuration in Git; every push runs the same checks
[4, 5]	Automated pipelines give repeatable, low-risk delivery	GitHub Actions workflow: tests, lint, render, build, deploy to kind
[6]	Containers give a consistent environment from development to deployment	One image is built once and promoted to every environment

2.2 Existing Approaches to Configuration Management

Four approaches are common in practice. Teams copy manifests per environment; they inject settings through environment variables or files baked into images, as the Twelve-Factor method recommends [1]; they layer overlays or patches on a shared base, for example with Kustomize [28]; or they package the application as a chart whose values are layered per environment [12]. A GitOps controller can then reconcile any of these from Git [19]. Table 2.2 compares them on the criteria that matter for this project.

Table 2.2 – Comparison of configuration-management approaches

Approach	Duplication	Input validation	History and rollback	Remark
Copied manifests per environment	High: every object is repeated	None beyond the API server checks	Only through Git history	Drift is likely
Environment variables or baked-in files	Low for settings, none for structure	Application-specific	Image tags only	Does not describe Kubernetes objects
Overlays and patches on a base	Low	The rendered output can be validated	Not built in; relies on Git	No release concept
Helm chart with values layering	Low: one template set	Schema in `values.schema.json`; lint	Numbered revisions; `helm rollback` [11]	Chosen approach
GitOps controller over any of these	Depends on the source	Depends on the source	Git revert and controller history	Planned extension

2.3 Objective

The survey leads to four requirements for the proposed system: configuration must be separated from structure; every value must be validated against an explicit

contract before deployment; every deployment must be recorded so that it can be reversed; and every change must pass the same automated checks. Section 3.3 turns these requirements into the functional and non-functional requirements of the project.

2.4 Research Gaps

The empirical studies cited above analyse existing repositories and describe the problems found in them. They do not supply a small, executable reference that combines the following properties, which motivate the contribution of this project:

1. **No small, complete, reproducible reference.** A reusable chart, schema-validated values, per-environment configuration, rollback and continuous integration are rarely available together in an example that can be read in one sitting and run on a laptop.
2. **Results without a statement of how they were obtained.** Reports on deployment tooling seldom say whether a result was measured on a real cluster or only derived. This report labels every result with its evidence level (Section 4.1).
3. **Little attention to what happens during a rollout.** A rolling update is often described as “zero downtime” without measurement. The project measures the failed requests during a rollout and compares the chosen strategy with an alternative (Section 4.9).
4. **Silent failure modes.** The behaviour of a release when a prerequisite is missing, such as an image or a Secret, is rarely shown. The project provokes both failures on purpose and records the result (Section 4.8).

CHAPTER 3
PROPOSED METHODOLOGY

3.1 Existing System and Its Limitations

In the existing arrangement, each environment has its own directory of Kubernetes manifests. A Deployment, a Service and a ConfigMap are written for development and then copied, with edited values, for test, staging and production. The arrangement needs no special tooling but has four limitations. Differences between the copies are not visible as differences; a change must be repeated in every copy; nothing checks the edited values before the cluster rejects or, worse, accepts them; and there is no recorded history of what was deployed, so reversing a bad release means finding an older file.

3.2 Proposed System

The proposed system replaces the copies with one Helm chart and four small values files. The templates describe the application once; each environment file lists only the settings that differ; a JSON Schema defines the allowed values; and every deployment is a numbered release revision that can be inspected and reverted. The application reads all of its settings from environment variables, so the same container image runs in every environment. A GitHub Actions workflow repeats the validation on every change and deploys all four environments to a temporary cluster. Table 3.1 compares the two systems.

Table 3.1 – Existing and proposed systems compared

Aspect	Existing system	Proposed system
Structure	One set of manifests per environment	One set of templates for all environments
Environment differences	Hidden inside the copies	Explicit in `values-<env>.yaml` (28 to 43 lines each)
Application image	Often rebuilt or edited per environment	One image `multi-env-app:1.0.0` promoted unchanged
Validation	By the API server at deploy time	Schema, lint, render and manifest checks before deploy
Deploy command	Apply several files by hand	`helm upgrade --install` with one values file
Effort for one change	Edit and review every copy	Edit the template once
Secrets	Often written into the manifests	Never in Git; Secret created at deploy time
History and rollback	Search Git for an older file	`helm history` and `helm rollback`

3.3 System Requirements

The requirements follow from the objectives in Chapter I and from the survey in Chapter II. The functional requirements are:

- **FR1** render the same application for four named environments from one chart;
- **FR2** take every environment-specific setting from a values file, never from an edited template;
- **FR3** reject unknown environments, wrong types, out-of-range values and secrets placed in the configuration before deployment;
- **FR4** show the application name, the environment, the version and the configuration on a dashboard and as JSON;
- **FR5** install and upgrade a release with one command and wait until it is ready;
- **FR6** provide an in-cluster test and a command that lists the revisions of a release;
- **FR7** restore an earlier revision with one command, and automatically after a failed upgrade;
- **FR8** run lint, render, manifest validation and a deployment test automatically on every change;
- **FR9** keep secrets out of the repository, so that production reads its `SECRET_KEY` from a Kubernetes Secret.

The non-functional requirements are reproducibility on a laptop cluster, readable files, secure defaults, version-pinned tools and honest reporting of verification results. Table 3.2 lists the software and hardware that were used.

Table 3.2 – Software and hardware environment

Component	Version or specification	Purpose
Helm	v3.16.2 (pinned in CI and used locally)	Packaging, rendering, release management
Kubernetes	v1.35.0 on kind v0.31.0 (CI); manifests schema-checked for 1.31.0	Target platform; local cluster
Minikube	Any current version (documented, not executed)	Local cluster on a laptop
kubectl	v1.35.0 (CI)	Inspection of objects and rollouts
Docker Engine	28.0.4 (CI), 29.9 (development machine)	Image build and local runs
Python	3.12 (image, CI), 3.13 (development machine)	Flask application and tests
Git and GitHub	GitHub-hosted repository and Actions	Version control and CI
CI runner	ubuntu-latest: AMD EPYC 7763, 4 vCPU, 15 GiB memory	Runs all workflows
Laptop (Minikube)	About 2 CPU cores and 4 GB of memory for the cluster (a practical minimum, not measured)	Local demonstration

3.4 Block Diagram and Explanation

Figure 3.1 shows the end-to-end flow. The numbered explanation that follows uses the same numbering as the figure.

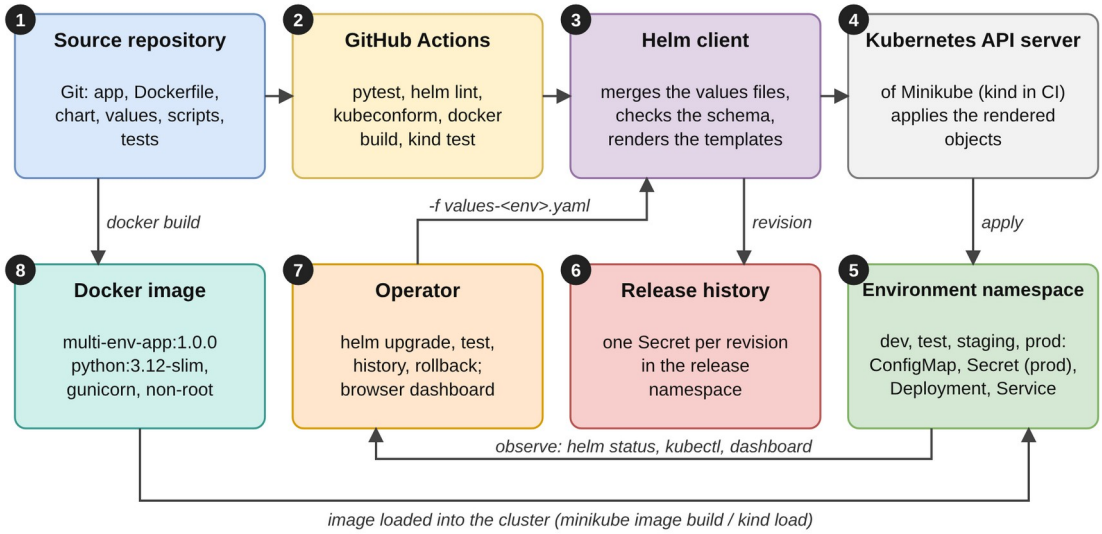


Figure 3.1 – Block diagram of the multi-environment Helm solution

Explanation:

1. **Source repository.** Git holds the Flask application in `app/`, the `Dockerfile`, the chart in `helm/multi-env-app/`, the scripts, the tests and the workflows. Every environment difference is a file in this repository.
2. **GitHub Actions.** On every push the workflow runs the tests, lints and renders the chart for the four environments, validates the rendered manifests, builds the image and deploys all four environments to a temporary kind cluster (Section 3.14).
3. **Helm client.** For one release the client merges `values.yaml` with one environment file and the command-line overrides, checks the result against `values.schema.json` and renders the templates into plain manifests.
4. **Kubernetes API server.** The API server of Minikube (or kind) receives the rendered ConfigMap, Deployment and Service and stores them; the controllers create the Pods.
5. **Environment namespace.** Each environment lives in its own namespace (`dev`, `test`, `staging`, `prod`) with its own ConfigMap, Deployment, Service and, in production, the Secret `multi-env-app-secrets`.

6. **Release history.** Helm stores every revision as a Secret of type `helm.sh/release.v1` in the namespace of the release. This is what `helm history` and `helm rollback` read.
7. **Operator.** The person running the demonstration uses `helm upgrade`, `helm test`, `helm history` and `helm rollback`, and observes the result with `kubectl` and the dashboard.
8. **Docker image.** The image is built once from the `Dockerfile` and loaded into the cluster (`minikube image build` or `kind load`); no registry is needed.

3.5 Application Design

The application is a small Flask [29] web service. It is deliberately simple: its only job is to make the configuration that Helm injected visible. Figure 3.2 shows the application inside a Pod and the sources of its configuration.

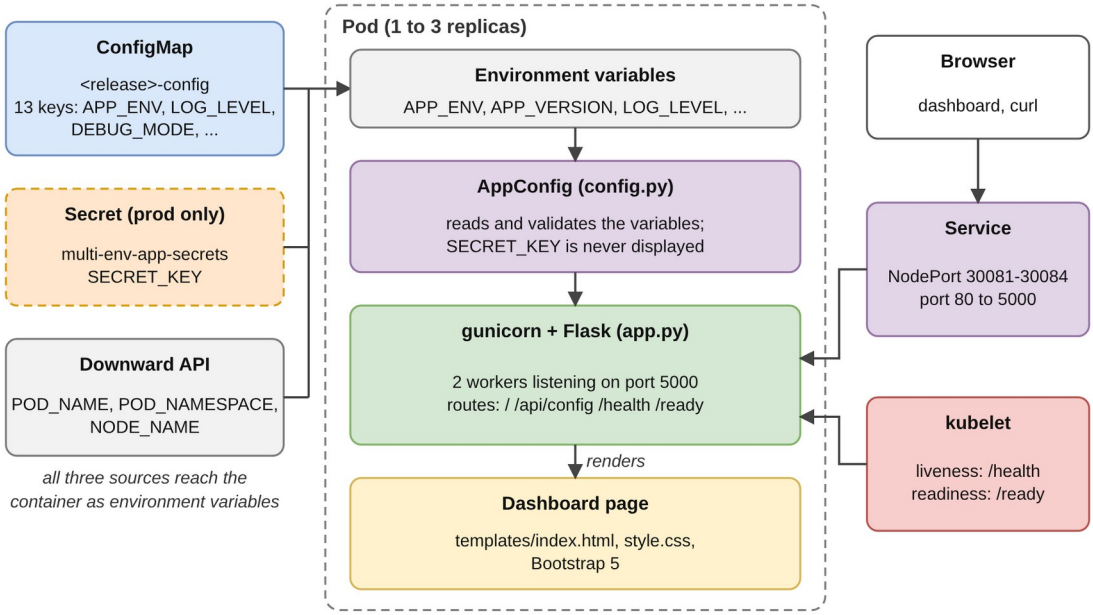


Figure 3.2 – Application inside a Pod: sources of configuration, request flow and probes

3.5.1 Routes and Endpoints

The application exposes four routes (Table 3.3). The dashboard and the JSON endpoint serve people and scripts; the two probe endpoints serve Kubernetes. All routes are read-only (HTTP GET).

Table 3.3 – Application routes and their roles

Route	Method	Purpose	Response	Used by
`/`	GET	Renders the dashboard `index.html`	HTML page, 200	Browser
`/api/config`	GET	Application name, environment, version, settings (secret masked) and runtime information	JSON, 200	curl, `helm test`, `test-environments.sh`
`/health`	GET	Liveness: the process answers	`{"status":"ok"}`, 200	kubelet liveness probe, Docker HEALTHCHECK
`/ready`	GET	Readiness: the application is ready for traffic	`{"status":"ready","environment":...}`, 200	kubelet readiness probe, availability probe

The routes are implemented in `create_app()` in `app/app.py`. The function builds the configuration object once and every route uses it, so the dashboard, the JSON endpoint and the probes can never disagree about the environment.

Code Snippet – Routes of the application (app/app.py, lines 37–63)

```
    @app.route("/")
    def dashboard():
        return render_template(
            "index.html",
            cfg=cfg,
            settings=cfg.public_settings(),
            runtime=cfg.runtime_info(),
            started_at=STARTED_AT.strftime("%Y-%m-%d %H:%M:%S UTC"),
        )

    @app.route("/health")
    def health():
        return jsonify(status="ok"), 200

    @app.route("/ready")
    def ready():
        return jsonify(status="ready", environment=cfg.environment), 200

    @app.route("/api/config")
    def api_config():
        return jsonify(
            application=cfg.app_name,
            environment=cfg.environment,
            version=cfg.app_version,
            settings=cfg.public_settings(),
            runtime=cfg.runtime_info(),
        )
```

3.5.2 Configuration Loading

`AppConfig` in `app/config.py` takes a snapshot of the environment variables when the process starts. It validates `APP_ENV` against the four allowed names, converts booleans and integers with safe defaults, and separates the values that may be shown (`public_settings()`) from the secret one. Table 3.4 lists the variables. Settings such as `MAX_CONNECTIONS` or `DATABASE_HOST` are shown by the dashboard

but not used by a database in this project; they stand for the kind of settings that differ between the environments of a real application.

Table 3.4 – Environment variables read by the application

Variable	Default in code	Set by	Effect
`APP_NAME`	Multi-Env Config Dashboard	ConfigMap, from `app.name`	Title and banner of the dashboard
`APP_ENV`	development	ConfigMap, from `environment`	Environment name and colour; an unknown value falls back to development with a warning
`APP_VERSION`	1.0.0	ConfigMap, from `app.version` or the image tag	Version card
`REPLICA_COUNT`	1	ConfigMap, from `replicaCount`	Replicas card
`HELM_RELEASE`, `HELM_CHART_VERSION`	not-deployed-with-helm, n/a	ConfigMap, from the release and the chart	Runtime panel
`LOG_LEVEL`	INFO	ConfigMap, `config.LOG_LEVEL`	Threshold of the application logger
`DEBUG_MODE`	false	ConfigMap, `config.DEBUG_MODE`	Shown on the dashboard; Flask debug mode only with `python app.py`
`FEATURE_BETA_BANNER`	false	ConfigMap, `config.FEATURE_BETA_BANNER`	Shows the beta banner
`MAX_CONNECTIONS`, `CACHE_TTL_SECONDS`, `DATABASE_HOST`, `API_BASE_URL`	10, 60, localhost, http://localhost:5000/api	ConfigMap, `config.*`	Shown on the dashboard (illustrative settings)
`SECRET_KEY`	random, per process	Secret `multi-env-app-secrets` (prod)	Flask secret key; shown only as a masked value
`POD_NAME`, `POD_NAMESPACE`, `NODE_NAME`	local	Downward API [30]	Runtime panel
`PORT`	5000	Deployment, from `containerPort`	Port of gunicorn and of `python app.py`

The class and the secret handling are shown below. If no `SECRET_KEY` is supplied, the application generates a random key for the process, so that no key is ever committed to the repository; if one is supplied it is recorded only as the *source* of the key (`kubernetes-secret` or `auto-generated`), and the dashboard and the API show this source instead of the value.

Code Snippet – Reading the settings (app/config.py, lines 39–58)

```
class AppConfig:
    """Snapshot of the application configuration taken from the environment."""

    def __init__(self, environ=None):
        env = os.environ if environ is None else environ

        self.app_name = env.get("APP_NAME", "Multi-Env Config Dashboard")
        raw_env = env.get("APP_ENV", "development").strip().lower()
        self.environment_is_valid = raw_env in VALID_ENVIRONMENTS
        self.environment = raw_env if self.environment_is_valid else
  "development"
        self.app_version = env.get("APP_VERSION", "1.0.0")

        self.log_level = env.get("LOG_LEVEL", "INFO").upper()
        self.debug = _as_bool(env.get("DEBUG_MODE"), default=False)
        self.feature_beta_banner = _as_bool(env.get("FEATURE_BETA_BANNER"))
        self.max_connections = _as_int(env.get("MAX_CONNECTIONS"), 10)
        self.cache_ttl_seconds = _as_int(env.get("CACHE_TTL_SECONDS"), 60)
        self.database_host = env.get("DATABASE_HOST", "localhost")
        self.api_base_url = env.get("API_BASE_URL", "http://localhost:5000/api")
        self.replica_count = env.get("REPLICA_COUNT", "1")
```

Code Snippet – Handling of the secret key (app/config.py, lines 67–71)

```
        # Secret key: supplied via a Kubernetes Secret, or generated per
        # process so that no secret is ever committed to source control.
        provided_secret = env.get("SECRET_KEY")
        self.secret_key_source = "kubernetes-secret" if provided_secret else
  "auto-generated"
        self.secret_key = provided_secret or secrets.token_hex(32)
```

3.5.3 Dashboard: HTML, CSS and Bootstrap

The dashboard is a Jinja2 template, `app/templates/index.html`, styled with Bootstrap 5 [31] from a CDN and a short `style.css`. It is rendered on the server, so the page needs no JavaScript. Figure 3.3 labels its parts. The banner and the cards take their Bootstrap colour from the environment (green for development, cyan for testing, yellow for staging, red with a striped background for production), so that the audience can tell at a glance which environment is shown. The configuration table is the content of the ConfigMap as the application sees it, and the runtime panel proves that the page was served by a Pod of a Helm release.

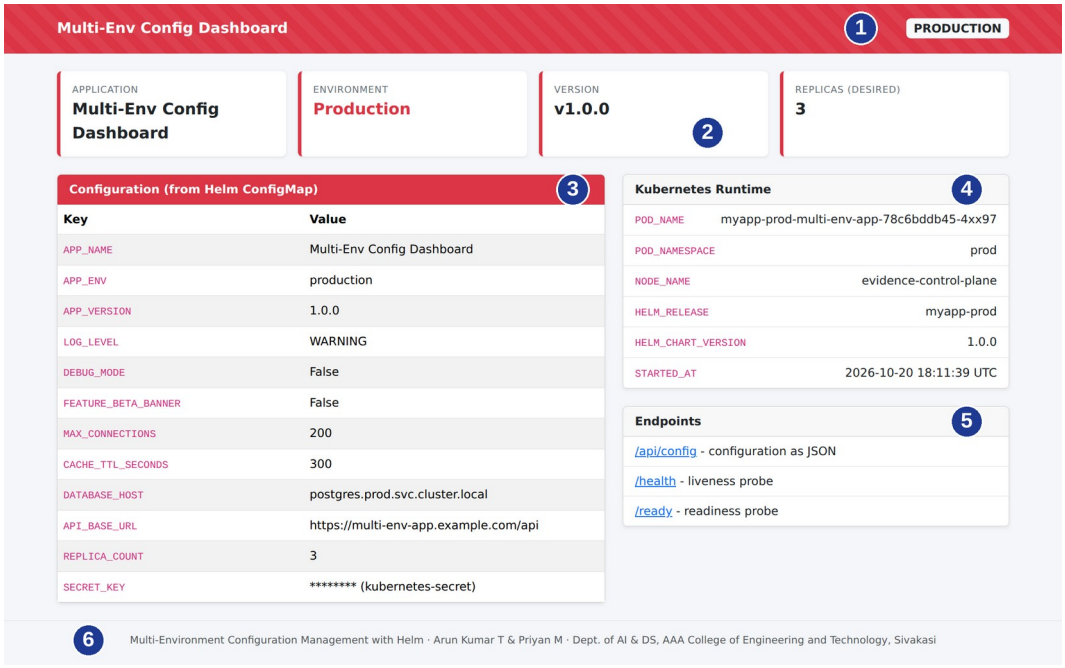


Figure 3.3 – Parts of the dashboard (production release)

1. **Navigation bar** with the application name and the badge of the environment; its colour is chosen by the environment.
2. **Summary cards** for the application name, the environment, the version and the desired number of replicas.
3. **Configuration table**: the settings from the ConfigMap as the application sees them, with the secret key masked.
4. **Kubernetes runtime panel**: Pod name, namespace, node, Helm release, chart version and start time of the process.
5. **Endpoints panel** with links to `/api/config`, `/health` and `/ready`.
6. **Footer** with the title of the project, the students and the institution.

Code Snippet – Environment colour in the template (app/templates/index.html, lines 10–16)

```
<body class="env-{{ cfg.environment }}">
  <nav class="navbar navbar-dark bg-{{ cfg.theme }} env-banner">
    <div class="container">
      <span class="navbar-brand fw-bold">{{ cfg.app_name }}</span>
      <span class="badge bg-light text-dark fs-6" id="env-
 badge">{{ cfg.environment|upper }}</span>
    </div>
  </nav>
```

The class `bg-{{ cfg.theme }}` in the navigation bar is the whole mechanism: `cfg.theme` is `success`, `info`, `warning` or `danger` according to the environment, which `config.py` defines in `ENVIRONMENT_THEMES` (lines 14–20).

3.5.4 Health Probes and Logging

Kubernetes calls `/health` for the liveness probe and `/ready` for the readiness probe [25]. A container whose liveness probe fails is restarted; a Pod whose readiness probe fails receives no traffic from its Service. The application configures the Python logger with the level taken from `LOG_LEVEL` (lines 28–35 of `app/app.py`), so the effect of the setting is visible in the logs: a development Pod prints the start-up message at level INFO, a production Pod with level WARNING does not (Section 4.3).

3.6 Containerisation

The `Dockerfile` (Table 3.5, Figure 3.4) builds one image that is used in all four environments. Dependencies are installed before the application code is copied, so that Docker can reuse the cached dependency layer when only the code changes [32]. The container runs as the unprivileged user 10001, which matches the `securityContext` of the chart, and a HEALTHCHECK instruction lets Docker report the state of the container when it runs outside Kubernetes. The application runs under gunicorn [33] with two worker processes instead of the Flask development server.

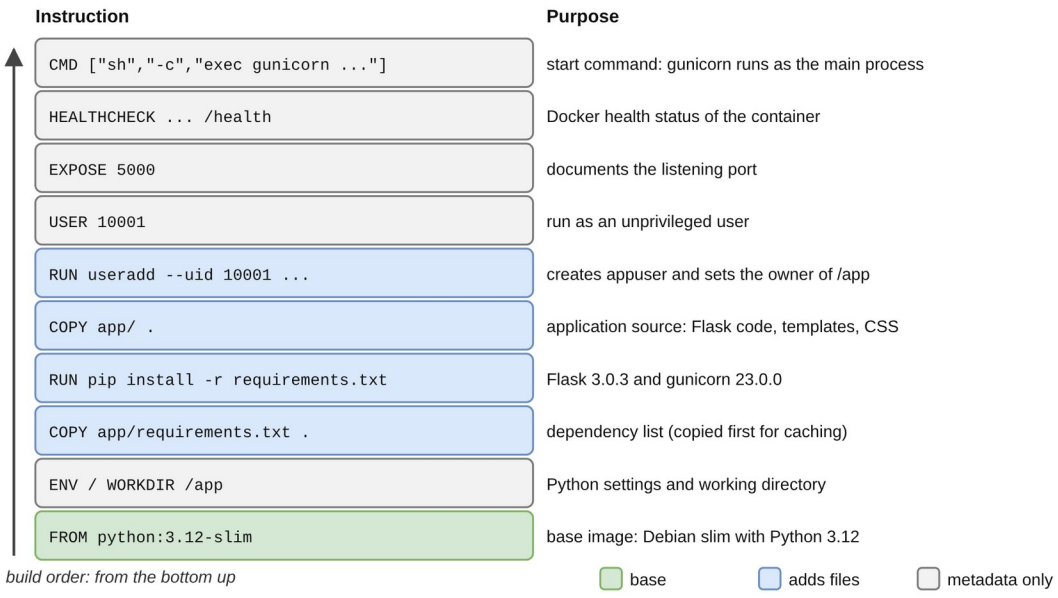


Figure 3.4 – Layered structure of the application image

Table 3.5 – Instructions of the Dockerfile and their purpose

Instruction	Purpose and reason
`FROM python:3.12-slim`	Base image with Python 3.12 and few extra packages; smaller than the full image
`ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 PORT=5000`	No `.pyc` files, unbuffered logs (visible at once in `kubectl logs`), no pip cache, default port
`WORKDIR /app`	Working directory of the application
`COPY app/requirements.txt .` and `RUN pip install ...`	Dependencies first, for layer caching: Flask 3.0.3 and gunicorn 23.0.0
`COPY app/ .`	Application code, templates and static files
`RUN useradd --uid 10001 ...` and `USER 10001`	Runs as an unprivileged user; the UID equals `runAsUser` in the chart
`EXPOSE 5000`	Documents the listening port
`HEALTHCHECK ... /health`	Docker marks the container healthy or unhealthy (interval 30 s, timeout 3 s, 3 retries)
`CMD ["sh", "-c", "exec gunicorn ..."]`	`exec` makes gunicorn the main process, so it receives the termination signal of Kubernetes

Code Snippet – Unprivileged user, health check and start command (Dockerfile, lines 21–31)

```
# Run as an unprivileged user (UID matches the Helm securityContext).
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser \
    && chown -R appuser /app
USER 10001

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request,sys; sys.exit(0 if
   urllib.request.urlopen('http://127.0.0.1:5000/health').status == 200 else 1)"

CMD ["sh", "-c", "exec gunicorn --bind 0.0.0.0:${PORT} --workers 2 --access-
   logfile - app:app"]
```

3.7 Helm Chart Design

3.7.1 Chart Structure and Metadata

The chart `multi-env-app` uses `apiVersion: v2`, chart version 1.0.0 and application version 1.0.0, which is the default image tag and the default version shown by the dashboard [10]. Table 3.6 lists the files and their roles; the complete directory tree is printed at the start of the Appendices.

Table 3.6 – Files of the Helm chart and their roles

File	Role
`Chart.yaml`	Name, chart version, application version, keywords, maintainers
`values.yaml`	Defaults shared by all environments (43 settings)
`values-dev.yaml`, `values-test.yaml`, `values-staging.yaml`, `values-prod.yaml`	Overrides: only the settings that differ in that environment

File	Role
`values.schema.json`	JSON Schema (draft-07) that Helm checks on lint, template, install and upgrade
`templates/_helpers.tpl`	Shared names, labels, image tag and application version
`templates/configmap.yaml`	The settings of the environment as environment variables
`templates/deployment.yaml`	The Pods: image, probes, resources, rolling update, security context
`templates/service.yaml`	Stable network address; NodePort in the local environments
`templates/tests/test-connection.yaml`	Pod run by `helm test`
`templates/NOTES.txt`, `.helmignore`	Message printed after an install; files excluded from the package

3.7.2 Helper Templates and Naming

Names and labels are produced once in `_helpers.tpl` and included by every template, so that no object can carry a different label set. The full name of an object is the release name; if the release name does not already contain the chart name, the chart name is appended. With the release names `myapp-dev`, `myapp-test`, `myapp-staging` and `myapp-prod` the objects are named `myapp-dev-multi-env-app` and so on. Two label sets are defined: the *selector labels* (`app.kubernetes.io/name` and `app.kubernetes.io/instance`), which must not change between upgrades because the selector of a Deployment is immutable, and the *common labels*, which add the chart, the application version, the manager and an environment label.

Code Snippet – Selector and common labels (templates/_helpers.tpl, lines 37–48)

```
{{- define "multi-env-app.selectorLabels" -}}
app.kubernetes.io/name: {{ include "multi-env-app.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{- define "multi-env-app.labels" -}}
helm.sh/chart: {{ include "multi-env-app.chart" . }}
{{ include "multi-env-app.selectorLabels" . }}
app.kubernetes.io/version: {{ include "multi-env-app.appVersion" . | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
app.kubernetes.io/environment: {{ .Values.environment }}
{{- end }}
```

3.8 Values and Environment-Specific Configuration

3.8.1 Layering of Values

Helm builds the values seen by the templates in layers. The chart's `values.yaml` has the lowest precedence, files passed with `-f` override it in the order given, and `--set` has the highest precedence [12]. Maps are merged key by key, so an environment file mentions only what differs. Figure 3.5 shows the sequence, which

ends in the schema check. The deployment script passes exactly one environment file, so `-f values-prod.yaml` is the only thing that distinguishes a production release from a development release.

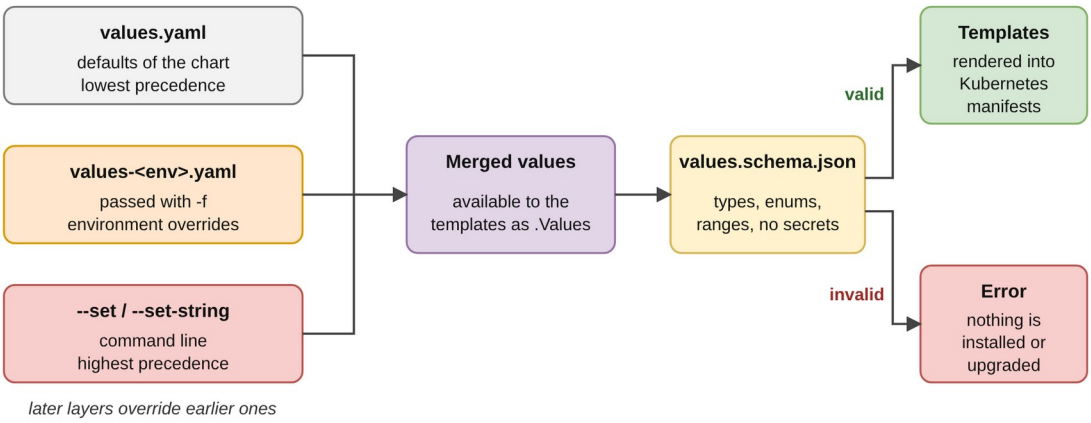


Figure 3.5 – Values layering and schema validation

Table 3.7 resolves six values for a staging release that is started with `--set replicaCount=3 --set config.LOG_LEVEL=DEBUG`. The last column was produced with `helm template` and is not a prediction. The command line wins over the file, the file wins over the defaults, and a value that no layer overrides keeps its default.

Table 3.7 – Worked example: effective values of a staging release with two command-line overrides

Value	values.yaml	values-staging.yaml	Command line	Effective value
`replicaCount`	1	2	3	3 (command line)
`config.LOG_LEVEL`	INFO	INFO	DEBUG	DEBUG (command line)
`config.MAX_CONNECTIONS`	10	50	none	50 (file)
`config.FEATURE_BETA_BANNER`	false	false	none	false
`service.nodePort`	empty	30083	none	30083 (file)
`image.tag`	empty	not set	none	empty, so `1.0.0` from `appVersion`

Numbers that are set with `--set` become integers, but the settings under `config` are strings because they end up in a ConfigMap. The schema therefore rejects `--set config.CACHE_TTL_SECONDS=130`; the correct form is `--set-string`. This is deliberate: it keeps the type of every value explicit (Helm also recommends clearly typed values [34]).

3.8.2 Configuration of the Four Environments

Table 3.8 lists the settings that differ between the environments; they are read from the values files by the build of this report. Development is the smallest and most verbose: one replica, debug logging and both feature flags on. Testing uses information-level logs and the beta banner so that testers see the same features as developers. Staging has the feature flags of production, two replicas and larger limits. Production has three replicas, warning-level logs, the largest limits and its secret key from a Kubernetes Secret. The values are design choices for demonstration; a real project would tune them from measurements.

Table 3.8 – Environment-specific configuration (values that differ)

Setting	dev	test	staging	prod
Release / namespace	myapp-dev / dev	myapp-test / test	myapp-staging / staging	myapp-prod / prod
`environment`	development	testing	staging	production
Replicas	1	1	2	3
`LOG_LEVEL`	DEBUG	INFO	INFO	WARNING
`DEBUG_MODE`	true	false	false	false
`FEATURE_BETA_BANNER`	true	true	false	false
`MAX_CONNECTIONS`	5	10	50	200
`CACHE_TTL_SECONDS`	10	30	120	300
CPU request / limit	25m / 100m	50m / 150m	100m / 250m	100m / 500m
Memory request / limit	64Mi / 128Mi	64Mi / 128Mi	96Mi / 192Mi	128Mi / 256Mi
NodePort	30081	30082	30083	30084
Secret	none (key generated)	none (key generated)	none (key generated)	multi-env-app-secrets
Lines in the values file	28	28	28	43

3.8.3 Schema Validation of Values

`values.schema.json` is a JSON Schema (draft-07) [35] that Helm checks against the merged values whenever a chart is installed, upgraded, linted or rendered [10]. It restricts `environment` to the four environment names, `replicaCount` to the range 1 to 10, `LOG_LEVEL` to the five standard levels, the two flags to the strings `true` and `false`, the two numeric settings to digits, `service.type` to three values and `service.nodePort` to the NodePort range 30000–32767. It requires the top-level keys `environment`, `replicaCount`, `image`, `app`, `config` and `service`, and it forbids the keys `SECRET_KEY`, `PASSWORD`, `DB_PASSWORD` and `API_TOKEN` inside `config`, so that a secret cannot end up in a ConfigMap by mistake. Table 3.9 summarises the rules; the part that handles `config` is quoted below.

Table 3.9 – Rules of values.schema.json

Value	Rule	Purpose
`environment`	One of development, testing, staging, production	A misspelt or unknown environment cannot be deployed
`replicaCount`	Integer, 1 to 10	No empty or runaway Deployments
`image.repository`, `app.name`, `tests.image`	Non-empty strings	No empty image reference or title
`image.pullPolicy`	Always, IfNotPresent or Never	Valid Kubernetes policies only
`config.LOG_LEVEL`	DEBUG, INFO, WARNING, ERROR or CRITICAL	Levels the application understands
`config.DEBUG_MODE`, `FEATURE_BETA_BANNER`	The string `true` or `false`	ConfigMap values are strings
`config.MAX_CONNECTIONS`, `CACHE_TTL_SECONDS`	String of digits	Numbers that the application can parse
Keys of `config`	Upper-case letters, digits and underscores	Valid environment-variable names
`SECRET_KEY`, `PASSWORD`, `DB_PASSWORD`, `API_TOKEN` in `config`	Forbidden	No secrets in a ConfigMap or in Git
`containerPort`, `service.port`	Integer, 1 to 65535	Valid ports
`service.type`	ClusterIP, NodePort or LoadBalancer	Valid Service types
`service.nodePort`	Empty, or integer 30000 to 32767	Valid NodePort range

Code Snippet – Schema rules for config (values.schema.json, lines 34–53)

```
    "config": {
      "type": "object",
      "required": ["LOG_LEVEL", "DEBUG_MODE"],
      "properties": {
        "LOG_LEVEL": { "type": "string", "enum": ["DEBUG", "INFO", "WARNING",
  "ERROR", "CRITICAL"] },
        "DEBUG_MODE": { "type": "string", "enum": ["true", "false"] },
        "FEATURE_BETA_BANNER": { "type": "string", "enum": ["true", "false"] },
        "MAX_CONNECTIONS": { "type": "string", "pattern": "^[0-9]+$" },
        "CACHE_TTL_SECONDS": { "type": "string", "pattern": "^[0-9]+$" }
      },
      "propertyNames": { "pattern": "^[A-Z][A-Z0-9_]*$" },
      "not": {
        "anyOf": [
          { "required": ["SECRET_KEY"] },
          { "required": ["PASSWORD"] },
          { "required": ["DB_PASSWORD"] },
          { "required": ["API_TOKEN"] }
        ]
      }
    },
```

3.9 Helm Templates

The chart contains three templates that create Kubernetes objects (ConfigMap, Deployment and Service), one template that creates the pod used by `helm test`, and a text template that prints a message after an installation. Every template reads its settings from `.Values`, so no environment name or number appears in a template. Names and labels come from the helpers of Section 3.7.2.

3.9.1 ConfigMap Template

The ConfigMap is the bridge between Helm values and the application. Six keys are fixed (`APP_NAME`, `APP_ENV`, `APP_VERSION`, `REPLICA_COUNT`, `HELM_RELEASE` and `HELM_CHART_VERSION`); the remaining keys are generated with a `range` loop over the `config` map, and every value is converted to a quoted string. A new setting therefore needs no template change: it is added to `config` in `values.yaml` and to the environment files, and it appears as an environment variable. For the production release the ConfigMap has 13 keys, which are shown in Section 4.2.

Code Snippet – ConfigMap template (templates/configmap.yaml, lines 7–16)

```
data:
  APP_NAME: {{ .Values.app.name | quote }}
  APP_ENV: {{ .Values.environment | quote }}
  APP_VERSION: {{ include "multi-env-app.appVersion" . | quote }}
  REPLICA_COUNT: {{ .Values.replicaCount | quote }}
  HELM_RELEASE: {{ .Release.Name | quote }}
  HELM_CHART_VERSION: {{ .Chart.Version | quote }}
  {{- range $key, $value := .Values.config }}
  {{ $key }}: {{ $value | toString | quote }}
  {{- end }}
```

3.9.2 Deployment Template

The Deployment describes the Pods. Table 3.10 lists the values that it consumes and the Kubernetes fields that they fill. Four aspects deserve explanation.

Table 3.10 – Values consumed by the Deployment template

Value	Kubernetes field	Value used in this project
`replicaCount`	`spec.replicas`	1, 1, 2, 3 (dev, test, staging, prod)
`strategy`	`spec.strategy`	RollingUpdate, maxSurge 1, maxUnavailable 0
`image.repository`, `image.tag` or `appVersion`	`containers[0].image`	`multi-env-app:1.0.0`
`image.pullPolicy`	`imagePullPolicy`	IfNotPresent
`podSecurityContext`	Pod `securityContext`	runAsNonRoot, runAsUser 10001, fsGroup 10001
`securityContext`	Container `securityContext`	allowPrivilegeEscalation false, capabilities drop ALL
`containerPort`	`ports[0].containerPort` and the variable `PORT`	5000
`existingSecret`	Second entry of `envFrom` (only when not empty)	`multi-env-app-secrets` in production
`livenessProbe`, `readinessProbe`	Probes	`/health` every 10 s, `/ready` every 5 s

Value	Kubernetes field	Value used in this project
`resources`	Requests and limits	Differs per environment (Table 3.8)
`podAnnotations`	Additional Pod annotations	Empty

Automatic restart on a configuration change. Environment variables that come from a ConfigMap are read when a container starts, so changing the ConfigMap alone does not change running Pods. The template therefore stores the SHA-256 checksum of the rendered ConfigMap in the annotation `checksum/config` of the Pod template. When a value in `config` changes, the checksum changes, the Pod template changes, and the Deployment starts a rolling update. The pytest case `test_config_change_changes_checksum` verifies this behaviour.

Code Snippet – Checksum annotation (templates/deployment.yaml, lines 21–23)

```
    annotations:
      # Roll the pods automatically whenever the ConfigMap content changes.
      checksum/config: {{ include (print $.Template.BasePath
"/configmap.yaml") . | sha256sum }}
```

Configuration injection. The container receives the ConfigMap with `envFrom`, so every key becomes an environment variable. If `existingSecret` is set, a `secretRef` is added to the same list. The variables `POD_NAME`, `POD_NAMESPACE` and `NODE_NAME` are filled by the Downward API [30], which is how the dashboard can show the Pod that served the page.

Code Snippet – Injection of configuration and Pod information (templates/deployment.yaml, lines 44–65)

```
        envFrom:
          - configMapRef:
              name: {{ include "multi-env-app.fullname" . }}-config
          {{- if .Values.existingSecret }}
          - secretRef:
              name: {{ .Values.existingSecret }}
          {{- end }}
        env:
          - name: PORT
            value: {{ .Values.containerPort | quote }}
          - name: POD_NAME
            valueFrom:
              fieldRef:
                fieldPath: metadata.name
          - name: POD_NAMESPACE
            valueFrom:
              fieldRef:
                fieldPath: metadata.namespace
          - name: NODE_NAME
            valueFrom:
              fieldRef:
                fieldPath: spec.nodeName
```

Probes and resources. The probe definitions and the resource requests and limits are copied from the values with `toYaml`, so an environment can change them without a template change. Table 3.11 lists the probe settings as the cluster reports them in Section 4.2.

Table 3.11 – Probe settings of the chart

Probe	Request	Initial delay	Period	Timeout	Failure threshold	Effect of failure
Liveness	GET `/health`	5 s	10 s	1 s	3	The container is restarted
Readiness	GET `/ready`	3 s	5 s	1 s	3	The Pod is removed from the Service endpoints

3.9.3 Rolling-Update Strategy

The strategy is a value, not a constant: `values.yaml` sets `RollingUpdate` with `maxSurge: 1` and `maxUnavailable: 0`. For a Deployment of N replicas, the controller keeps the number of available Pods A(t) and the total number of Pods P(t) within the bounds Equation (3.1) and Equation (3.2) during an update [9], where *s* is `maxSurge` and *u* is `maxUnavailable`.

$$N - u \leq A(t) \tag{3.1}$$

$$P(t) \leq N + s \tag{3.2}$$

With $u = 0$ and $s = 1$ the controller first starts one additional Pod and removes an old Pod only after a new one is ready, so A(t) never falls below N and P(t) never exceeds N + 1. In production (N = 3) this means at least three available and at most four Pods at any moment. The alternative $s = 0$ and $u = 1$ keeps P(t) at N but allows A(t) to fall to N − 1, which is zero for a single replica. Section 4.9 measures what these bounds mean for the requests of a client. The bounds describe Pods, not requests: they do not say how quickly a terminating Pod disappears from the list of Service endpoints.

3.9.4 Service Template

The Service gives the Pods of one release a stable address. Its selector uses the selector labels of the helpers, so a Service selects only the Pods of its own release even when several environments share a cluster. The target port is the *name* `http`, which the Deployment gives to the container port, so a change of `containerPort` needs no change in the Service. A fixed `nodePort` is rendered only for the type NodePort and only when a value is given; the four environments use 30081 to 30084 so that every dashboard has a predictable address on a laptop cluster [26].

Code Snippet – Service template (templates/service.yaml, lines 7–18)

```
spec:
  type: {{ .Values.service.type }}
  selector:
    {{- include "multi-env-app.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      {{- if and (eq .Values.service.type
   "NodePort") .Values.service.nodePort }}
      nodePort: {{ .Values.service.nodePort }}
      {{- end }}
```

3.9.5 Helm Test and Release Notes

`templates/tests/test-connection.yaml` defines a Pod with the annotation `helm.sh/hook: test`. It is not created by an install or an upgrade; it is created when `helm test <release>` is run [20]. The Pod uses a `busybox` image, requests `/health` and `/api/config` from the Service of its own release, and checks that the JSON contains the environment name of the release. A production release therefore fails the test if it answers with another environment. The policy `before-hook-creation` deletes the Pod of the previous test run before a new one is created.

Code Snippet – Chart test (templates/tests/test-connection.yaml, lines 9–27)

```
  annotations:
    "helm.sh/hook": test
    "helm.sh/hook-delete-policy": before-hook-creation
spec:
  restartPolicy: Never
  containers:
    - name: check-environment
      image: {{ .Values.tests.image }}
      imagePullPolicy: IfNotPresent
      command: ["sh", "-c"]
      args:
        - |
          set -e
          URL="http://{{ include "multi-env-app.fullname" . }}:
   {{ .Values.service.port }}"
          wget -qO- "$URL/health"
          BODY="$(wget -qO- "$URL/api/config")"
          echo "$BODY"
          echo "$BODY" | grep -Eq '"environment": ?"{{ .Values.environment }}"'
          echo "OK: service reachable and environment is
   {{ .Values.environment }}"
```

`templates/NOTES.txt` is printed after every install and upgrade. It states the release, revision, namespace, environment, version and replica count, and it prints the commands to open the dashboard and to run the test (Section 4.3 shows the output).

3.9.6 Rendered Objects of the Four Environments

`helm template` renders four objects for every environment: the ConfigMap, the Service, the Deployment and the test Pod. Table 3.12 gives the size of the rendered manifests and the number of values that an environment changes with respect to the defaults. The numbers were computed from the rendered output and from the values files; they are used in Section 4.5.

Table 3.12 – Rendered manifests and size of the environment files

Environment	Rendered lines	Objects	Values that differ from `values.yaml`	Lines in the values file
dev	177	4	11	28
test	177	4	8	28
staging	177	4	12	28
prod	179	4	13	43

Sixteen different settings are changed by at least one environment; 153 of the rendered lines are identical in all four environments. The production manifest is two lines longer because of the `secretRef` *entry.*

3.10 Kubernetes Topology and Objects

Figure 3.6 shows the cluster as it is deployed by `scripts/deploy.sh all`. One cluster hosts four namespaces [36]; each namespace contains one release with its ConfigMap, Deployment, Service and Pods, and the production namespace additionally contains the Secret `multi-env-app-secrets`. Because all objects of a release carry the labels `app.kubernetes.io/instance` and `app.kubernetes.io/name`, one label selector finds everything that belongs to a release.

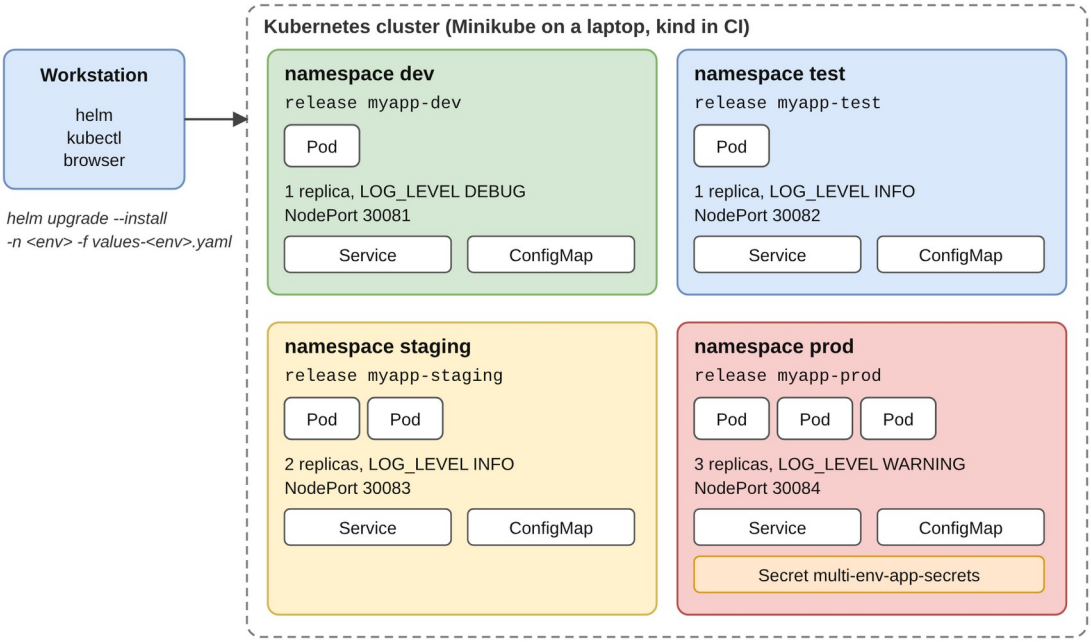


Figure 3.6 – Cluster topology: one namespace and one Helm release per environment

Figure 3.7 shows how the objects of one release refer to each other, and Table 3.13 lists the object kinds, their names and who creates them. The Deployment creates one ReplicaSet for every distinct Pod template; a configuration change or an image change creates a new ReplicaSet, and the old one is scaled down. Helm itself stores every revision of a release as a Secret of the type `helm.sh/release.v1` in the namespace of the release [10].

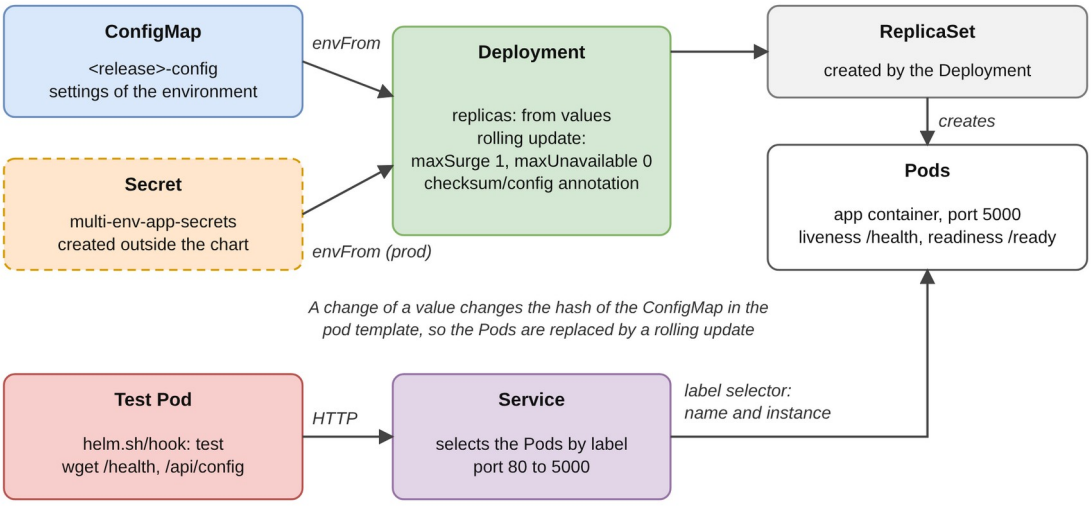


Figure 3.7 – Objects of one release and the references between them

Table 3.13 – Kubernetes objects of one release

Object	Name (example for production)	Created by	Purpose
Namespace	`prod`	`deploy.sh`	Isolates the environment
ConfigMap	`myapp-prod-multi-env-app-config`	Chart	Settings as environment variables
Secret	`multi-env-app-secrets`	`deploy.sh` (production only)	`SECRET_KEY`; never stored in Git
Deployment	`myapp-prod-multi-env-app`	Chart	Desired state of the Pods and the update strategy
ReplicaSet	`myapp-prod-multi-env-app-78c6bddb45`	Deployment controller	One per Pod template; owns the Pods
Pod	`myapp-prod-multi-env-app-78c6bddb45-4xx97`	ReplicaSet	Runs the application container
Service	`myapp-prod-multi-env-app`	Chart	Stable address, NodePort 30084
Release Secret	`sh.helm.release.v1.myapp-prod.v1`	Helm	Stores one revision; read by `helm rollback`
Test Pod	`myapp-prod-multi-env-app-test-connection`	Chart (hook)	Created by `helm test`

3.11 Release Management with Helm

A *release* is one installation of a chart with one set of values. The project names the releases `myapp-<env>` and installs each into the namespace `<env>`. Figure 3.8 shows the states of a release. `helm upgrade --install` creates the release if it does not exist and upgrades it otherwise, so one command serves both cases; every change creates a new numbered revision, and `helm rollback` creates yet another revision that has the content of an earlier one [11].

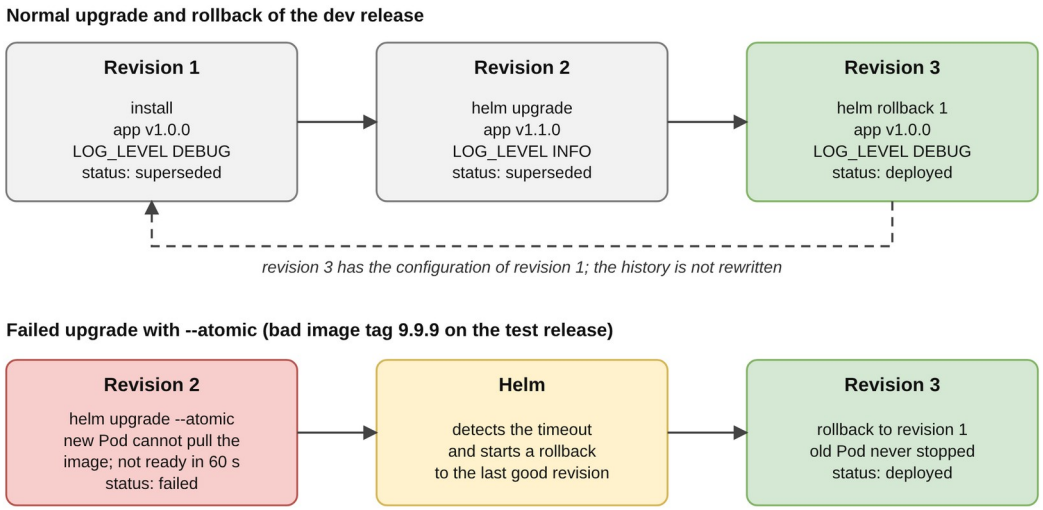


Figure 3.8 – Lifecycle of a Helm release

Table 3.14 lists the commands that the project uses. Two options matter for safety. `--wait` makes Helm return only after the Pods are ready, and `--atomic`

implies `--wait` and rolls the release back automatically if the upgrade does not become ready within the timeout [22]. The upgrade script does not use `--reuse-values`, because the values file of the environment should always be the source of truth; every upgrade repeats `-f values-<env>.yaml` and adds only the intended `--set` overrides.

Table 3.14 – Helm commands used in the project

Command	Purpose	Used by
`helm lint ./helm/multi-env-app --strict -f values-dev.yaml`	Static checks; enforces `values.schema.json`; warnings are errors	`validate-helm.sh`, CI, pytest
`helm template myapp-dev ./helm/multi-env-app -f values-dev.yaml -n dev`	Renders the manifests without a cluster	`validate-helm.sh`, pytest
`helm upgrade --install myapp-dev ./helm/multi-env-app -n dev -f values-dev.yaml --atomic --wait --timeout 5m`	Installs or upgrades a release and waits until it is ready	`deploy.sh`
`helm test myapp-dev -n dev`	Runs the test Pod of the release	`test-environments.sh`
`helm history myapp-dev -n dev`	Lists the revisions of a release	`upgrade-rollback-demo.sh`
`helm get values myapp-dev -n dev [--all]`	Shows the supplied (or all computed) values	Evidence capture
`helm rollback myapp-dev 1 -n dev --wait`	Creates a new revision with the content of revision 1	`upgrade-rollback-demo.sh`
`helm list -A`	Lists the releases of all namespaces	Evidence capture, CI
`helm uninstall myapp-dev -n dev`	Removes a release and its objects	`cleanup.sh`

Figure 3.9 shows the sequence of a deployment from the command to the running Pod. The client renders and validates the chart before it contacts the cluster, so an invalid value never reaches the API server.

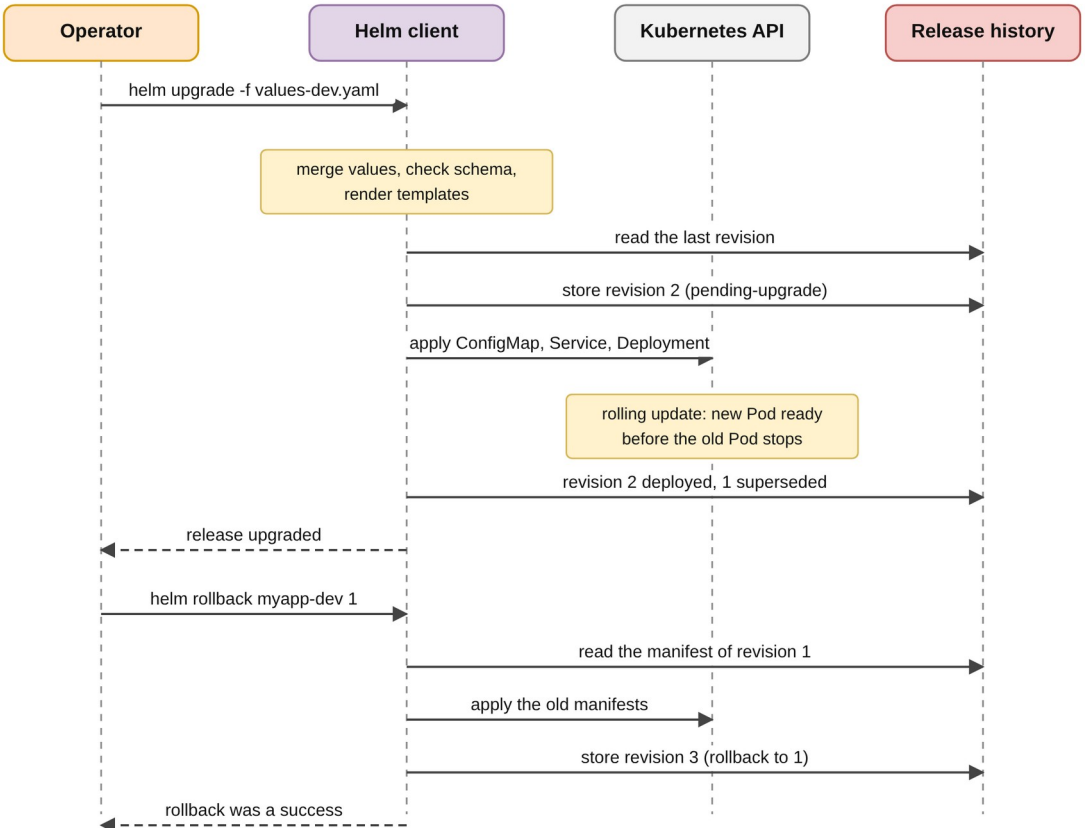


Figure 3.9 – Sequence of a deployment with `helm upgrade --install --atomic --wait`

Code Snippet – Deployment command (scripts/deploy.sh, lines 36–40)

```
  helm upgrade --install "$release" "$CHART_DIR" \
    --namespace "$ns" \
    -f "$values" \
    --atomic --wait --timeout 5m \
    "$@"
```

The production Secret is created by the script and not by the chart. `ensure_prod_secret` creates `multi-env-app-secrets` with a random 64-character key generated from `/dev/urandom` only if the Secret does not yet exist, so that the key is never written to Git, and a repeated deployment does not rotate it.

Code Snippet – Creation of the production Secret (scripts/deploy.sh, lines 13–22)

```
ensure_prod_secret() {
  local ns="$1"
  if ! kubectl get secret "${SECRET_NAME}" -n "$ns" >/dev/null 2>&1; then
    log "Creating Secret ${SECRET_NAME} in ${ns} with a random SECRET_KEY"
    # Generated at deploy time - never stored in Git.
    local key
    key="$(head -c 32 /dev/urandom | od -An -tx1 | tr -d ' \n')"
    kubectl create secret generic "${SECRET_NAME}" -n "$ns" --from-
   literal=SECRET_KEY="${key}"
  fi
}
```

3.12 Automation Scripts

Nine shell scripts in `scripts/` automate the work. They share the settings in `common.sh` (the namespace, values file, release name, expected `APP_ENV` and NodePort of each environment), they stop at the first error (`set -euo pipefail`), and they check that the tools they need are installed. Table 3.15 lists them.

Table 3.15 – Scripts of the project

Script	Lines	Purpose
`common.sh`	43	Shared settings and helper functions; sourced by the other scripts
`minikube-setup.sh`	25	Starts Minikube (2 CPUs, 4096 MB), builds the image inside it and creates the four namespaces
`deploy.sh <env|all>`	49	Creates the namespace and the production Secret and runs `helm upgrade --install`
`validate-helm.sh`	45	Lint, render and kubeconform check of all four environments, without a cluster
`test-environments.sh`	68	26 live checks of the deployed releases, including `helm test`
`upgrade-rollback-demo.sh`	50	Upgrade to version 1.1.0, show the history, roll back, verify
`docker-demo.sh`	63	Runs the four environments as plain Docker containers with the same settings (fallback without Kubernetes)
`cleanup.sh`	12	Uninstalls the releases and deletes the namespaces
`capture-evidence.sh`	238	Records the evidence of this report on a cluster (Section 4.1)

A complete session on a laptop uses the scripts in the order shown below. `validate-helm.sh` needs no cluster and can be run first; `test-environments.sh` fails with a non-zero exit status if any check fails, which is what makes it usable in the CI workflow.

Code Snippet – Typical session (scripts/ directory of the repository)

```
./scripts/minikube-setup.sh              # cluster, image, namespaces
./scripts/validate-helm.sh               # lint, render, kubeconform
./scripts/deploy.sh all                  # dev, test, staging, prod
./scripts/test-environments.sh all       # 26 live checks
./scripts/upgrade-rollback-demo.sh dev   # upgrade, history, rollback
./scripts/cleanup.sh                     # remove everything
```

Code Snippet – Validation loop (scripts/validate-helm.sh, lines 16–29)

```
for env in "${ALL_ENVS[@]}"; do
  values="$(values_for "$env")"
  log "[$env] helm lint --strict"
  helm lint "$CHART_DIR" --strict -f "$values" || failures=$((failures + 1))

  log "[$env] helm template"
  out="${OUT_DIR}/${env}.yaml"
  if helm template "$(release_for "$env")" "$CHART_DIR" -f "$values" -n "$
   (namespace_for "$env")" > "$out"; then
    grep -q "APP_ENV: \"$(app_env_for "$env")\"" "$out" \
      && echo "APP_ENV correctly rendered as $(app_env_for "$env")" \
      || { echo "APP_ENV mismatch in $out"; failures=$((failures + 1)); }
  else
    failures=$((failures + 1))
  fi
```

3.13 Testing Strategy

The project is tested at four levels (Table 3.16). The lowest two levels run without a cluster and give feedback in seconds; the upper two need a cluster and show that the rendered objects behave as intended. All 62 pytest cases [37] pass on the development machine and in the CI workflow, and the 26 live checks pass on the kind cluster.

Table 3.16 – Test levels

Level	Tool and file	Cases	Examples	Needs
1. Application	pytest, `test_app.py`	15	Dashboard shows name, environment and version; `/health` and `/ready` answer; the secret is masked and never returned; an unknown environment falls back to development; bad numbers use defaults	Python
2. Static configuration	pytest, `test_helm_config.py` (part 1)	32	Required files exist; chart metadata; replica policy; same `config` keys in every environment; unique NodePorts in range; production hardening; no secret-like key has a value	Python
3. Chart behaviour	pytest, `test_helm_config.py` (part 2)	15	`helm lint --strict` for four environments; rendered ConfigMap, Deployment and Service match the values; production mounts its Secret; a config change changes the checksum; five invalid values are rejected	Helm
4. Live cluster	`test-environments.sh`, `helm test`	26	Release deployed; replicas ready; `APP_ENV` and `REPLICA_COUNT` as expected; `/health` ok; production `DEBUG_MODE` false and `SECRET_KEY` from a Secret; the test Pod of each release succeeds	Cluster

Four of the 26 live checks are the `helm test` *runs of the four releases; they exercise the chart's own test Pod.*

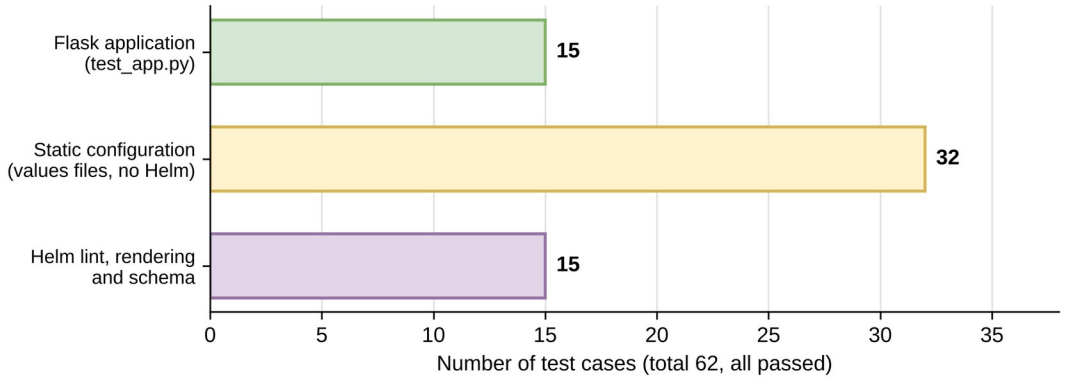


Figure 3.10 – Distribution of the 62 pytest cases over the three automated levels

Two of the test functions show the style of the suite. `test_production_hardening` states the rules for production as assertions, so that a later edit of `values-prod.yaml` that enables debug mode or removes the Secret fails the test run before it reaches a cluster. `test_config_change_changes_checksum` renders the chart twice and compares the checksum annotation, which proves the automatic restart of Section 3.9.2.

Code Snippet – Production rules as a test (tests/test_helm_config.py, lines 106–113)

```
def test_production_hardening():
    prod = merged_values("prod")
    assert prod["config"]["DEBUG_MODE"] == "false"
    assert prod["config"]["LOG_LEVEL"] in ("WARNING", "ERROR")
    assert prod["config"]["FEATURE_BETA_BANNER"] == "false"
    assert prod["existingSecret"], "production must load SECRET_KEY from a
  Kubernetes Secret"
    assert prod["strategy"]["rollingUpdate"]["maxUnavailable"] == 0
    assert prod["resources"]["limits"]["memory"]
```

Code Snippet – Configuration change changes the checksum (tests/test_helm_config.py, lines 208–220)

```
@needs_helm
def test_config_change_changes_checksum():
    """Changing a config value must roll the pods (new checksum annotation)."""

    def checksum(*extra):
        out = subprocess.run(
            [HELM, "template", "r", CHART_DIR, "-f", os.path.join(CHART_DIR,
  "values-dev.yaml"), *extra],
            check=True, capture_output=True, text=True,
        ).stdout
        dep = next(d for d in yaml.safe_load_all(out) if d and d["kind"] ==
  "Deployment")
        return
  dep["spec"]["template"]["metadata"]["annotations"]["checksum/config"]

    assert checksum() != checksum("--set", "config.LOG_LEVEL=INFO")
```

The tests that need Helm are skipped, not failed, when the `helm` binary is absent (`needs_helm`), so the application and static tests can still be run on a machine without Helm. In CI the binary is always installed, so all 62 cases run.

3.14 Continuous Integration with GitHub Actions

The workflow `.github/workflows/helm-validation.yml` runs on every push, on every pull request and on demand. It has four jobs (Figure 3.11, Table 3.17). The first three are independent and run in parallel; the fourth, which deploys to a cluster, starts only if all three pass (`needs:`). The versions of Helm (v3.16.2) and kubeconform (v0.6.7) [38] and the Kubernetes version of the manifest schemas (1.31.0) are fixed in the `env:` block, so that a new release of a tool cannot change the result of an unchanged commit [27].

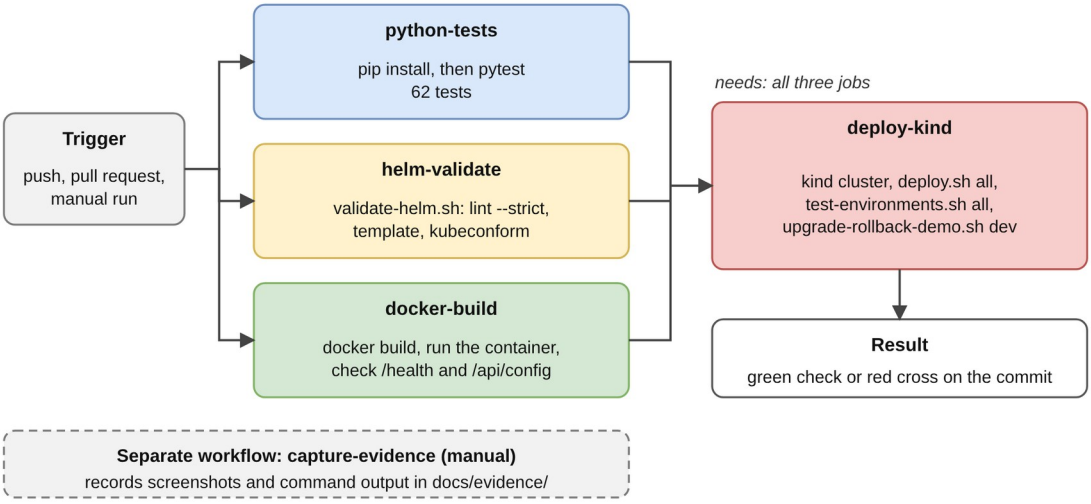


Figure 3.11 – Jobs of the validation workflow

Table 3.17 – Jobs of the workflow `helm-validation.yml`

Job	Steps	Failure means
Unit and configuration tests	Install dependencies; `python -m pytest -v` (62 cases)	Application, values or chart logic is wrong
Helm lint, template, kubeconform	`scripts/validate-helm.sh` for four environments; upload the rendered manifests as an artefact	Schema violation, template error or invalid Kubernetes object
Docker build and smoke test	Build the image; run it with `APP_ENV=testing`; check `/health`, `/api/config` and the dashboard	The image does not build or the application does not start
Deploy all environments to a kind cluster	Create a kind cluster; build and load the image; `deploy.sh all`; `test-environments.sh all`; `upgrade-rollback-demo.sh dev`	A release does not become ready, a live check fails, or an upgrade or rollback fails

Code Snippet – Cluster job of the workflow (.github/workflows/helm-validation.yml, lines 81–111)

```
  deploy-kind:
    name: Deploy all environments to a kind cluster
    runs-on: ubuntu-latest
    needs: [python-tests, helm-validate, docker-build]
    steps:
      - uses: actions/checkout@v5

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Create kind cluster
        uses: helm/kind-action@v1
        with:
          cluster_name: chart-testing

      - name: Build image and load it into kind
        run: |
          docker build -t "$IMAGE" .
          kind load docker-image "$IMAGE" --name chart-testing
          docker pull busybox:1.36
          kind load docker-image busybox:1.36 --name chart-testing

      - name: Deploy dev, test, staging and prod
        run: ./scripts/deploy.sh all

      - name: Verify environment-specific configuration (live)
        run: ./scripts/test-environments.sh all

      - name: Upgrade and rollback (dev)
        run: ./scripts/upgrade-rollback-demo.sh dev
```

A second workflow, `capture-evidence.yml`, was added for this report. It starts a kind cluster, runs `scripts/capture-evidence.sh` and commits the recorded files to `docs/evidence/` with the marker `[skip ci]`, so that the commit does not start the validation workflow again. Section 4.1 explains which results of this report come from it.

3.15 Security and Secret Handling

Table 3.18 lists the risks that the design addresses, the measure for each and the place where it can be checked. The project is a teaching example and does not claim to be a hardened production system; the limits are stated below the table.

Table 3.18 – Security measures

Risk	Measure	Where
Secret committed to Git	No secret in any values file; production reads `SECRET_KEY` from the Secret `multi-env-app-secrets`, which `deploy.sh` creates with a random key	`values-prod.yaml`, `deploy.sh`, test `test_no_hardcoded_secrets` (5 cases)
Secret placed in a ConfigMap by mistake	The schema forbids `SECRET_KEY`, `PASSWORD`, `DB_PASSWORD` and `API_TOKEN` in `config`	`values.schema.json`; `helm lint`
Secret shown on the dashboard or in the API	The key is replaced by `********` (`<source>`) in both	`config.py`; `test_secret_key_is_never_exposed`

Risk	Measure	Where
Container runs as root	UID 10001, `runAsNonRoot`, `allowPrivilegeEscalation: false`, all capabilities dropped	`Dockerfile`; `values.yaml`
Resource exhaustion by one Pod	CPU and memory requests and limits in every environment	`values-*.yaml`
Invalid or unsafe values	JSON Schema with enums, ranges and patterns	`values.schema.json`
A bad release stays in service	`--atomic` rollback; readiness probe keeps new Pods out of service until they answer	`deploy.sh`; `values.yaml`
Moving dependencies	Pinned Flask 3.0.3, gunicorn 23.0.0, Helm, kubeconform and schema versions; slim base image	`requirements.txt`; workflow `env:`

Kubernetes Secrets are only base64-encoded by default and are readable by anyone who may read Secrets in the namespace, so encryption at rest, role-based access control and an external secret manager are needed in a real production cluster [24]. The image was not scanned for vulnerabilities, and the base image is selected by a tag, not by a digest. Chapter V lists these points as future work.

3.16 Resource Requirements of the Four Environments

The requests and limits of every environment are values (Table 3.8). Table 3.19 multiplies them by the number of replicas to show what each environment asks of a cluster (level C: computed from the values files, not measured). A request is the amount that the scheduler reserves for a Pod; a limit is the ceiling that a container may not exceed.

Table 3.19 – Resources requested by the four environments (m = millicores of a CPU, Mi = mebibytes)

Environment	Replicas	CPU request / limit per Pod	Memory request / limit per Pod	Total requests	Total limits
dev	1	25 m / 100 m	64 Mi / 128 Mi	25 m, 64 Mi	100 m, 128 Mi
test	1	50 m / 150 m	64 Mi / 128 Mi	50 m, 64 Mi	150 m, 128 Mi
staging	2	100 m / 250 m	96 Mi / 192 Mi	200 m, 192 Mi	500 m, 384 Mi
prod	3	100 m / 500 m	128 Mi / 256 Mi	300 m, 384 Mi	1,500 m, 768 Mi
All four	**7**			**575 m, 704 Mi**	**2,250 m, 1,408 Mi**

All four environments together request 0.575 CPU and 704 MiB, and their limits add up to 2.25 CPU and 1,408 MiB. The requests fit easily into the Minikube cluster of 2 CPUs and 4096 MB that `minikube-setup.sh` creates, which must also run the system Pods of Kubernetes (their use was not measured). The limits add up to more than two CPUs. This is normal, because a limit is a ceiling that is not reserved, but it means that all Pods could not run at their limits at the same time.

CHAPTER 4
RESULTS AND DISCUSSION

4.1 Basis of the Results

A result is only as useful as the account of how it was obtained. Every result in this chapter therefore carries one of four evidence levels (Table 4.1), and the text states the level wherever a result is introduced.

Table 4.1 – Evidence levels used in this report

Level	How the result was obtained	Used in this report for
A Cluster run	Executed on a kind cluster in GitHub Actions. The output was written to files in `docs/evidence/` by `scripts/capture-evidence.sh` (workflow `capture-evidence.yml`, run 2) or taken from the validation workflow (run 6)	Deployments, dashboards, live checks, `helm test`, upgrade and rollback, forced failures, availability, CI timings
B Local run	Executed on the development machine (Linux, Python 3.13, Helm v3.16.2 and kubeconform v0.6.7 built from source) without a cluster	The 62 pytest cases, `validate-helm.sh`, the schema rejection tests, the rendered manifests
C Derived	Computed from level A or B data or from the repository files	Line counts, the break-even analysis, failure rates and percentages
D Documented only	Described in the README and supported by the scripts, but not executed for this report	Minikube on a laptop, Windows, cloud clusters

The dashboards in this chapter are screenshots of the pages that the Pods of the kind cluster served, taken with headless Google Chrome through `kubectl port-forward`; they are not mock-ups. The terminal figures are rendered from the text files in `docs/evidence/`. In these figures, columns that carry no information for the argument (for example the update time or the IP address), the requests of the kubelet probes in the logs, and unrelated lines of long descriptions are omitted, and long commands are wrapped; every such omission is stated in the text, and the complete output is in the files.

Table 4.2 describes the system on which the level A results were recorded. The cluster has a single node, so the results about networking and rollouts apply to a single-node cluster.

Table 4.2 – System of the cluster run (level A)

Item	Value
Runner	GitHub-hosted `ubuntu-latest`, Linux 7.0.0-1017-azure, x86-64
Processor and memory	AMD EPYC 7763, 4 vCPUs used, 15 GiB memory
Container engine	Docker 28.0.4

Item	Value
Cluster	kind v0.31.0; one control-plane node (`evidence-control-plane`, Debian 12, containerd 2.2.0)
Kubernetes	v1.35.0 (client and server)
Helm	v3.16.2
Application image	`multi-env-app:1.0.0`, 126 MB (base image Python 3.12.15)
Browser for the screenshots	Google Chrome 155.0.8059.84, headless
Workflow runs	`helm-validation.yml` run 6 and `capture-evidence.yml` run 2, both for commit 9f0e6bc

The following were **not** executed for this report: a run on Minikube (the scripts support it, and the README documents it), a run on Windows, a run on a multi-node or cloud cluster, a load test with realistic traffic, a vulnerability scan of the image and tests with other browsers. Statements about these are marked as level D or are listed as future work.

4.2 Deployment of the Four Environments

`scripts/deploy.sh all` created the four namespaces and installed the four releases one after the other. In the validation workflow this step took 40 s for all four environments, and in the evidence run every release reached the status *deployed* at revision 1 (Figure 4.1). Table 4.3 summarises the objects of the four releases as the cluster reported them.

bash - helm

```
$ helm list -A
NAME           NAMESPACE  REVISION  STATUS    CHART                APP VERSION
myapp-dev      dev        1         deployed  multi-env-app-1.0.0  1.0.0
myapp-prod     prod       1         deployed  multi-env-app-1.0.0  1.0.0
myapp-staging  staging    1         deployed  multi-env-app-1.0.0  1.0.0
myapp-test     test       1         deployed  multi-env-app-1.0.0  1.0.0
```

Figure 4.1 – Helm releases after `scripts/deploy.sh all` (`helm list -A`, update time omitted)

Table 4.3 – Releases of the four environments in the cluster (level A)

Environment	Namespace	Release	Ready / desired	Pods	NodePort	ConfigMap keys
Development	dev	myapp-dev	1/1	1	30081	13
Testing	test	myapp-test	1/1	1	30082	13
Staging	staging	myapp-staging	2/2	2	30083	13
Production	prod	myapp-prod	3/3	3	30084	13

The production environment shows the objects in detail (Figure 4.2). The Deployment is 3/3 ready, up to date and available; the Service is of the type NodePort

and maps port 80 to the NodePort 30084; the ConfigMap `myapp-prod-multi-env-app-config` has 13 keys (the second ConfigMap, `kube-root-ca.crt`, is created by Kubernetes in every namespace); and the three Pods are running without restarts.

bash - kubectl, kind cluster

```
$ kubectl get deploy,svc,cm,pods -n prod -o wide
NAME                                          READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/myapp-prod-multi-env-app   3/3     3            3           8s

NAME                                TYPE       PORT(S)        AGE
service/myapp-prod-multi-env-app   NodePort   80:30084/TCP   8s

NAME                                          DATA   AGE
configmap/kube-root-ca.crt                    1      9s
configmap/myapp-prod-multi-env-app-config   13     8s

NAME                                              READY   STATUS    RESTARTS   AGE
pod/myapp-prod-multi-env-app-78c6bddb45-4xx97   1/1     Running   0          8s
pod/myapp-prod-multi-env-app-78c6bddb45-j2t8b   1/1     Running   0          8s
pod/myapp-prod-multi-env-app-78c6bddb45-nm9nv   1/1     Running   0          8s
```

Figure 4.2 – Objects of the production environment (`kubectl get deploy,svc,cm,pods -n prod`; address, selector and node columns omitted)

The content of the production ConfigMap (Figure 4.3) is the rendering of `values.yaml` and `values-prod.yaml`: every value is a string, the six fixed keys of the template are present, and `HELM_RELEASE` and `HELM_CHART_VERSION` record which release and chart produced the object.

bash - kubectl

```
$ kubectl get configmap myapp-prod-multi-env-app-config -n prod -o yaml
data:
  API_BASE_URL: https://multi-env-app.example.com/api
  APP_ENV: production
  APP_NAME: Multi-Env Config Dashboard
  APP_VERSION: 1.0.0
  CACHE_TTL_SECONDS: "300"
  DATABASE_HOST: postgres.prod.svc.cluster.local
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "false"
  HELM_CHART_VERSION: 1.0.0
  HELM_RELEASE: myapp-prod
  LOG_LEVEL: WARNING
  MAX_CONNECTIONS: "200"
  REPLICA_COUNT: "3"
```

Figure 4.3 – Data section of the production ConfigMap (`kubectl get configmap ... -o yaml`, metadata omitted)

Figure 4.4 shows the Deployment of production as described by Kubernetes. It confirms that the values reached the cluster: three replicas; the strategy `RollingUpdate` with 0 maximum unavailable and 1 maximum surge; the checksum

annotation; the image `multi-env-app:1.0.0`; the resource requests and limits of production; the two probes; the environment taken from the ConfigMap *and* from the Secret `multi-env-app-secrets`; and the three Downward API variables.

bash - kubectl describe

```
$ kubectl describe deployment myapp-prod-multi-env-app -n prod
Name:                   myapp-prod-multi-env-app
Namespace:              prod
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
RollingUpdateStrategy:  0 max unavailable, 1 max surge
  Annotations:  checksum/config: a36e621c071f2c2bd77b55dd9aff5a6ff345417abb758870d2f060df7b50e7cb
    Image:      multi-env-app:1.0.0
    Port:       5000/TCP (http)
    Limits:
      cpu:     500m
      memory:  256Mi
    Requests:
      cpu:      100m
      memory:   128Mi
    Liveness:   http-get http://:http/health delay=5s timeout=1s period=10s #success=1 #failure=3
    Readiness:  http-get http://:http/ready delay=3s timeout=1s period=5s #success=1 #failure=3
    Environment Variables from:
      myapp-prod-multi-env-app-config  ConfigMap  Optional: false
      multi-env-app-secrets            Secret     Optional: false
    Environment:
      PORT:           5000
      POD_NAME:        (v1:metadata.name)
      POD_NAMESPACE:   (v1:metadata.namespace)
      NODE_NAME:       (v1:spec.nodeName)
```

Figure 4.4 – Deployment of the production environment (`kubectl describe deployment`, selected lines)

Two Secrets exist in the production namespace (Figure 4.5). `multi-env-app-secrets` holds the `SECRET_KEY` and was created by `deploy.sh`, not by the chart; `sh.helm.release.v1.myapp-prod.v1` is the record of revision 1 that Helm stores and that `helm rollback` reads.

bash - kubectl

```
$ kubectl get secret -n prod
NAME                                TYPE                 DATA   AGE
multi-env-app-secrets               Opaque               1      9s
sh.helm.release.v1.myapp-prod.v1    helm.sh/release.v1   1      9s
```

Figure 4.5 – Secrets of the production namespace (`kubectl get secret -n prod`)

4.3 Dashboards of the Four Environments

Figure 4.6 to Figure 4.9 show the dashboards of the four environments. They are produced by the same image and the same templates; only the values file differs. The colour of the banner, the cards and the table header identifies the environment

(green, cyan, yellow and striped red). Development and testing show the beta banner because `FEATURE_BETA_BANNER` is true there, and staging and production do not.

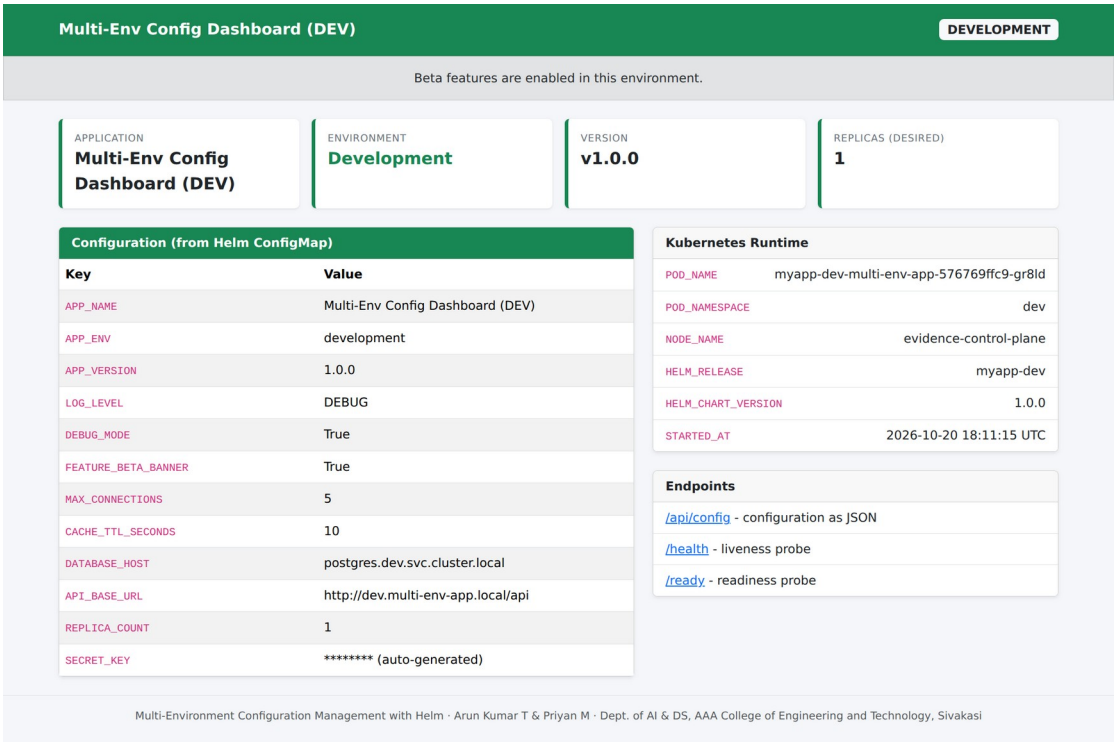


Figure 4.6 – Dashboard of the development environment, served by the Pod myapp-dev-multi-env-app-576769ffc9-gr8ld

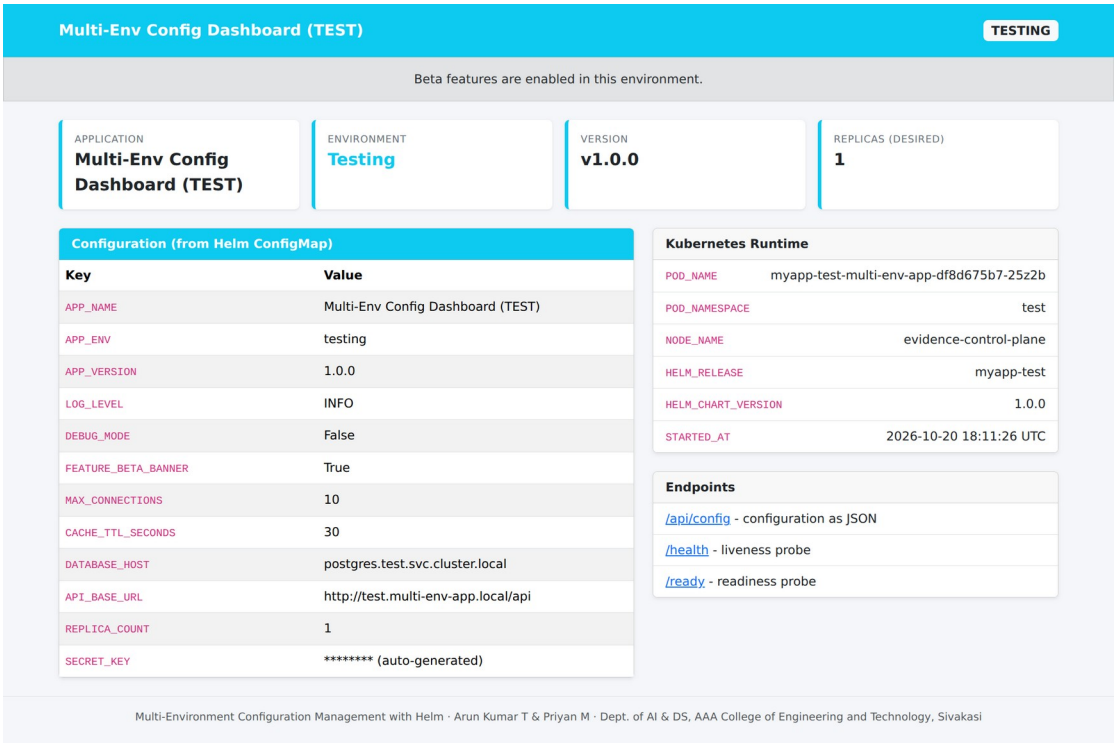


Figure 4.7 – Dashboard of the testing environment, served by the Pod myapp-test-multi-env-app-df8d675b7-25z2b

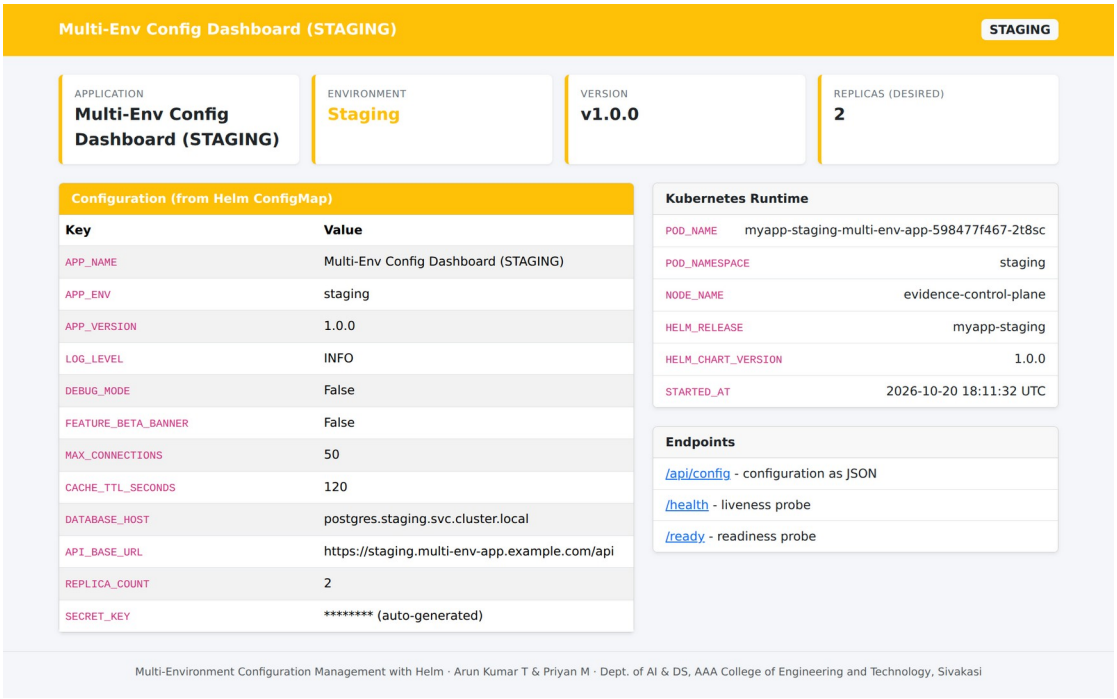


Figure 4.8 – Dashboard of the staging environment, served by the Pod myapp-staging-multi-env-app-598477f467-2t8sc

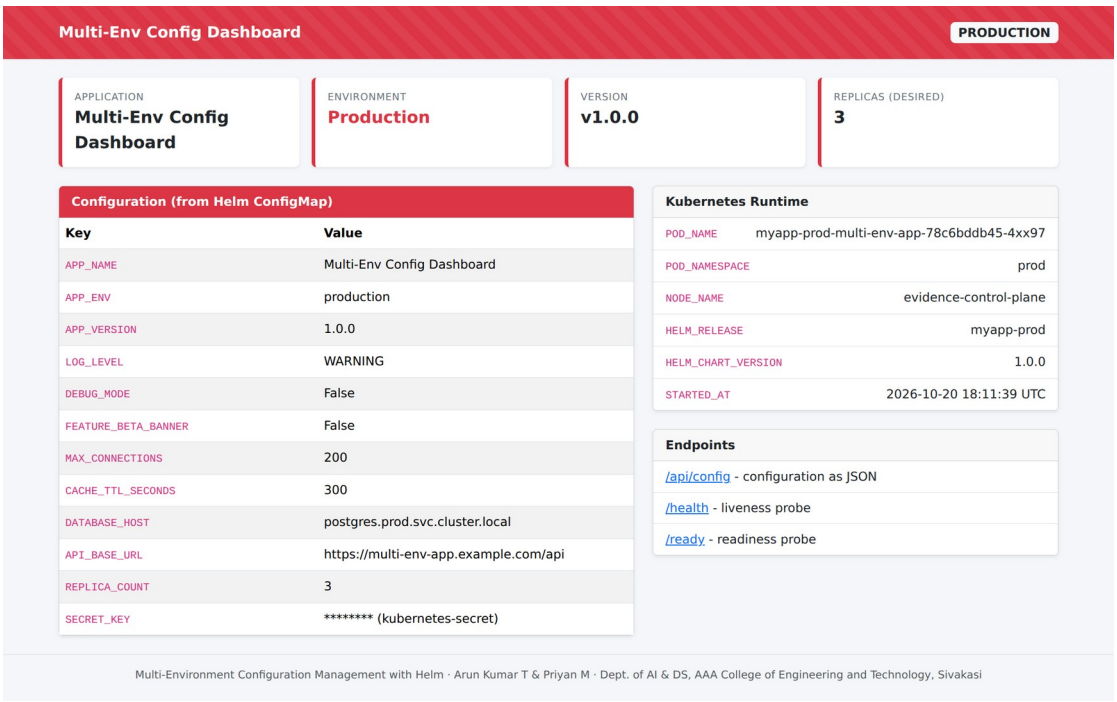


Figure 4.9 – Dashboard of the production environment, served by the Pod myapp-prod-multi-env-app-78c6bddb45-4xx97

The JSON that every Pod returns from `/api/config` was compared, value by value, with the values that Helm merges from `values.yaml` and the environment file (11 settings in each of the four environments, 44 comparisons). In addition, the source of the secret key, the release name and the namespace were compared with the expected values (12 comparisons). All 56 comparisons were equal (Table 4.4), which

closes the chain from the file to the running container. The `SECRET_KEY` is shown only as a masked value with its source: *auto-generated* in development, testing and staging, and *kubernetes-secret* in production.

Table 4.4 – Settings reported by the running Pods through /api/config (level A)

Setting	dev	test	staging	prod
`APP_NAME`	Multi-Env Config Dashboard (DEV)	Multi-Env Config Dashboard (TEST)	Multi-Env Config Dashboard (STAGING)	Multi-Env Config Dashboard
`APP_ENV`	development	testing	staging	production
`APP_VERSION`	1.0.0	1.0.0	1.0.0	1.0.0
`LOG_LEVEL`	DEBUG	INFO	INFO	WARNING
`DEBUG_MODE`	true	false	false	false
`FEATURE_BETA_BANNER`	true	true	false	false
`MAX_CONNECTIONS`	5	10	50	200
`CACHE_TTL_SECONDS`	10	30	120	300
`DATABASE_HOST`	postgres.dev.svc.cluster.local	postgres.test.svc.cluster.local	postgres.staging.svc.cluster.local	postgres.prod.svc.cluster.local
`API_BASE_URL`	http://dev.multi-env-app.local/api	http://test.multi-env-app.local/api	https://staging.multi-env-app.example.com/api	https://multi-env-app.example.com/api
`REPLICA_COUNT`	1	1	2	3
`SECRET_KEY`	(auto-generated)	(auto-generated)	(auto-generated)	(kubernetes-secret)
`HELM_RELEASE`	myapp-dev	myapp-test	myapp-staging	myapp-prod
`POD_NAMESPACE`	dev	test	staging	prod

Figure 4.10 compares the numeric settings of the environments. The resource limits and the number of connections grow from development to production, and the two feature flags are on in the two lower environments.

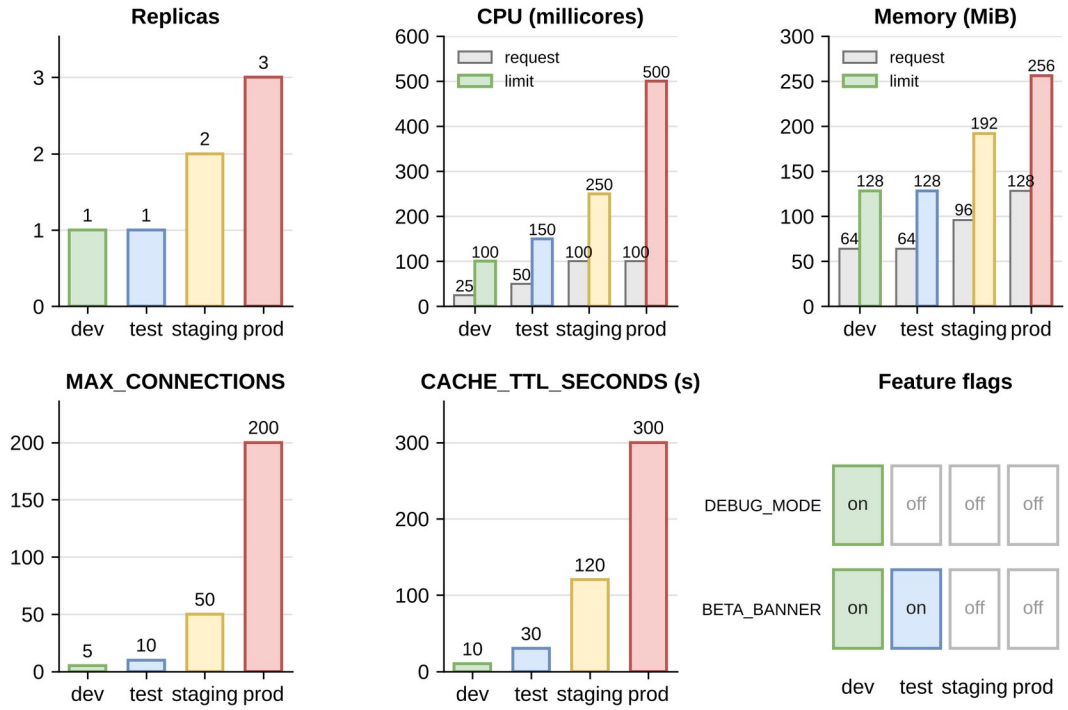


Figure 4.10 – Environment-specific settings read from the values files

The effect of the setting `LOG_LEVEL` is visible in the logs (Figure 4.11). The development Pod, with the level DEBUG, prints the start-up message of the application (at level INFO) and the access log of gunicorn; the production Pod, with the level WARNING, prints only the messages of gunicorn and the access log, and the start-up message of the application is suppressed.

bash - kubectl logs

```
$ kubectl logs -n dev deploy/myapp-dev-multi-env-app --tail=12
2026-10-20 18:11:15,549 INFO [multi-env-app] Starting Multi-Env Config Dashboard (DEV) v1.0.0 in
    development environment
  ... (requests of the kubelet probes omitted)

$ kubectl logs -n prod deploy/myapp-prod-multi-env-app --tail=12
Found 3 pods, using pod/myapp-prod-multi-env-app-78c6bddb45-4xx97
[2026-10-20 18:11:39 +0000] [1] [INFO] Starting gunicorn 23.0.0
[2026-10-20 18:11:39 +0000] [1] [INFO] Listening at: http://0.0.0.0:5000 (1)
[2026-10-20 18:11:39 +0000] [1] [INFO] Using worker: sync
[2026-10-20 18:11:39 +0000] [14] [INFO] Booting worker with pid: 14
[2026-10-20 18:11:39 +0000] [15] [INFO] Booting worker with pid: 15
  ... (requests of the kubelet probes omitted)
```

Figure 4.11 – Logs of a development Pod and a production Pod (`kubectl logs`, probe requests omitted)

4.4 Test Cases and Results

Table 4.5 lists the test cases and their results. All automated cases passed. The runs behind the cases TC-01 to TC-04, TC-06 and TC-07 are described below, each with the figure that shows its output.

Table 4.5 – Test cases and results

ID	Test	Expected result	Result	Level
TC-01	Application tests (`tests/test_app.py`)	15 cases pass	15 passed	B, A
TC-02	Static configuration tests (values files)	32 cases pass	32 passed	B, A
TC-03	Chart tests with Helm (lint, render, checksum, schema)	15 cases pass	15 passed	B, A
TC-04	`validate-helm.sh`: lint, render and kubeconform for 4 environments	No error; `APP_ENV` rendered correctly; 4 of 4 objects valid per environment	Passed for all four environments	B, A
TC-05	Docker smoke test in CI	Container starts; `/health`, `/api/config` and the dashboard answer	Job succeeded	A
TC-06	Live checks `test-environments.sh all`	26 checks pass	26 passed, 0 failed	A
TC-07	`helm test` for each release	Test Pod phase Succeeded	Succeeded (4 of 4)	A
TC-08	Schema rejection of invalid values	14 invalid inputs are rejected before deployment	14 of 14 rejected	B
TC-09	Upgrade and rollback of development	Revision 2 shows version 1.1.0 and INFO; revision 3 restores version 1.0.0 and DEBUG	As expected	A
TC-10	Upgrade with a non-existent image and `--atomic`	Rollback to the previous revision; old Pod keeps running	As expected	A
TC-11	Production installed without its Secret	Pods do not start; the cause is visible in the events	CreateContainerConfig Error; Secret not found	A
TC-12	Availability during rolling updates (13 runs)	Measured, not asserted	Failed requests found; see Section 4.9	A
TC-13	Validation workflow	All four jobs succeed	Succeeded (run 6)	A

Unit and configuration tests (TC-01 to TC-03). All 62 pytest cases passed on the development machine in about one and a half seconds (Figure 4.12). The CI job runs the same command with Helm installed, so the cases that need Helm are run there as well.

bash - multi-env-helm-config

```
$ python -m pytest -q
............................................................          [100%]
62 passed in 1.46s
```

Figure 4.12 – Result of the pytest run on the development machine

Validation without a cluster (TC-04). `validate-helm.sh` lints, renders and validates each of the four environments (Figure 4.13). All twelve stages (three for each environment) succeeded: `helm lint --strict` reported no failed chart, every environment rendered the expected `APP_ENV`, and kubeconform found four of four objects valid in each environment.

```
bash - multi-env-helm-config
$ ./scripts/validate-helm.sh | grep -E '==>|linted|APP_ENV|Summary|validated'
==> [dev] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [dev] helm template
APP_ENV correctly rendered as development
==> [dev] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
==> [test] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [test] helm template
APP_ENV correctly rendered as testing
==> [test] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
==> [staging] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [staging] helm template
APP_ENV correctly rendered as staging
==> [staging] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
==> [prod] helm lint --strict
1 chart(s) linted, 0 chart(s) failed
==> [prod] helm template
APP_ENV correctly rendered as production
==> [prod] kubeconform
Summary: 4 resources found in 1 file - Valid: 4, Invalid: 0, Errors: 0, Skipped: 0
All environments validated successfully.
```

Figure 4.13 – Output of `scripts/validate-helm.sh` (lines of the three stages of each environment)

Live checks (TC-06). `test-environments.sh all` ran 26 checks on the cluster (Figure 4.14): six for each of development, testing and staging, and eight for production, which has two additional checks, one for the debug mode and one for the source of the secret key. The last check of every environment is `helm test`.



Figure 4.14 – Live checks `scripts/test-environments.sh all` on the kind cluster

Chart test (TC-07). The test Pod of a release asks the Service for `/health` and `/api/config` and checks the environment name in the answer. Figure 4.15 shows the run for production, including the release notes that Helm prints; the other three releases passed in the same way (Figure 4.14).

```
bash - helm test
$ helm test myapp-prod -n prod
NAME: myapp-prod
LAST DEPLOYED: Tue Oct 20 18:11:37 2026
NAMESPACE: prod
STATUS: deployed
REVISION: 1
TEST SUITE:     myapp-prod-multi-env-app-test-connection
Last Started:   Tue Oct 20 18:11:58 2026
Last Completed: Tue Oct 20 18:12:00 2026
Phase:          Succeeded
NOTES:
Multi-Env Config Dashboard has been deployed.

  Release     : myapp-prod (revision 1)
  Namespace   : prod
  Environment : production
  Version     : 1.0.0
  Replicas    : 3

Open the dashboard:
  minikube service myapp-prod-multi-env-app -n prod --url
  kubectl port-forward -n prod svc/myapp-prod-multi-env-app 8080:80
  then browse to http://localhost:8080

Verify the release:
  helm test myapp-prod -n prod
```

Figure 4.15 – `helm test` for the production release

A test that never fails proves little, so the suite was also checked from the other side. The 14 invalid inputs of Section 4.6 are rejected, the two failures of Section 4.8 are detected, and the availability experiment of Section 4.9 was designed to find a problem and found one. The automated cases cannot detect what they do not test: they do not measure traffic, do not scan the image and do not cover Minikube or Windows.

4.5 Reduction of Duplicated Configuration

The main argument for the chart is that it removes duplication. To make this argument measurable, the effort of keeping E environments consistent is modelled by the number of lines that must be maintained (level C; lines are only a proxy for effort). With copied manifests every environment needs its own rendered manifest of about R lines; with the chart there is one fixed part, made of the templates (T lines) and the default values (V lines), and every environment adds one values file of about d lines:

$$L_{copy}(E) = E \cdot R \tag{4.1}$$

$$L_{chart}(E) = T + V + E \cdot d \tag{4.2}$$

The chart needs fewer lines than copies as soon as E exceeds the break-even number given in Equation (4.3).

$$E^* = (T + V) / (R - d) \tag{4.3}$$

The values were counted from the repository: T = 186 lines (five templates including the test), V = 98 lines, R = 177.5 lines (the average rendered size of the four environments, Table 3.12), and d = 31.75 lines (the average size of the four environment files). The break-even number is E* = 1.95; if the schema (75 lines) and `Chart.yaml` (18 lines) are counted as part of the fixed cost, it is 2.59. The chart is therefore smaller than copied manifests from the third environment onwards. For the four environments of this project the copies would amount to 710 lines and the chart to 411 lines, which is 42 % fewer (29 % fewer if schema and metadata are included). A fifth environment would add 31.75 lines instead of 177.5, a saving of 82 % per additional environment. Figure 4.16 shows the two views.

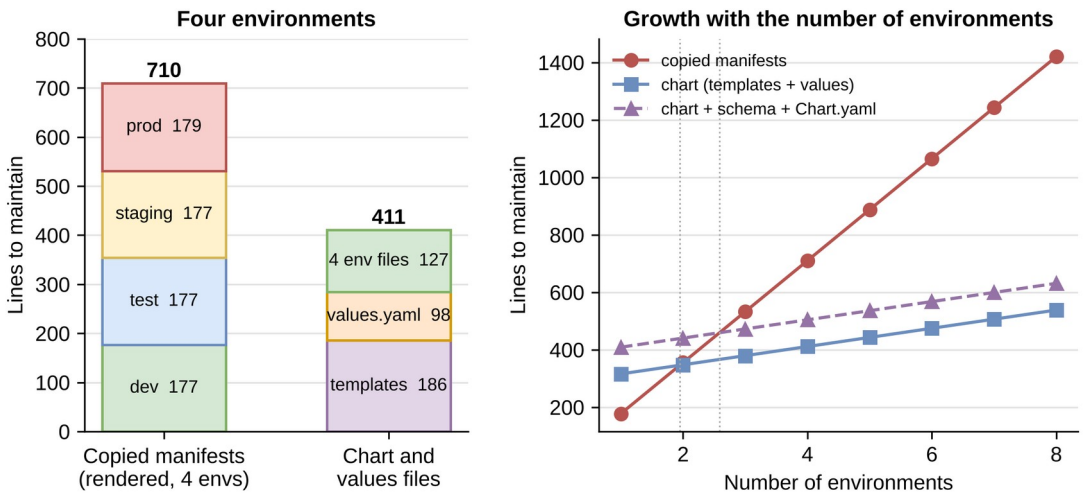


Figure 4.16 – Lines to maintain: copied manifests compared with the chart and values files (level C)

Table 4.6 – Lines to maintain for E environments

E	Copied manifests	Chart and values	Chart, values, schema and metadata	Saving (chart and values)
1	177.5	315.75	408.75	none
2	355	347.5	440.5	2 %
3	532.5	379.25	472.25	29 %
4	710	411	504	42 %
6	1065	474.5	567.5	55 %
8	1420	538	631	62 %

Assumptions: the copies are the rendered manifests (including the test Pod); lines are a proxy for effort; the numbers of lines are those of the delivered files.

The model shows the size of the maintained text, not the time that a change takes. Its more important consequence is qualitative: with the chart a change of a template is made once, and the differences between environments are visible as the few lines of a values file (Table 3.12 lists 8 to 13 differing values per environment).

4.6 Schema Validation Results

To test the schema from the outside, 14 invalid inputs were each given to `helm lint ./helm/multi-env-app --strict --set <input>` (level B). All 14 were rejected with the exit status 1 and a message that names the offending key (Table 4.7). Five of them are also pytest cases (`test_schema_rejects_invalid_values`), and Figure 4.17 shows four of the runs. The four environment files, in contrast, pass the same command (TC-04, Figure 4.13).

Table 4.7 – Invalid values rejected by values.schema.json

Input (--set)	Exit	Message of Helm
`environment=qa`	1	environment: environment must be one of the following: "development", "testing", "staging", "production"
`replicaCount=0`	1	replicaCount: Must be greater than or equal to 1
`replicaCount=11`	1	replicaCount: Must be less than or equal to 10
`config.LOG_LEVEL=VERBOSE`	1	config.LOG_LEVEL: config.LOG_LEVEL must be one of the following: "DEBUG", "INFO", "WARNING", "ERROR", "CRITICAL"
`config.DEBUG_MODE=maybe`	1	config.DEBUG_MODE: config.DEBUG_MODE must be one of the following: "true", "false"
`config.SECRET_KEY=abc`	1	config: Must not validate the schema (not)
`config.log_level=info`	1	config: Does not match pattern '^[A-Z][A-Z0-9_]*$'
`service.type=Foo`	1	service.type: service.type must be one of the following: "ClusterIP", "NodePort", "LoadBalancer"
`service.nodePort=80`	1	service.nodePort: Must be greater than or equal to 30000
`image.pullPolicy=Sometimes`	1	image.pullPolicy: image.pullPolicy must be one of the following: "Always", "IfNotPresent", "Never"
`containerPort=0`	1	containerPort: Must be greater than or equal to 1
`config.MAX_CONNECTIONS=abc`	1	config.MAX_CONNECTIONS: Does not match pattern '^[0-9]+$'
`config.CACHE_TTL_SECONDS=130 (number, not string)`	1	config.CACHE_TTL_SECONDS: Invalid type. Expected: string, given: integer
`image.repository=""`	1	image.repository: String length must be greater than or equal to 1

```
bash - multi-env-helm-config
$ helm lint ./helm/multi-env-app --strict --set environment=qa
[ERROR] values.yaml: - environment: environment must be one of the following: "development",
    "testing", "staging", "production"
Error: 1 chart(s) linted, 1 chart(s) failed

$ helm lint ./helm/multi-env-app --strict --set replicaCount=0
[ERROR] values.yaml: - replicaCount: Must be greater than or equal to 1
Error: 1 chart(s) linted, 1 chart(s) failed

$ helm lint ./helm/multi-env-app --strict --set config.SECRET_KEY=abc
[ERROR] values.yaml: - config: Must not validate the schema (not)
Error: 1 chart(s) linted, 1 chart(s) failed

$ helm lint ./helm/multi-env-app --strict --set config.LOG_LEVEL=VERBOSE
[ERROR] values.yaml: - config.LOG_LEVEL: config.LOG_LEVEL must be one of the following: "DEBUG",
    "INFO", "WARNING", "ERROR", "CRITICAL"
Error: 1 chart(s) linted, 1 chart(s) failed
```

Figure 4.17 – Four invalid values rejected by `helm lint --strict` (error messages shortened to the first line)

The rejected inputs cover the classes of mistake that the studies of Chapter II report as frequent: an unknown name (`environment=qa`), numbers outside their range (`replicaCount=0` and `11`, `containerPort=0`, `service.nodePort=80`), values outside an enumeration (`LOG_LEVEL`, `DEBUG_MODE`, `service.type`, `image.pullPolicy`), values with the wrong format (`MAX_CONNECTIONS=abc`, a lower-case key), a value of the wrong type, an empty required string, and a secret placed in the ConfigMap (`config.SECRET_KEY`).

Two observations are worth recording. First, the type check has a practical consequence: `--set config.CACHE_TTL_SECONDS=130` is rejected because Helm reads 130 as an integer, whereas the schema and the ConfigMap require a string; the correct form is `--set-string`. The availability experiment of Section 4.9 uses `--set-string` for this reason. Second, the message for a forbidden key (`Must not validate the schema (not)`) does not name the key, which is a weakness of the report of the `not` keyword in Helm's validator and makes this one rule harder to diagnose than the others. The cases were written by the authors of the schema, so they show that the rules do what they were written to do; they do not show that the set of rules is complete.

4.7 Upgrade and Rollback

The release lifecycle was exercised on the development release with `scripts/upgrade-rollback-demo.sh dev` (level A). The upgrade set `app.version` to 1.1.0 and `LOG_LEVEL` to INFO with `--set`; the rollback returned to

revision 1. Figure 4.18 shows the commands and the history, and Figure 4.19 shows the dashboards of the two later revisions.

```
bash - helm
$ helm upgrade myapp-dev ./helm/multi-env-app -n dev -f values-dev.yaml --set \
    app.version=1.1.0 --set config.LOG_LEVEL=INFO --atomic --wait --timeout 5m
Release "myapp-dev" has been upgraded. Happy Helming!
NAME: myapp-dev
LAST DEPLOYED: Tue Oct 20 18:12:08 2026
NAMESPACE: dev

$ helm history myapp-dev -n dev
REVISION  STATUS      CHART                APP VERSION  DESCRIPTION
1         superseded  multi-env-app-1.0.0  1.0.0        Install complete
2         deployed    multi-env-app-1.0.0  1.0.0        Upgrade complete

$ helm rollback myapp-dev 1 -n dev --wait --timeout 5m
Rollback was a success! Happy Helming!

$ helm history myapp-dev -n dev
REVISION  STATUS      CHART                APP VERSION  DESCRIPTION
1         superseded  multi-env-app-1.0.0  1.0.0        Install complete
2         superseded  multi-env-app-1.0.0  1.0.0        Upgrade complete
3         deployed    multi-env-app-1.0.0  1.0.0        Rollback to 1
```

Figure 4.18 – Upgrade, history, rollback and history of the development release (update times omitted)

Four points follow from Figure 4.18, Figure 4.19 and Table 4.8. (1) The upgrade replaced the Pod: the change of `LOG_LEVEL` and `app.version` changed the ConfigMap, therefore the checksum annotation and the Pod template, so a new ReplicaSet and a new Pod were created without any manual restart. (2) The rollback did not delete revision 2; it created revision 3 with the description *Rollback to 1*, so the history remains a complete record. (3) The Pod of revision 3 carries the same template hash (`576769ffc9`) as the Pod of revision 1, which shows that the Deployment reused the ReplicaSet of revision 1. (4) The column APP VERSION of `helm history` stays at 1.0.0 for revision 2 because it shows the `appVersion` of the chart, not the value `app.version` that the dashboard displays; the two are different things, and the dashboard is the place to read the version of the running application.

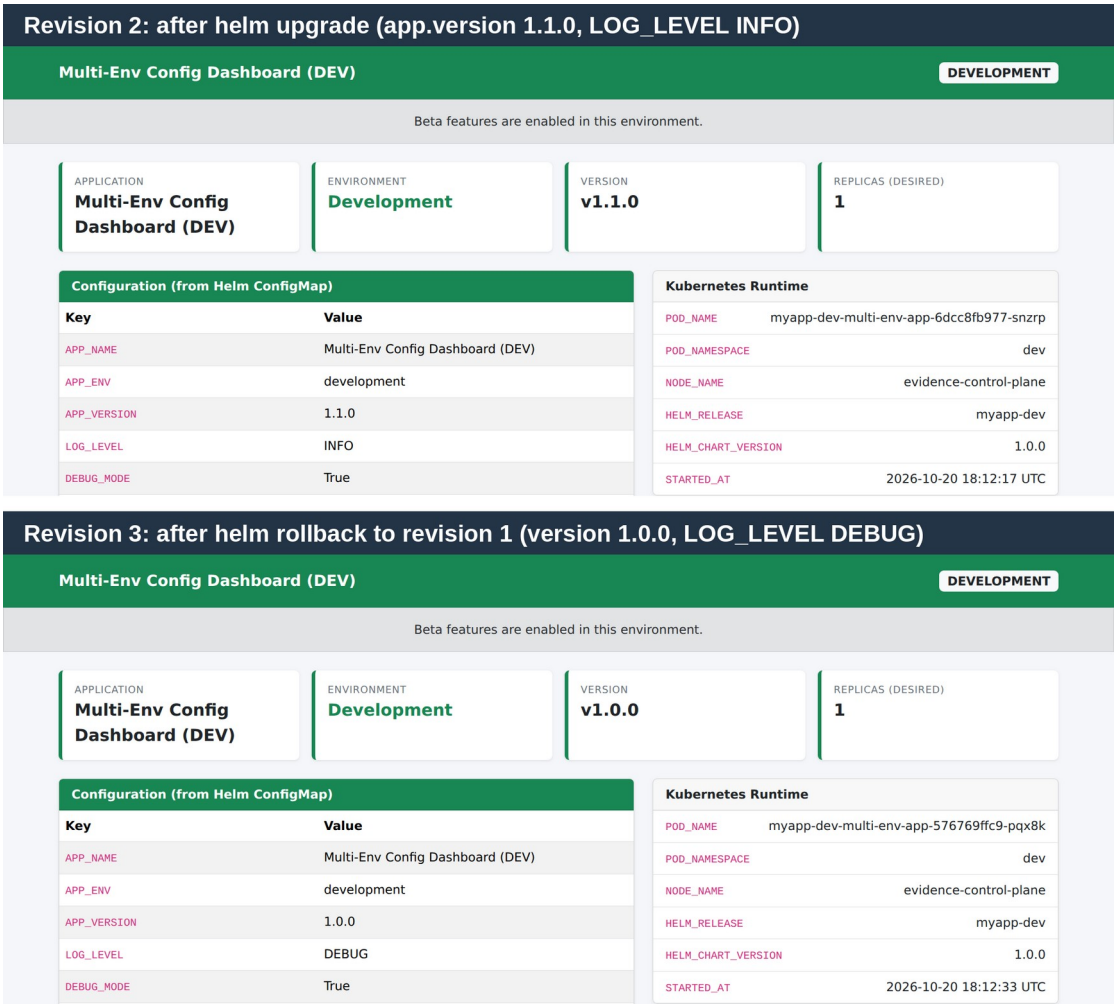


Figure 4.19 – Dashboard of the development release after the upgrade (revision 2) and after the rollback (revision 3); upper part of the page

Table 4.8 – Revisions of the development release

Revision	Action	Version shown	LOG_LEVEL	Pod serving the page	Status in the history
1	`helm upgrade --install`	1.0.0	DEBUG	`...-576769ffc9-gr8ld`	superseded
2	`helm upgrade --set app.version=1.1.0 --set config.LOG_LEVEL=INFO`	1.1.0	INFO	`...-6dcc8fb977-snzrp`	superseded
3	`helm rollback myapp-dev 1`	1.0.0	DEBUG	`...-576769ffc9-pqx8k`	deployed

4.8 Forced Failures

Two failures were provoked on purpose to find out how the release behaves when a prerequisite is missing. Both were run on the kind cluster (level A).

4.8.1 Upgrade to a non-existent image with --atomic

The test release was upgraded with `--set image.tag=9.9.9`, an image that does not exist, with `--atomic --wait --timeout 60s` (Figure 4.20). The new Pod could not pull its image and stayed in `ErrImagePull` and later `ImagePullBackOff`;

the event reports that the repository does not exist or that access is denied. The old Pod, in the meantime, was still *Running* and *Ready* (1/1) 35 s into the upgrade. This is the guarantee of `maxUnavailable: 0`: an old Pod is removed only after a new one is ready, so a broken revision never replaces a working one. After the timeout of 60 s, Helm reported *UPGRADE FAILED ... has been rolled back due to atomic being set*. The history shows revision 2 as *failed* and a new revision 3 *Rollback to 1* as *deployed*, and the final listing shows only the original Pod and the test Pod.

bash - helm and kubectl

```
# second terminal, 35 seconds after the upgrade was started
$ kubectl get pods -n test
NAME                                      READY   STATUS         RESTARTS   AGE
myapp-test-multi-env-app-67f47d647f-4tgzx   0/1     ErrImagePull   0          35s
myapp-test-multi-env-app-df8d675b7-25z2b    1/1     Running        0          115s
myapp-test-multi-env-app-test-connection    0/1     Completed      0          85s

# first terminal: the upgrade command returns after its 60 s timeout
$ helm upgrade myapp-test ./helm/multi-env-app -n test -f values-test.yaml --set image.tag=9.9.9 \
    --atomic --wait --timeout 60s
Error: UPGRADE FAILED: release myapp-test failed, and has been rolled back due to atomic being set:
    context deadline exceeded

$ helm history myapp-test -n test
REVISION  STATUS      DESCRIPTION
1         superseded  Install complete
2         failed      Upgrade "myapp-test" failed: context deadline exceeded
3         deployed    Rollback to 1

$ kubectl get pods -n test
NAME                                      READY   STATUS      RESTARTS   AGE
myapp-test-multi-env-app-df8d675b7-25z2b   1/1     Running     0          2m22s
myapp-test-multi-env-app-test-connection   0/1     Completed   0          112s
```

Figure 4.20 – Failed upgrade with `--atomic`: the old Pod keeps running and Helm restores revision 1 as revision 3

`deploy.sh` always uses `--atomic`, with a timeout of five minutes, so every deployment of the project has this protection. The experiment also shows a limit: the rollback took the full timeout, so a failed deployment is detected only after that time.

4.8.2 Production installed without its Secret

The production values were installed as the release `demo-prod` in an empty namespace in which the Secret `multi-env-app-secrets` had not been created (Figure 4.21). The command ended with `INSTALLATION FAILED: context deadline exceeded` after 40 s. The three Pods were in the state `CreateContainerConfigError`, and the events name the cause: *secret "multi-env-app-secrets" not found*.

```
bash - helm and kubectl
$ helm install demo-prod ./helm/multi-env-app -n demo -f values-prod.yaml --set \
    service.nodePort=30089 --wait --timeout 40s
Error: INSTALLATION FAILED: context deadline exceeded

$ kubectl get pods -n demo
NAME                                    READY   STATUS                       RESTARTS   AGE
demo-prod-multi-env-app-6b776c5d67-8lj9p   0/1     CreateContainerConfigError   0          41s
demo-prod-multi-env-app-6b776c5d67-jgz5f   0/1     CreateContainerConfigError   0          41s
demo-prod-multi-env-app-6b776c5d67-pft9d   0/1     CreateContainerConfigError   0          41s

$ kubectl get events -n demo --sort-by=.lastTimestamp | grep -i secret | tail -3
14s  Warning  Failed  pod/demo-prod-multi-env-app-6b776c5d67-jgz5f  Error: secret
    "multi-env-app-secrets" not found
```

Figure 4.21 – Production installed without its Secret (`helm install`, `kubectl get pods`, events)

Three conclusions follow. First, Helm cannot detect the problem: the chart only refers to the Secret by name, and the error is found by the kubelet when it creates the container. The message of Helm (a timeout) does not mention the Secret, so the diagnosis needs `kubectl get events`. Second, this is the reason why `deploy.sh` creates the Secret before it installs production. Third, `--atomic` was not used in this experiment (with it, Helm would have removed the failed installation); the evidence script removed the release and the namespace afterwards. A check inside the chart (for example with the Helm function `lookup`, which works only against a live cluster), a pre-install hook or an external secrets operator would detect the problem earlier; these are listed in Chapter V.

4.9 Availability during Rolling Updates

4.9.1 Question and Design of the Experiment

The bounds Equation (3.1) and Equation (3.2) say how many Pods exist during an update. They do not say whether a client notices the update. To find out, a probe Pod (`busybox:1.36`) was started in the namespace of the release. It requested `/ready` from the Service of the release with `wget` about nine times per second for 45 s and logged every failed request with the second at which it failed. Eight seconds after the probe had started, `helm upgrade --wait` changed the value `config.CACHE_TTL_SECONDS` (with `--set-string`), which changed the ConfigMap and the checksum annotation and thereby started a rolling update (Table 4.9). Five scenarios were run: a control without an update; the strategy of the chart on two replicas (staging) and on one replica (test); the alternative strategy `maxSurge: 0, maxUnavailable: 1`; and the strategy of the chart with a `preStop` hook that sleeps

for 5 s. The hook was added to the live Deployment with `kubectl patch` and is **not** part of the delivered chart.

Table 4.9 – Scenarios of the availability experiment

ID	Release	Replicas	Strategy	Runs
Z	`myapp-test`	1	No change of the release (control)	1
A	`myapp-staging`	2	Chart: `maxSurge: 1`, `maxUnavailable: 0`	3
B	`myapp-test`	1	Chart: `maxSurge: 1`, `maxUnavailable: 0`	3
C	`myapp-test`	1	Alternative: `maxSurge: 0`, `maxUnavailable: 1`	3
D	`myapp-test`	1	Chart strategy plus `lifecycle.preStop: sleep 5` (experiment only)	3

4.9.2 Results

Table 4.10 and Figure 4.22 give the results. The control produced no failure in 424 requests, so the probe and the path through the Service are reliable and the failures of the other scenarios are caused by the rollouts. With the strategy of the chart, 18 of 1,205 requests failed in scenario A (1.49 %) and 15 of 1,247 in scenario B (1.20 %). The alternative strategy lost 38 of 1,074 (3.54 %), and with the `preStop` delay none of 1,285 requests failed.

Table 4.10 – Failed requests during rolling updates (level A; totals over the runs of a scenario)

Scenario	Runs	Requests	Failed	Failure rate	Failed per run	Longest burst (s)	Bursts per run
Z control	1	424	0	0.00 %	0	none	0
A chart, staging	3	1,205	18	1.49 %	5–7	1–3	2
B chart, test	3	1,247	15	1.20 %	3–7	2–4	1
C alternative, test	3	1,074	38	3.54 %	12–13	8–10	1
D chart + preStop	3	1,285	0	0.00 %	0	none	0

A burst is a group of failed requests whose times differ by at most 3 s. Rollout times reported by Helm: A 15–16 s, B 8–9 s, C 2–3 s, D 8 s.

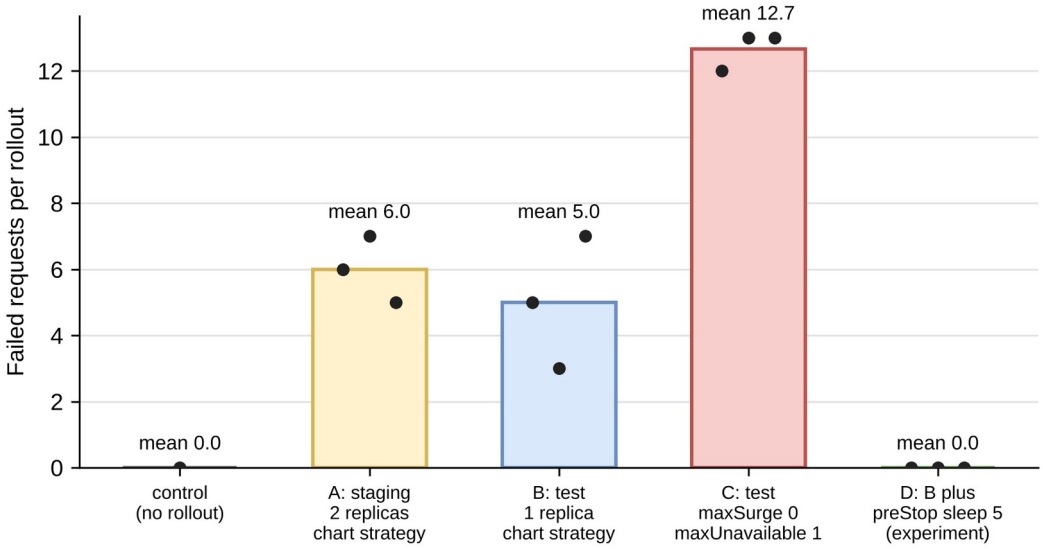


Figure 4.22 – Failed requests per rollout in the five scenarios (bars: mean; dots: individual runs)

4.9.3 Interpretation

1. **The chart's strategy loses few requests, but not none.** Although the number of available Pods never fell below N, 1.2 to 1.5 % of the requests failed. The failures occurred in short bursts of 1 to 4 s immediately after the new Pod had become ready, which is when the old Pod is shut down. The messages were *Connection refused*, *Connection reset by peer* and *download timed out*. A terminating Pod is removed from the endpoints of the Service while the container receives the termination signal, and for a short time the Service can still send a request to a Pod that no longer accepts connections [39].
2. **The loss scales with the number of Pods that are replaced.** In scenario A, with two replicas, every run shows two bursts about 7 s apart (for example at 16 s and 23 s in run A1), one for each replaced Pod; in scenario B there is one burst.
3. **The alternative strategy is worse.** With one replica and `maxUnavailable: 1` the old Pod is removed first, and the Service has no ready Pod until the new one passes its readiness probe. The failures form a single burst of 8 to 10 s. Helm reported the rollout as finished after only 2 to 3 s, although requests kept failing for several more seconds. The reason is the wait rule of Helm: `--wait` waits for a minimum number of ready Pods [22], and for a Deployment this minimum is the number of replicas minus `maxUnavailable` [40], which is zero for one replica and `maxUnavailable: 1`. With this strategy `--wait` and

`--atomic` could not detect a broken new revision, which is a second reason for the default `maxUnavailable: 0`.

4. **A preStop delay removed the failures in this experiment.** With `sleep 5` the old Pod keeps answering for 5 s after its deletion has started, which gives the endpoint update time to reach the Service. No request failed in three runs (1,285 requests). With zero failures in n = 1,285 requests, the rule of three gives an upper bound of 3/n = 0.23 % for the failure rate with 95 % confidence (level C).
5. **Consequence for the project.** The strategy of the chart is the better of the two that were compared, but it does not give *zero* downtime for a client that is connected while an old Pod is removed. For this reason the documentation of the project says that the rollout *keeps full capacity*, not that it has zero downtime. The `preStop` hook, together with a `terminationGracePeriodSeconds` and a graceful shutdown of gunicorn, is the first item of future work (Chapter V); it was not added to the chart because the finding was made after the chart had been tested, and the report describes the chart as delivered.

4.9.4 Limits of the Experiment

The experiment has clear limits. The cluster has one node; the probe runs inside the cluster and uses the cluster address of the Service, not a NodePort or an external load balancer; one client sends about nine requests per second; each scenario was repeated only three times; and a failing `wget` slows the client down, so the number of failed requests depends on the client and should be compared only within this experiment. A multi-node cluster may propagate endpoint changes more slowly, which would make the effect larger, not smaller. A first run of the experiment was discarded, because the command that waited for the probe Pod was given a timeout without a unit and returned at once, which truncated the logs; the script was corrected and the whole experiment was repeated, and only the second run is reported.

4.10 Image and Pipeline Results

Image. The application image `multi-env-app:1.0.0` is 126 MB (Figure 4.23). The Debian base layer (78.8 MB) and the Python runtime layers (40.6 MB) account for 95 % of it; Flask and gunicorn add 6.39 MB, and the application code and

the user add 27 kB. Because the dependencies are installed before the code is copied, a change of the code rebuilds only the last two, very small layers. The image build took 9 s in the validation workflow.

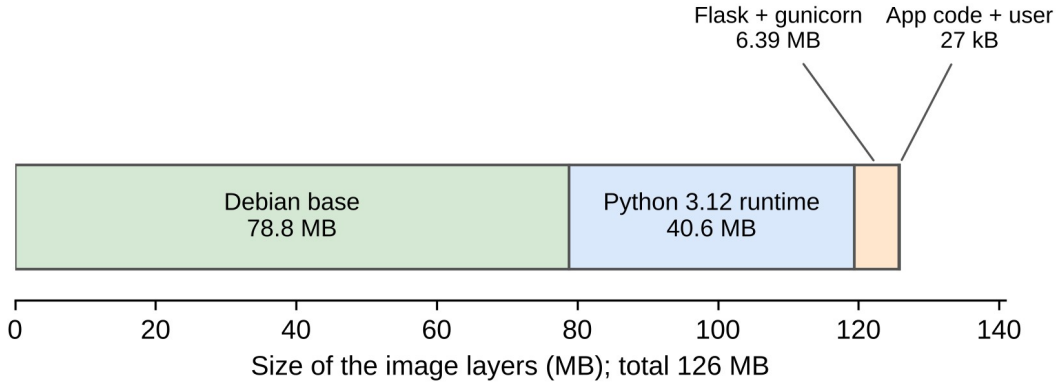


Figure 4.23 – Size of the layers of the application image (from `docker history`)

Pipeline. Figure 4.24 shows the timeline of the validation workflow for commit 9f0e6bc. The three independent jobs took 10 s, 17 s and 10 s and ran in parallel; the cluster job, which started when they had finished, took 143 s, of which the creation of the kind cluster was 41 s, the deployment of the four environments 40 s, the live checks 12 s, and the upgrade and rollback 29 s. The whole run needed 164 s from the start to the end. Table 4.11 lists the six runs of the workflow on the branch up to that commit; all six succeeded.

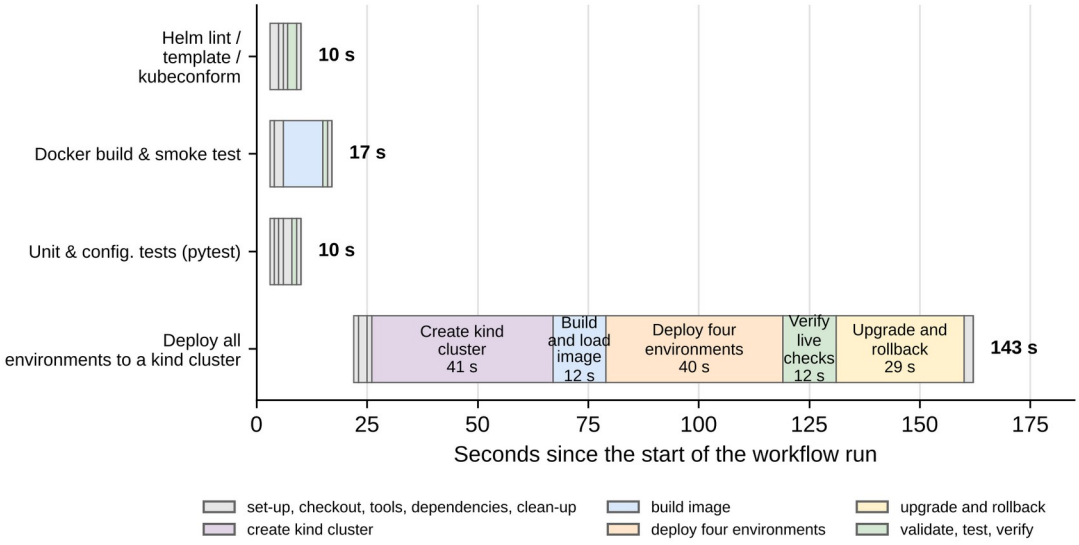


Figure 4.24 – Timeline of the four jobs of the validation workflow (run 6, commit 9f0e6bc)

Table 4.11 – Runs of the validation workflow up to commit 9f0e6bc

Run	Commit	Change	Result	Elapsed
1	80f1a57	Initial project: application, chart, scripts, tests, workflow	success	2 min 30 s
2	c0b2738	README, architecture documents, screenshots; Actions updated to Node 24 versions	success	2 min 27 s
3	9699de0	`docker-demo.sh` added	success	2 min 36 s
4	b08a794	Demonstration video added	success	7 min 22 s
5	961c078	Evidence script and workflow added	success	2 min 47 s
6	9f0e6bc	Availability experiment corrected; wording of the rollout changed	success	2 min 45 s

Elapsed time is the time between the creation and the last update of the run as reported by the GitHub API. The longer time of run 4 was not investigated.

The workflow `capture-evidence.yml` (run 2) took 15 min 41 s. Most of this time was the availability experiment, which consists of 13 runs of about one minute each.

4.11 Discussion

4.11.1 Objectives and Results

Table 4.12 compares the objectives of Section 1.6 with the results. Nine of the ten objectives were achieved as stated; objective O6 was achieved on a kind cluster, and the Minikube path is supported by the scripts but was not executed (level D).

Table 4.12 – Objectives compared with the results

No.	Objective	Result	Evidence
O1	Flask, HTML, CSS and Bootstrap application	Achieved	Dashboards (Section 4.3); 15 application tests
O2	Dashboard with name, environment, version and configuration	Achieved	Table 4.4: 56 of 56 values equal to the files
O3	Four environments	Achieved	Table 4.3; Figure 4.1
O4	Reusable chart with the required files	Achieved	Table 3.6; lint, render and kubeconform valid for four environments
O5	Docker configuration	Achieved	Image of 126 MB; smoke test in CI; non-root user
O6	Kubernetes deployment on Minikube	Achieved on kind; Minikube not executed	Section 4.1; scripts `minikube-setup.sh`, `deploy.sh`
O7	Install, validation, upgrade and rollback with Helm	Achieved	TC-04, TC-09, TC-10; Section 4.7
O8	GitHub Actions validation	Achieved	Table 4.11; Figure 4.24
O9	Test scripts for environment-specific configuration	Achieved	62 pytest cases; 26 live checks; 4 `helm test` runs
O10	No hard-coded secrets; consistent configuration	Achieved within the stated limits	Table 3.18; Section 4.6

4.11.2 Findings

1. **Separation of configuration works.** One set of templates and four values files of 28 to 43 lines produced four different environments; all 56 compared values of the running Pods equalled the values of the files (Section 4.3), and the model of Section 4.5 shows that the chart is smaller than copied manifests from the third environment onwards.
2. **Validation before deployment is effective and cheap.** The schema rejected all 14 invalid inputs before any cluster was contacted. Its weak point is the message for forbidden keys.
3. **Release history makes changes reversible.** The rollback restored version 1.0.0 and the level DEBUG as revision 3, and `--atomic` restored revision 1 after a failed upgrade while the old Pod kept serving.
4. **“Zero downtime” needs a measurement.** A rolling update with `maxUnavailable: 0` keeps capacity but lost 1.2 to 1.5 % of the requests in short bursts; a `preStop` delay removed these failures in three of three runs. The claim was therefore weakened to *keeps full capacity* throughout the project.
5. **Helm does not see a missing Secret.** The chart refers to the Secret by name; the failure is visible only in the events of Kubernetes. The deployment script creates the Secret first.
6. **Strict types cost a little convenience.** Numeric settings must be given with `--set-string`; in return, the type of every value is explicit.

The findings are consistent with the literature of Chapter II in two respects. Misconfigurations are frequent and often consist of illegal values [2], and the schema addresses exactly this class by turning the rules of a parameter into a check that runs before deployment. The security studies of Kubernetes manifests [16] report missing resource limits and weak security contexts, which the chart avoids by default; the project does not, however, claim to have removed all misconfigurations of this kind.

4.11.3 Threats to Validity

- **Platform.** The cluster runs were made on one single-node kind cluster on one type of CI runner, and Minikube and Windows were not executed. Behaviour on other platforms, in particular the timing of the availability experiment, may differ.

- **Synthetic application.** The application has no real database or traffic; settings such as `DATABASE_HOST` are illustrative. The results about configuration management transfer more easily than the results about request failures.
- **Sample size.** Each availability scenario was repeated three times with one client. The ranges of the scenarios do not overlap (B: 3–7, C: 12–13, D: 0), but no statistical test was applied.
- **Proxy measures.** Line counts are a proxy for effort, and the baseline of copied manifests is hypothetical.
- **Author-written tests.** The tests and the schema cases were written by the authors of the code and may share their blind spots.
- **Version skew.** kubeconform checked the manifests against the schemas of Kubernetes 1.31.0, and the cluster ran v1.35.0. The chart uses only the stable APIs `v1` and `apps/v1`, so the risk is small, but it was not eliminated.

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

The project set out to show that one application can be deployed to development, testing, staging and production from a single Helm chart, without copying manifests, and that the result can be validated, tested, upgraded and reverted. The delivered solution consists of a Flask dashboard with Bootstrap, one container image of 126 MB, the chart `multi-env-app` with one file of default values, four environment files of 28 to 43 lines, a JSON Schema for the values, nine scripts, 62 automated tests and a GitHub Actions workflow of four jobs.

The solution was run on a kind cluster in GitHub Actions. All four releases were deployed in 40 s; every one of the 56 compared settings of the running Pods equalled the values of the files; all 26 live checks, which include the four chart tests, passed; a rollback restored an earlier revision as a new revision; and `--atomic` restored a release after an upgrade to an image that does not exist. The schema rejected all 14 invalid inputs before any cluster was contacted, and the model of the effort shows that the chart is smaller than copied manifests from the third environment onwards.

The report also records two results that go against a simple story. A rolling update with `maxUnavailable: 0` keeps full capacity, but 1.5 % and 1.2 % of the requests failed in short bursts while old Pods were removed, whereas a `preStop` delay removed the failures in the experiment; “zero downtime” is therefore not claimed anywhere in the project. And Helm cannot detect that a Secret is missing, because the chart refers to it only by name. The report states for every result whether it was run on a cluster, run locally or derived, and it lists what was not executed, most importantly Minikube.

5.2 Summary of Contributions

1. A small, complete and reproducible reference for multi-environment configuration management with Helm, which can be read in one sitting and run on a laptop cluster.

2. A typed interface for the values, enforced by a JSON Schema, that rejects unknown environments, wrong types, out-of-range numbers and secrets placed in the configuration.
3. A deployment and release-management workflow with `--atomic`, history and rollback, with scripts for each step and a live test of every environment.
4. A continuous-integration workflow that validates the chart statically and deploys all four environments to a real cluster on every change.
5. Measurements that go beyond a demonstration: two forced failures, and a rolling-update experiment with a control, a comparison of two strategies and a mitigation.
6. An explicit statement of the evidence level of every result.

5.3 Comparison with the Literature

Table 5.1 relates the observations of this report to the sources of Chapter II. The relation is one of consistency, not of replication: the project is a single small example, and the studies cited analyse hundreds or thousands of real repositories.

Table 5.1 – Observations of this project compared with the literature

Source	What the literature reports	What this project observed	Relation
[1]	Configuration that varies between deploys belongs in the environment, apart from the code	All settings come from environment variables; one image runs in all four environments	Followed
[2]	Many misconfigurations are illegal parameter values	The schema rejected 14 of 14 invalid inputs before deployment	Consistent
[3]	Prevention, detection and diagnosis are separate tasks	Schema (prevention), lint, tests and CI (detection); diagnosis relied on events and logs, and the missing Secret showed a gap	Partly addressed
[14, 15]	Configuration code is defect-prone and needs testing	62 tests and four validation stages; the rollout failures were found by an experiment, not by a test	Consistent
[16]	Missing limits and weak security contexts are common	Limits in every environment, non-root user, all capabilities dropped; no policy scanner	Addressed within limits
[13]	Public charts contain outdated and vulnerable components	Versions pinned and slim base image; no vulnerability scan was run	Not verified
[4, 5, 19]	Desired state in version control and an automated delivery pipeline	All files in Git; one workflow with four jobs and six successful runs; no GitOps controller	Partly (pipeline, no controller)

5.4 Practical Deployment Considerations

The project is a teaching example. Before a similar chart is used for a real application, the following points should be considered.

- **Separate clusters.** Namespaces separate names, not resources or failure domains. Production usually runs in its own cluster; the chart does not change for this, because only the values file and the cluster that `helm` addresses differ.
- **Secrets.** Use an external secrets operator or sealed secrets, enable encryption at rest and restrict who may read Secrets.
- **Sizing.** The requests and limits are demonstration values. Measure the application under load and tune them per environment.
- **Traffic.** Place an Ingress with TLS in front of the Service, add a disruption budget, and add a graceful shutdown to avoid request loss during rollouts (Section 4.9).
- **Promotion.** Promote the same chart version and the same image tag from environment to environment, and record both in the pipeline.
- **Observation.** Add metrics, structured logs and alerts; the probe of the availability experiment could become a permanent synthetic check.

5.5 Limitations

- Minikube, Windows, multi-node and cloud clusters were not executed. The scripts support Minikube, but its results are level D (Section 4.1).
- The application is synthetic and stateless; there is no database, no real traffic and no persistent storage.
- The chart contains no Ingress, TLS, autoscaling, disruption budget or network policy, and the production Secret is created outside the chart by a script. Kubernetes Secrets are not encrypted by default.
- The rollout of the chart is not free of request failures (Section 4.9); the `preStop` hook that removed them is not part of the chart.
- The image was not scanned for vulnerabilities and the base image is selected by tag, not by digest.
- The availability experiment is small (one single-node cluster, one client, three runs per scenario).

5.6 Future Enhancements

Figure 5.1 orders the enhancements by effort and dependency, and Table 5.2 gives the reason for each of the first ones, taken from the results of this report. The

first two items follow directly from the two failures found in Section 4.8 and Section 4.9.

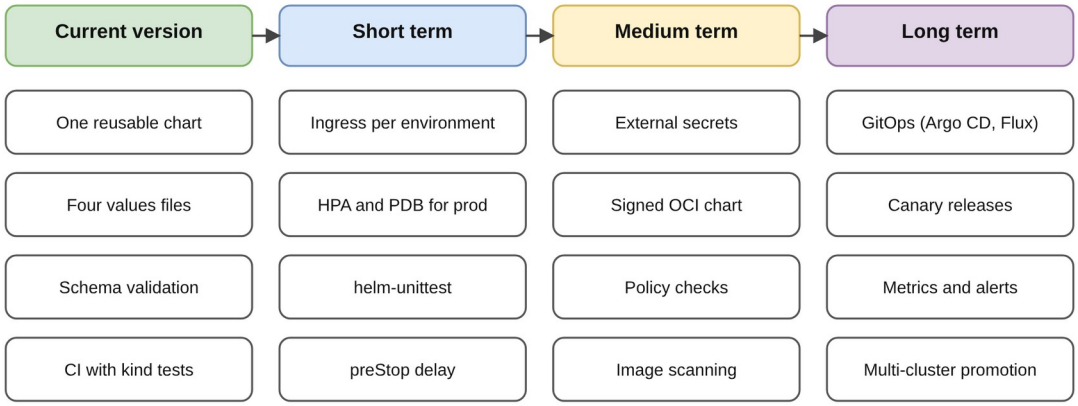


Figure 5.1 – Roadmap of future enhancements

Table 5.2 – Future enhancements and their reasons

Enhancement	Reason (evidence in this report)	Outline
Graceful shutdown: `preStop` delay, `terminationGracePeriodSeconds`, handling of SIGTERM by gunicorn	1.2 to 1.5 % of the requests failed during rollouts; none with a 5 s delay (Section 4.9)	Add a `lifecycle.preStop` value to the chart; repeat the experiment with the delivered chart
Check or provide the Secret	Helm did not detect the missing Secret (Section 4.8.2)	A `lookup` check, a pre-install hook, or an external secrets operator
Ingress per environment with TLS	The dashboards are reached by NodePort only	One Ingress host per environment; certificates by cert-manager
HPA and PDB for production	Three fixed replicas; no protection against node drains	`HorizontalPodAutoscaler` and `PodDisruptionBudget` in `values-prod.yaml`
Unit tests for templates	Template logic is tested through `helm template` and pytest	`helm-unittest` suites for the three templates
Image scanning and a signed OCI chart	The image was not scanned; the base image is selected by tag	Trivy in CI; `helm push` to an OCI registry; signatures
Policy checks	Hardening is checked by hand-written tests	Conftest or Kyverno policies on the rendered manifests
GitOps controller	Deployments are started by scripts and CI	Argo CD or Flux reconcile the values files from Git
Metrics and alerts; canary releases; multi-cluster promotion	Failures were found by a probe written for this report	Prometheus metrics, alert rules, progressive delivery
Run on Minikube, Windows, a multi-node and a cloud cluster	These platforms are level D	Repeat the evidence workflow on each platform

None of these enhancements changes the central idea of the project: the application is described once, the differences between the environments are data, the data are validated before deployment, and every deployment is a numbered revision that can be reverted.

APPENDICES

Appendix A – Directory structure of the project

The project consists of the application, the container file, the Helm chart, the scripts, the tests and the workflows. The line count of each file is given at the right. The files in the folder `docs/` (the diagram, the video and the evidence files) are not listed.

Code Snippet – Directory structure (git ls-files)

```
multi-env-helm-config/
├── .github/
│   └── workflows/
│       ├── capture-evidence.yml                      56 lines
│       └── helm-validation.yml                      117 lines
├── app/
│   ├── static/
│   │   └── css/
│   │       └── style.css                             43 lines
│   ├── templates/
│   │   └── index.html                               111 lines
│   ├── app.py                                        73 lines
│   ├── config.py                                    101 lines
│   └── requirements.txt                               2 lines
├── helm/
│   └── multi-env-app/
│       ├── templates/
│       │   ├── tests/
│       │   │   └── test-connection.yaml              27 lines
│       │   ├── _helpers.tpl                          48 lines
│       │   ├── configmap.yaml                        16 lines
│       │   ├── deployment.yaml                       77 lines
│       │   ├── NOTES.txt                             17 lines
│       │   └── service.yaml                          18 lines
│       ├── .helmignore                               10 lines
│       ├── Chart.yaml                                18 lines
│       ├── values-dev.yaml                           28 lines
│       ├── values-prod.yaml                          43 lines
│       ├── values-staging.yaml                       28 lines
│       ├── values-test.yaml                          28 lines
│       ├── values.schema.json                        75 lines
│       └── values.yaml                               98 lines
├── scripts/
│   ├── capture-evidence.sh                          238 lines
│   ├── cleanup.sh                                    12 lines
│   ├── common.sh                                     43 lines
│   ├── deploy.sh                                     49 lines
│   ├── docker-demo.sh                                63 lines
│   ├── minikube-setup.sh                             25 lines
│   ├── test-environments.sh                          68 lines
│   ├── upgrade-rollback-demo.sh                      50 lines
│   └── validate-helm.sh                              45 lines
├── tests/
│   ├── conftest.py                                    9 lines
│   ├── test_app.py                                   93 lines
│   └── test_helm_config.py                          230 lines
├── .dockerignore                                     13 lines
├── Dockerfile                                        31 lines
├── README.md                                        447 lines
└── requirements-dev.txt                               3 lines

docs/                                    diagram, video, evidence files
```

Appendix B – Application source code

The listings in this and the following appendices are the complete files of the repository. They were not typed: a program reads each file and writes its lines, with the line numbers of the file, into the report, and the text of the finished document was afterwards compared with the files.

Code Snippet – app/app.py (complete, 73 lines)

```
"""Flask web application that displays its environment-specific configuration.

Endpoints:
    /             HTML dashboard (Bootstrap)
    /health       Liveness probe
    /ready        Readiness probe
    /api/config   JSON view of the (non-secret) configuration
"""

import logging
import os
from datetime import datetime, timezone

from flask import Flask, jsonify, render_template

from config import AppConfig

STARTED_AT = datetime.now(timezone.utc)


def create_app(environ=None):
    cfg = AppConfig(environ)

    app = Flask(__name__)
    app.config["SECRET_KEY"] = cfg.secret_key
    app.config["APP_CONFIG"] = cfg

    logging.basicConfig(
        level=getattr(logging, cfg.log_level, logging.INFO),
        format="%(asctime)s %(levelname)s [%(name)s] %(message)s",
    )
    log = logging.getLogger("multi-env-app")
    if not cfg.environment_is_valid:
        log.warning("APP_ENV is not one of the supported environments; defaulting to 'development'")
    log.info("Starting %s v%s in %s environment", cfg.app_name, cfg.app_version, cfg.environment)

    @app.route("/")
    def dashboard():
        return render_template(
            "index.html",
            cfg=cfg,
            settings=cfg.public_settings(),
            runtime=cfg.runtime_info(),
            started_at=STARTED_AT.strftime("%Y-%m-%d %H:%M:%S UTC"),
        )

    @app.route("/health")
    def health():
        return jsonify(status="ok"), 200

    @app.route("/ready")
    def ready():
        return jsonify(status="ready", environment=cfg.environment), 200

    @app.route("/api/config")
    def api_config():
        return jsonify(
            application=cfg.app_name,
            environment=cfg.environment,
            version=cfg.app_version,
            settings=cfg.public_settings(),
            runtime=cfg.runtime_info(),
```

```
        )

    return app


app = create_app()


if __name__ == "__main__":
    # Local development server only. In containers the app runs under gunicorn.
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", "5000")),
      debug=app.config["APP_CONFIG"].debug)
```

Code Snippet – app/config.py (complete, 101 lines)

```
"""Runtime configuration for the Multi-Environment Config Dashboard.

All values are read from environment variables. In Kubernetes these
variables are injected from the Helm-generated ConfigMap (non-sensitive
settings) and, optionally, from a pre-created Kubernetes Secret.
Nothing sensitive is hardcoded here.
"""

import os
import secrets

VALID_ENVIRONMENTS = ("development", "testing", "staging", "production")

# Bootstrap colour used for the environment badge / banner on the dashboard.
ENVIRONMENT_THEMES = {
    "development": "success",
    "testing": "info",
    "staging": "warning",
    "production": "danger",
}

# Keys whose values must never be displayed or returned by the API.
SENSITIVE_KEYS = ("SECRET_KEY",)


def _as_bool(value, default=False):
    if value is None:
        return default
    return str(value).strip().lower() in ("1", "true", "yes", "on")


def _as_int(value, default):
    try:
        return int(value)
    except (TypeError, ValueError):
        return default


class AppConfig:
    """Snapshot of the application configuration taken from the environment."""

    def __init__(self, environ=None):
        env = os.environ if environ is None else environ

        self.app_name = env.get("APP_NAME", "Multi-Env Config Dashboard")
        raw_env = env.get("APP_ENV", "development").strip().lower()
        self.environment_is_valid = raw_env in VALID_ENVIRONMENTS
        self.environment = raw_env if self.environment_is_valid else
   "development"
        self.app_version = env.get("APP_VERSION", "1.0.0")

        self.log_level = env.get("LOG_LEVEL", "INFO").upper()
        self.debug = _as_bool(env.get("DEBUG_MODE"), default=False)
        self.feature_beta_banner = _as_bool(env.get("FEATURE_BETA_BANNER"))
        self.max_connections = _as_int(env.get("MAX_CONNECTIONS"), 10)
        self.cache_ttl_seconds = _as_int(env.get("CACHE_TTL_SECONDS"), 60)
        self.database_host = env.get("DATABASE_HOST", "localhost")
        self.api_base_url = env.get("API_BASE_URL", "http://localhost:5000/api")
        self.replica_count = env.get("REPLICA_COUNT", "1")

        # Kubernetes downward-API values (present only inside a pod).
        self.pod_name = env.get("POD_NAME", "local")
        self.pod_namespace = env.get("POD_NAMESPACE", "local")
        self.node_name = env.get("NODE_NAME", "local")
```

```
            self.helm_release = env.get("HELM_RELEASE", "not-deployed-with-helm")
            self.chart_version = env.get("HELM_CHART_VERSION", "n/a")

            # Secret key: supplied via a Kubernetes Secret, or generated per
            # process so that no secret is ever committed to source control.
            provided_secret = env.get("SECRET_KEY")
            self.secret_key_source = "kubernetes-secret" if provided_secret else
        "auto-generated"
            self.secret_key = provided_secret or secrets.token_hex(32)

        @property
        def theme(self):
            return ENVIRONMENT_THEMES[self.environment]

        def public_settings(self):
            """Settings that are safe to show on the dashboard / JSON API."""
            return {
                "APP_NAME": self.app_name,
                "APP_ENV": self.environment,
                "APP_VERSION": self.app_version,
                "LOG_LEVEL": self.log_level,
                "DEBUG_MODE": self.debug,
                "FEATURE_BETA_BANNER": self.feature_beta_banner,
                "MAX_CONNECTIONS": self.max_connections,
                "CACHE_TTL_SECONDS": self.cache_ttl_seconds,
                "DATABASE_HOST": self.database_host,
                "API_BASE_URL": self.api_base_url,
                "REPLICA_COUNT": self.replica_count,
                "SECRET_KEY": "******** ({})".format(self.secret_key_source),
            }

        def runtime_info(self):
            return {
                "POD_NAME": self.pod_name,
                "POD_NAMESPACE": self.pod_namespace,
                "NODE_NAME": self.node_name,
                "HELM_RELEASE": self.helm_release,
                "HELM_CHART_VERSION": self.chart_version,
            }
```

Code Snippet – app/templates/index.html (complete, 111 lines)

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ cfg.app_name }} - {{ cfg.environment|capitalize }}</title>
  <link rel="stylesheet"
   href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
   >
  <link rel="stylesheet" href="{{ url_for('static',
   filename='css/style.css') }}">
</head>
<body class="env-{{ cfg.environment }}">
  <nav class="navbar navbar-dark bg-{{ cfg.theme }} env-banner">
    <div class="container">
      <span class="navbar-brand fw-bold">{{ cfg.app_name }}</span>
      <span class="badge bg-light text-dark fs-6" id="env-
   badge">{{ cfg.environment|upper }}</span>
    </div>
  </nav>

  {% if cfg.feature_beta_banner %}
  <div class="alert alert-secondary rounded-0 mb-0 text-center" role="alert">
    Beta features are enabled in this environment.
  </div>
  {% endif %}

  <main class="container py-4">
    <div class="row g-3 mb-4">
      <div class="col-md-3 col-sm-6">
        <div class="card stat-card border-{{ cfg.theme }} h-100">
          <div class="card-body">
            <div class="stat-label">Application</div>
            <div class="stat-value" id="app-name">{{ cfg.app_name }}</div>
```

```
            </div>
          </div>
        </div>
        <div class="col-md-3 col-sm-6">
          <div class="card stat-card border-{{ cfg.theme }} h-100">
            <div class="card-body">
              <div class="stat-label">Environment</div>
              <div class="stat-value text-{{ cfg.theme }}" id="app-
env">{{ cfg.environment|capitalize }}</div>
            </div>
          </div>
        </div>
        <div class="col-md-3 col-sm-6">
          <div class="card stat-card border-{{ cfg.theme }} h-100">
            <div class="card-body">
              <div class="stat-label">Version</div>
              <div class="stat-value" id="app-version">v{{ cfg.app_version }}</div>
            </div>
          </div>
        </div>
        <div class="col-md-3 col-sm-6">
          <div class="card stat-card border-{{ cfg.theme }} h-100">
            <div class="card-body">
              <div class="stat-label">Replicas (desired)</div>
              <div class="stat-value">{{ cfg.replica_count }}</div>
            </div>
          </div>
        </div>
      </div>

      <div class="row g-4">
        <div class="col-lg-7">
          <div class="card shadow-sm">
            <div class="card-header bg-{{ cfg.theme }} text-white fw-semibold">
              Configuration (from Helm ConfigMap)
            </div>
            <div class="table-responsive">
              <table class="table table-striped mb-0 config-table">
                <thead><tr><th>Key</th><th>Value</th></tr></thead>
                <tbody>
                {% for key, value in settings.items() %}
                  <tr><td><code>{{ key }}</code></td><td>{{ value }}</td></tr>
                {% endfor %}
                </tbody>
              </table>
            </div>
          </div>
        </div>

        <div class="col-lg-5">
          <div class="card shadow-sm mb-4">
            <div class="card-header fw-semibold">Kubernetes Runtime</div>
            <ul class="list-group list-group-flush">
              {% for key, value in runtime.items() %}
              <li class="list-group-item d-flex justify-content-between">
                <span><code>{{ key }}</code></span><span class="text-break text-
end">{{ value }}</span>
              </li>
              {% endfor %}
              <li class="list-group-item d-flex justify-content-between">
                <span><code>STARTED_AT</code></span><span>{{ started_at }}</span>
              </li>
            </ul>
          </div>

          <div class="card shadow-sm">
            <div class="card-header fw-semibold">Endpoints</div>
            <ul class="list-group list-group-flush">
              <li class="list-group-item"><a href="/api/config">/api/config</a> -
configuration as JSON</li>
              <li class="list-group-item"><a href="/health">/health</a> - liveness
probe</li>
              <li class="list-group-item"><a href="/ready">/ready</a> - readiness
probe</li>
            </ul>
          </div>
        </div>
```

```
      </div>
    </main>

    <footer class="text-center text-muted small py-3 border-top">
      Multi-Environment Configuration Management with Helm &middot;
      Arun Kumar T &amp;amp; Priyan M &middot; Dept. of AI &amp;amp; DS, AAA College of
     Engineering and Technology, Sivakasi
    </footer>
  </body>
  </html>
```

Code Snippet – app/static/css/style.css (complete, 43 lines)

```
/* Custom styles layered on top of Bootstrap 5. */

body {
  background-color: #f4f6f9;
  font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
}

.env-banner {
  padding-top: 0.9rem;
  padding-bottom: 0.9rem;
}

.stat-card {
  border-width: 0 0 0 5px;
  border-style: solid;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
}

.stat-label {
  font-size: 0.8rem;
  text-transform: uppercase;
  letter-spacing: 0.05em;
  color: #6c757d;
}

.stat-value {
  font-size: 1.35rem;
  font-weight: 600;
  word-break: break-word;
}

.config-table td:first-child {
  width: 45%;
}

/* Production gets a subtle warning stripe so it is never mistaken for dev. */
body.env-production .env-banner {
  background-image: repeating-linear-gradient(
    45deg,
    rgba(255, 255, 255, 0.06) 0 12px,
    transparent 12px 24px
  );
}
```

Code Snippet – app/requirements.txt (complete, 2 lines)

```
Flask==3.0.3
gunicorn==23.0.0
```

Appendix C – Container files

Code Snippet – Dockerfile (complete, 31 lines)

```
# ------------------------------------------------------------------
# Multi-Environment Config Dashboard - container image
# The same image is promoted through dev -> test -> staging -> prod;
# only the Helm values (ConfigMap) change between environments.
# ------------------------------------------------------------------
FROM python:3.12-slim

```

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1 \
    PORT=5000

WORKDIR /app

# Install dependencies first to make good use of the layer cache.
COPY app/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ .

# Run as an unprivileged user (UID matches the Helm securityContext).
RUN useradd --uid 10001 --no-create-home --shell /usr/sbin/nologin appuser \
    && chown -R appuser /app
USER 10001

EXPOSE 5000

HEALTHCHECK --interval=30s --timeout=3s --start-period=10s --retries=3 \
  CMD python -c "import urllib.request,sys; sys.exit(0 if
   urllib.request.urlopen('http://127.0.0.1:5000/health').status == 200 else 1)"

CMD ["sh", "-c", "exec gunicorn --bind 0.0.0.0:${PORT} --workers 2 --access-
   logfile - app:app"]
```

Code Snippet – .dockerignore (complete, 13 lines)

```
.git
.github
helm
docs
scripts
tests
dist
**/__pycache__
**/*.pyc
.pytest_cache
.venv
venv
*.md
```

Appendix D – Helm chart

The chart is listed in the order in which Helm reads it: metadata, defaults, the four environment files, the schema, the helpers and the templates.

Code Snippet – helm/multi-env-app/Chart.yaml (complete, 18 lines)

```
apiVersion: v2
name: multi-env-app
description: >-
  Reusable Helm chart that deploys the Multi-Environment Config Dashboard
  (Flask) to Development, Testing, Staging and Production using
  environment-specific values files.
type: application
# Chart version - bump whenever the chart templates change.
version: 1.0.0
# Application version - used as the default image tag and APP_VERSION.
appVersion: "1.0.0"
keywords:
  - flask
  - multi-environment
  - configuration-management
maintainers:
  - name: Arun Kumar T
  - name: Priyan M
```

Code Snippet – helm/multi-env-app/values.yaml (complete, 98 lines)

```
# -------------------------------------------------------------------------
# Default (base) values for the multi-env-app chart.
# Environment files (values-dev.yaml, values-test.yaml, values-staging.yaml,
# values-prod.yaml) override only what differs from these defaults:
#
#   helm install myapp-dev ./helm/multi-env-app -f ./helm/multi-env-app/values-dev.yaml
#
# Do NOT put passwords, tokens or keys here. Sensitive values belong in a
# Kubernetes Secret that is referenced through `existingSecret`.
# -------------------------------------------------------------------------

# One of: development, testing, staging, production
environment: development

replicaCount: 1

image:
  repository: multi-env-app
  # Defaults to .Chart.AppVersion when empty.
  tag: ""
  pullPolicy: IfNotPresent

nameOverride: ""
fullnameOverride: ""

app:
  name: "Multi-Env Config Dashboard"
  # Version shown on the dashboard. Defaults to the image tag, then appVersion.
  version: ""

# Non-sensitive settings. Every key becomes an environment variable via the
# ConfigMap (templates/configmap.yaml). Values are always rendered as strings.
config:
  LOG_LEVEL: "INFO"
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "false"
  MAX_CONNECTIONS: "10"
  CACHE_TTL_SECONDS: "60"
  DATABASE_HOST: "localhost"
  API_BASE_URL: "http://localhost:5000/api"

# Name of a pre-existing Secret whose keys are exposed as env vars
# (for example SECRET_KEY). Leave empty to let the app generate a random key.
#   kubectl create secret generic myapp-secrets -n prod --from-literal=SECRET_KEY=$(openssl rand -hex 32)
existingSecret: ""

containerPort: 5000

service:
  type: NodePort
  port: 80
  # Fixed NodePort (30000-32767) so each environment has a predictable URL.
  # Leave empty to let Kubernetes choose one.
  nodePort: ""

strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 200m
    memory: 128Mi

livenessProbe:
  httpGet:
    path: /health
    port: http
  initialDelaySeconds: 5
  periodSeconds: 10
```

```

readinessProbe:
  httpGet:
    path: /ready
    port: http
  initialDelaySeconds: 3
  periodSeconds: 5

podAnnotations: {}

podSecurityContext:
  runAsNonRoot: true
  runAsUser: 10001
  fsGroup: 10001

securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]

# Image used by `helm test` (templates/tests/test-connection.yaml).
tests:
  image: busybox:1.36
```

Code Snippet – helm/multi-env-app/values-dev.yaml (complete, 28 lines)

```
# Development: fast feedback, verbose logging, single replica, beta features on.
environment: development

replicaCount: 1

app:
  name: "Multi-Env Config Dashboard (DEV)"

config:
  LOG_LEVEL: "DEBUG"
  DEBUG_MODE: "true"
  FEATURE_BETA_BANNER: "true"
  MAX_CONNECTIONS: "5"
  CACHE_TTL_SECONDS: "10"
  DATABASE_HOST: "postgres.dev.svc.cluster.local"
  API_BASE_URL: "http://dev.multi-env-app.local/api"

service:
  type: NodePort
  nodePort: 30081

resources:
  requests:
    cpu: 25m
    memory: 64Mi
  limits:
    cpu: 100m
    memory: 128Mi
```

Code Snippet – helm/multi-env-app/values-test.yaml (complete, 28 lines)

```
# Testing: mirrors dev features but with INFO logging for QA test runs.
environment: testing

replicaCount: 1

app:
  name: "Multi-Env Config Dashboard (TEST)"

config:
  LOG_LEVEL: "INFO"
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "true"
  MAX_CONNECTIONS: "10"
  CACHE_TTL_SECONDS: "30"
  DATABASE_HOST: "postgres.test.svc.cluster.local"
  API_BASE_URL: "http://test.multi-env-app.local/api"

service:
  type: NodePort
  nodePort: 30082
```

```

resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 150m
    memory: 128Mi
```

Code Snippet – helm/multi-env-app/values-staging.yaml (complete, 28 lines)

```
# Staging: production-like (no debug, no beta) with 2 replicas for HA testing.
environment: staging

replicaCount: 2

app:
  name: "Multi-Env Config Dashboard (STAGING)"

config:
  LOG_LEVEL: "INFO"
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "false"
  MAX_CONNECTIONS: "50"
  CACHE_TTL_SECONDS: "120"
  DATABASE_HOST: "postgres.staging.svc.cluster.local"
  API_BASE_URL: "https://staging.multi-env-app.example.com/api"

service:
  type: NodePort
  nodePort: 30083

resources:
  requests:
    cpu: 100m
    memory: 96Mi
  limits:
    cpu: 250m
    memory: 192Mi
```

Code Snippet – helm/multi-env-app/values-prod.yaml (complete, 43 lines)

```
# Production: high availability, minimal logging, debug disabled.
# Sensitive values (SECRET_KEY) come from a pre-created Kubernetes Secret.
environment: production

replicaCount: 3

image:
  pullPolicy: IfNotPresent

app:
  name: "Multi-Env Config Dashboard"

config:
  LOG_LEVEL: "WARNING"
  DEBUG_MODE: "false"
  FEATURE_BETA_BANNER: "false"
  MAX_CONNECTIONS: "200"
  CACHE_TTL_SECONDS: "300"
  DATABASE_HOST: "postgres.prod.svc.cluster.local"
  API_BASE_URL: "https://multi-env-app.example.com/api"

# Create before installing (see README):
#   kubectl create secret generic multi-env-app-secrets -n prod \
#     --from-literal=SECRET_KEY="$(openssl rand -hex 32)"
existingSecret: "multi-env-app-secrets"

service:
  type: NodePort
  nodePort: 30084

strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 0
```

```

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi
```

Code Snippet – helm/multi-env-app/values.schema.json (complete, 75 lines)

```
{
  "$schema": "https://json-schema.org/draft-07/schema#",
  "title": "multi-env-app values",
  "type": "object",
  "required": ["environment", "replicaCount", "image", "app", "config",
   "service"],
  "properties": {
    "environment": {
      "type": "string",
      "enum": ["development", "testing", "staging", "production"],
      "description": "Target environment"
    },
    "replicaCount": {
      "type": "integer",
      "minimum": 1,
      "maximum": 10
    },
    "image": {
      "type": "object",
      "required": ["repository", "pullPolicy"],
      "properties": {
        "repository": { "type": "string", "minLength": 1 },
        "tag": { "type": "string" },
        "pullPolicy": { "type": "string", "enum": ["Always", "IfNotPresent",
   "Never"] }
      }
    },
    "app": {
      "type": "object",
      "required": ["name"],
      "properties": {
        "name": { "type": "string", "minLength": 1 },
        "version": { "type": "string" }
      }
    },
    "config": {
      "type": "object",
      "required": ["LOG_LEVEL", "DEBUG_MODE"],
      "properties": {
        "LOG_LEVEL": { "type": "string", "enum": ["DEBUG", "INFO", "WARNING",
   "ERROR", "CRITICAL"] },
        "DEBUG_MODE": { "type": "string", "enum": ["true", "false"] },
        "FEATURE_BETA_BANNER": { "type": "string", "enum": ["true", "false"] },
        "MAX_CONNECTIONS": { "type": "string", "pattern": "^[0-9]+$" },
        "CACHE_TTL_SECONDS": { "type": "string", "pattern": "^[0-9]+$" }
      },
      "propertyNames": { "pattern": "^[A-Z][A-Z0-9_]*$" },
      "not": {
        "anyOf": [
          { "required": ["SECRET_KEY"] },
          { "required": ["PASSWORD"] },
          { "required": ["DB_PASSWORD"] },
          { "required": ["API_TOKEN"] }
        ]
      }
    },
    "existingSecret": { "type": "string" },
    "containerPort": { "type": "integer", "minimum": 1, "maximum": 65535 },
    "service": {
      "type": "object",
      "required": ["type", "port"],
      "properties": {
        "type": { "type": "string", "enum": ["ClusterIP", "NodePort",
   "LoadBalancer"] },
        "port": { "type": "integer", "minimum": 1, "maximum": 65535 },
```

```
        "nodePort": {
          "oneOf": [
            { "type": "string", "maxLength": 0 },
            { "type": "integer", "minimum": 30000, "maximum": 32767 }
          ]
        }
      }
    },
    "tests": {
      "type": "object",
      "properties": { "image": { "type": "string", "minLength": 1 } }
    }
  }
}
```

Code Snippet – helm/multi-env-app/templates/_helpers.tpl (complete, 48 lines)

```
{{/* Chart name, overridable with .Values.nameOverride. */}}
{{- define "multi-env-app.name" -}}
{{- default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{/*
Fully qualified app name. Uses the release name so several environments
can live side by side (myapp-dev, myapp-test, ...).
*/}}
{{- define "multi-env-app.fullname" -}}
{{- if .Values.fullnameOverride }}
{{- .Values.fullnameOverride | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- $name := default .Chart.Name .Values.nameOverride }}
{{- if contains $name .Release.Name }}
{{- .Release.Name | trunc 63 | trimSuffix "-" }}
{{- else }}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" }}
{{- end }}
{{- end }}
{{- end }}

{{- define "multi-env-app.chart" -}}
{{- printf "%s-%s" .Chart.Name .Chart.Version | replace "+" "_" | trunc 63 |
   trimSuffix "-" }}
{{- end }}

{{/* Image tag: explicit value or the chart's appVersion. */}}
{{- define "multi-env-app.imageTag" -}}
{{- .Values.image.tag | default .Chart.AppVersion }}
{{- end }}

{{/* Application version shown on the dashboard. */}}
{{- define "multi-env-app.appVersion" -}}
{{- .Values.app.version | default (include "multi-env-app.imageTag" .) }}
{{- end }}

{{- define "multi-env-app.selectorLabels" -}}
app.kubernetes.io/name: {{ include "multi-env-app.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end }}

{{- define "multi-env-app.labels" -}}
helm.sh/chart: {{ include "multi-env-app.chart" . }}
{{ include "multi-env-app.selectorLabels" . }}
app.kubernetes.io/version: {{ include "multi-env-app.appVersion" . | quote }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
app.kubernetes.io/environment: {{ .Values.environment }}
{{- end }}
```

Code Snippet – helm/multi-env-app/templates/configmap.yaml (complete, 16 lines)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "multi-env-app.fullname" . }}-config
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
data:
  APP_NAME: {{ .Values.app.name | quote }}
  APP_ENV: {{ .Values.environment | quote }}
  APP_VERSION: {{ include "multi-env-app.appVersion" . | quote }}
  REPLICA_COUNT: {{ .Values.replicaCount | quote }}
  HELM_RELEASE: {{ .Release.Name | quote }}
  HELM_CHART_VERSION: {{ .Chart.Version | quote }}
  {{- range $key, $value := .Values.config }}
  {{ $key }}: {{ $value | toString | quote }}
  {{- end }}
```

Code Snippet – helm/multi-env-app/templates/deployment.yaml (complete, 77 lines)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "multi-env-app.fullname" . }}
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  revisionHistoryLimit: 5
  {{- with .Values.strategy }}
  strategy:
    {{- toYaml . | nindent 4 }}
  {{- end }}
  selector:
    matchLabels:
      {{- include "multi-env-app.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "multi-env-app.labels" . | nindent 8 }}
      annotations:
        # Roll the pods automatically whenever the ConfigMap content changes.
        checksum/config: {{ include (print $.Template.BasePath
 "/configmap.yaml") . | sha256sum }}
        {{- with .Values.podAnnotations }}
        {{- toYaml . | nindent 8 }}
        {{- end }}
    spec:
      {{- with .Values.podSecurityContext }}
      securityContext:
        {{- toYaml . | nindent 8 }}
      {{- end }}
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ include "multi-env-
 app.imageTag" . }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          {{- with .Values.securityContext }}
          securityContext:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          ports:
            - name: http
              containerPort: {{ .Values.containerPort }}
              protocol: TCP
          envFrom:
            - configMapRef:
                name: {{ include "multi-env-app.fullname" . }}-config
            {{- if .Values.existingSecret }}
            - secretRef:
                name: {{ .Values.existingSecret }}
```

```
            {{- end }}
          env:
            - name: PORT
              value: {{ .Values.containerPort | quote }}
            - name: POD_NAME
              valueFrom:
                fieldRef:
                  fieldPath: metadata.name
            - name: POD_NAMESPACE
              valueFrom:
                fieldRef:
                  fieldPath: metadata.namespace
            - name: NODE_NAME
              valueFrom:
                fieldRef:
                  fieldPath: spec.nodeName
          {{- with .Values.livenessProbe }}
          livenessProbe:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          {{- with .Values.readinessProbe }}
          readinessProbe:
            {{- toYaml . | nindent 12 }}
          {{- end }}
          {{- with .Values.resources }}
          resources:
            {{- toYaml . | nindent 12 }}
          {{- end }}
```

Code Snippet – helm/multi-env-app/templates/service.yaml (complete, 18 lines)

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "multi-env-app.fullname" . }}
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
spec:
  type: {{ .Values.service.type }}
  selector:
    {{- include "multi-env-app.selectorLabels" . | nindent 4 }}
  ports:
    - name: http
      port: {{ .Values.service.port }}
      targetPort: http
      protocol: TCP
      {{- if and (eq .Values.service.type "NodePort") .Values.service.nodePort }}
      nodePort: {{ .Values.service.nodePort }}
      {{- end }}
```

Code Snippet – helm/multi-env-app/templates/tests/test-connection.yaml (complete, 27 lines)

```
# Run with: helm test <release> -n <namespace>
# Checks that the Service answers and that the pod reports the expected
   environment.
apiVersion: v1
kind: Pod
metadata:
  name: {{ include "multi-env-app.fullname" . }}-test-connection
  labels:
    {{- include "multi-env-app.labels" . | nindent 4 }}
  annotations:
    "helm.sh/hook": test
    "helm.sh/hook-delete-policy": before-hook-creation
spec:
  restartPolicy: Never
  containers:
    - name: check-environment
      image: {{ .Values.tests.image }}
      imagePullPolicy: IfNotPresent
      command: ["sh", "-c"]
      args:
        - |
```

```
          set -e
          URL="http://{{ include "multi-env-app.fullname" . }}:{{ .Values.service.port }}"
          wget -qO- "$URL/health"
          BODY="$(wget -qO- "$URL/api/config")"
          echo "$BODY"
          echo "$BODY" | grep -Eq '"environment": ?"{{ .Values.environment }}"'
          echo "OK: service reachable and environment is {{ .Values.environment }}"
```

Code Snippet – helm/multi-env-app/templates/NOTES.txt (complete, 17 lines)

```
{{ .Values.app.name }} has been deployed.

  Release     : {{ .Release.Name }} (revision {{ .Release.Revision }})
  Namespace   : {{ .Release.Namespace }}
  Environment : {{ .Values.environment }}
  Version     : {{ include "multi-env-app.appVersion" . }}
  Replicas    : {{ .Values.replicaCount }}

Open the dashboard:
{{- if eq .Values.service.type "NodePort" }}
  minikube service {{ include "multi-env-app.fullname" . }} -n {{ .Release.Namespace }} --url
{{- end }}
  kubectl port-forward -n {{ .Release.Namespace }} svc/{{ include "multi-env-app.fullname" . }} 8080:{{ .Values.service.port }}
  then browse to http://localhost:8080

Verify the release:
  helm test {{ .Release.Name }} -n {{ .Release.Namespace }}
```

Code Snippet – helm/multi-env-app/.helmignore (complete, 10 lines)

```
.DS_Store
.git/
.gitignore
*.swp
*.bak
*.tmp
*.orig
*~
.idea/
.vscode/
```

Appendix E – Scripts

Code Snippet – scripts/common.sh (complete, 43 lines)

```
#!/usr/bin/env bash
# Shared settings for all scripts. Source this file; do not execute it.

PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
CHART_DIR="${PROJECT_ROOT}/helm/multi-env-app"
IMAGE_REPO="${IMAGE_REPO:-multi-env-app}"
IMAGE_TAG="${IMAGE_TAG:-$(awk '/^appVersion:/ {gsub(/"/, "", $2); print $2}' "${CHART_DIR}/Chart.yaml")}"
RELEASE_PREFIX="${RELEASE_PREFIX:-myapp}"
SECRET_NAME="multi-env-app-secrets"
ALL_ENVS=(dev test staging prod)

# Environment short name -> namespace / values file / expected APP_ENV / NodePort.
namespace_for()  { echo "$1"; }
values_for()     { echo "${CHART_DIR}/values-$1.yaml"; }
release_for()    { echo "${RELEASE_PREFIX}-$1"; }
app_env_for() {
  case "$1" in
    dev) echo development ;; test) echo testing ;;
    staging) echo staging ;; prod) echo production ;;
    *) return 1 ;;
  esac
}
```

```
node_port_for() {
  case "$1" in
    dev) echo 30081 ;; test) echo 30082 ;; staging) echo 30083 ;; prod) echo 30084 ;;
    *) return 1 ;;
  esac
}

require_env() {
  if ! app_env_for "${1:-}" >/dev/null; then
    echo "ERROR: environment must be one of: ${ALL_ENVS[*]} (got '${1:-}')" >&2
    exit 1
  fi
}

require_cmd() {
  for cmd in "$@"; do
    command -v "$cmd" >/dev/null 2>&1 || { echo "ERROR: '$cmd' is not installed or not on PATH" >&2; exit 1; }
  done
}

log() { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
```

Code Snippet – scripts/minikube-setup.sh (complete, 25 lines)

```
#!/usr/bin/env bash
# Start Minikube, build the application image inside it and create the
# four environment namespaces.
#
# Usage: ./scripts/minikube-setup.sh
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd minikube kubectl helm

log "Starting Minikube (skipped if already running)"
if ! minikube status >/dev/null 2>&1; then
  minikube start --cpus=2 --memory=4096 --driver="${MINIKUBE_DRIVER:-docker}"
fi
kubectl cluster-info

log "Building image ${IMAGE_REPO}:${IMAGE_TAG} inside Minikube"
# Builds directly into Minikube's container runtime, so no registry is needed.
minikube image build -t "${IMAGE_REPO}:${IMAGE_TAG}" "${PROJECT_ROOT}"

log "Creating namespaces"
for env in "${ALL_ENVS[@]}"; do
  kubectl create namespace "$(namespace_for "$env")" --dry-run=client -o yaml | kubectl apply -f -
done

log "Done. Next: ./scripts/deploy.sh dev   (or: ./scripts/deploy.sh all)"
```

Code Snippet – scripts/deploy.sh (complete, 49 lines)

```
#!/usr/bin/env bash
# Install or upgrade the chart for one environment (or all of them).
#
# Usage: ./scripts/deploy.sh <dev|test|staging|prod|all> [extra helm args...]
# Example: ./scripts/deploy.sh staging --set replicaCount=3
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm

TARGET="${1:-}"
shift || true

ensure_prod_secret() {
  local ns="$1"
  if ! kubectl get secret "${SECRET_NAME}" -n "$ns" >/dev/null 2>&1; then
    log "Creating Secret ${SECRET_NAME} in ${ns} with a random SECRET_KEY"
    # Generated at deploy time - never stored in Git.
    local key
    key="$(head -c 32 /dev/urandom | od -An -tx1 | tr -d ' \n')"
    kubectl create secret generic "${SECRET_NAME}" -n "$ns" --from-literal=SECRET_KEY="${key}"
```

```
  fi
}

deploy_one() {
  local env="$1"; shift
  require_env "$env"
  local ns release values
  ns="$(namespace_for "$env")"
  release="$(release_for "$env")"
  values="$(values_for "$env")"

  log "Deploying ${release} to namespace ${ns} using $(basename "$values")"
  kubectl create namespace "$ns" --dry-run=client -o yaml | kubectl apply -f -
   >/dev/null
  [[ "$env" == "prod" ]] && ensure_prod_secret "$ns"

  helm upgrade --install "$release" "$CHART_DIR" \
    --namespace "$ns" \
    -f "$values" \
    --atomic --wait --timeout 5m \
    "$@"

  kubectl get deploy,svc,configmap,pods -n "$ns" -l "app.kubernetes.io/instance=$
   {release}"
}

if [[ "$TARGET" == "all" ]]; then
  for env in "${ALL_ENVS[@]}"; do deploy_one "$env" "$@"; done
else
  deploy_one "$TARGET" "$@"
fi
```

Code Snippet – scripts/validate-helm.sh (complete, 45 lines)

```
#!/usr/bin/env bash
# Validate the Helm chart for every environment WITHOUT touching a cluster:
#   1. helm lint --strict (also enforces values.schema.json)
#   2. helm template (render) for each values file
#   3. kubeconform schema validation of the rendered manifests (if installed)
#
# Usage: ./scripts/validate-helm.sh
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd helm

OUT_DIR="${PROJECT_ROOT}/dist/rendered"
mkdir -p "$OUT_DIR"
failures=0

for env in "${ALL_ENVS[@]}"; do
  values="$(values_for "$env")"
  log "[$env] helm lint --strict"
  helm lint "$CHART_DIR" --strict -f "$values" || failures=$((failures + 1))

  log "[$env] helm template"
  out="${OUT_DIR}/${env}.yaml"
  if helm template "$(release_for "$env")" "$CHART_DIR" -f "$values" -n "$
   (namespace_for "$env")" > "$out"; then
    grep -q "APP_ENV: \"$(app_env_for "$env")\"" "$out" \
      && echo "APP_ENV correctly rendered as $(app_env_for "$env")" \
      || { echo "APP_ENV mismatch in $out"; failures=$((failures + 1)); }
  else
    failures=$((failures + 1))
  fi

  if command -v kubeconform >/dev/null 2>&1; then
    log "[$env] kubeconform"
    kubeconform -strict -summary -kubernetes-version "${KUBERNETES_VERSION:-
   1.31.0}" "$out" \
      || failures=$((failures + 1))
  else
    echo "kubeconform not installed - skipping Kubernetes schema validation"
  fi
done

log "Rendered manifests written to ${OUT_DIR}"
```

```
if [[ $failures -gt 0 ]]; then
  echo "VALIDATION FAILED (${failures} problem(s))" >&2
  exit 1
fi
echo "All environments validated successfully."
```

Code Snippet – scripts/test-environments.sh (complete, 68 lines)

```
#!/usr/bin/env bash
# Live verification of deployed environments. For each environment it checks:
#   - the Helm release is deployed
#   - all desired replicas are ready
#   - the running app reports the expected environment (via /api/config)
#   - /health returns ok
#   - `helm test` passes
#
# Usage: ./scripts/test-environments.sh [dev|test|staging|prod|all]   (default: all)
set -uo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm curl python3

TARGET="${1:-all}"
[[ "$TARGET" == "all" ]] && ENVS=("${ALL_ENVS[@]}") || { require_env "$TARGET"; ENVS=("$TARGET"); }

pass=0; fail=0
check() {  # check <description> <command...>
  local desc="$1"; shift
  if "$@"; then echo "  PASS  $desc"; pass=$((pass + 1)); else echo "  FAIL  $desc"; fail=$((fail + 1)); fi
}

json_field() {  # json_field <python expression on d>
  python3 -c "import json,sys; d=json.load(sys.stdin); print($1)"
}

local_port=18080
for env in "${ENVS[@]}"; do
  ns="$(namespace_for "$env")"; release="$(release_for "$env")"; expected="$(app_env_for "$env")"
  log "Testing environment '${env}' (release ${release}, namespace ${ns})"

  if ! helm status "$release" -n "$ns" >/dev/null 2>&1; then
    echo "  FAIL  release ${release} is not installed"; fail=$((fail + 1)); continue
  fi
  check "helm release status is deployed" \
    bash -c "helm status '$release' -n '$ns' -o json | python3 -c 'import json,sys; sys.exit(json.load(sys.stdin)[\"info\"][\"status\"] != \"deployed\")'"

  deploy="$(kubectl get deploy -n "$ns" -l "app.kubernetes.io/instance=${release}" -o jsonpath='{.items[0].metadata.name}')"
  desired="$(kubectl get deploy "$deploy" -n "$ns" -o jsonpath='{.spec.replicas}')"
  ready="$(kubectl get deploy "$deploy" -n "$ns" -o jsonpath='{.status.readyReplicas}')"
  check "replicas ready (${ready:-0}/${desired})" test "${ready:-0}" = "$desired"

  svc="$(kubectl get svc -n "$ns" -l "app.kubernetes.io/instance=${release}" -o jsonpath='{.items[0].metadata.name}')"
  kubectl port-forward -n "$ns" "svc/${svc}" "${local_port}:80" >/dev/null 2>&1 &
  pf_pid=$!
  for _ in $(seq 1 20); do curl -fs "http://127.0.0.1:${local_port}/health" >/dev/null 2>&1 && break; sleep 0.5; done

  body="$(curl -fs "http://127.0.0.1:${local_port}/api/config" || echo '{}')"
  actual="$(echo "$body" | json_field 'd.get("environment")')"
  check "APP_ENV is '${expected}' (got '${actual}')" test "$actual" = "$expected"
  replicas_cfg="$(echo "$body" | json_field 'd.get("settings", {}).get("REPLICA_COUNT")')"
  check "REPLICA_COUNT in ConfigMap matches deployment (${replicas_cfg})" test "$replicas_cfg" = "$desired"
  check "/health returns ok" bash -c "curl -fs http://127.0.0.1:${local_port}/health | grep -q ok"
  if [[ "$env" == "prod" ]]; then
```

```
    debug="$(echo "$body" | json_field 'd["settings"]["DEBUG_MODE"]')"
    check "production DEBUG_MODE is False" test "$debug" = "False"
    src="$(echo "$body" | json_field 'd["settings"]["SECRET_KEY"]')"
    check "production SECRET_KEY comes from a Kubernetes Secret" bash -c "echo '$src' | grep -q kubernetes-secret"
  fi
  kill "$pf_pid" >/dev/null 2>&1; wait "$pf_pid" 2>/dev/null
  local_port=$((local_port + 1))

  check "helm test ${release}" bash -c "helm test '$release' -n '$ns' --timeout 2m >/dev/null"
done

echo
echo "Result: ${pass} passed, ${fail} failed"
[[ $fail -eq 0 ]]
```

Code Snippet – scripts/upgrade-rollback-demo.sh (complete, 50 lines)

```
#!/usr/bin/env bash
# Demonstrates the Helm release lifecycle on one environment:
#   install (v1.0.0) -> upgrade (v1.1.0) -> history -> rollback -> history
#
# Usage: ./scripts/upgrade-rollback-demo.sh [dev|test|staging|prod]   (default: dev)
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm

ENV_NAME="${1:-dev}"
require_env "$ENV_NAME"
NS="$(namespace_for "$ENV_NAME")"
RELEASE="$(release_for "$ENV_NAME")"
VALUES="$(values_for "$ENV_NAME")"
NEW_VERSION="${NEW_VERSION:-1.1.0}"

show_version() {
  kubectl get configmap -n "$NS" -l "app.kubernetes.io/instance=${RELEASE}" \
    -o jsonpath='{.items[0].data.APP_VERSION}'
}

if ! helm status "$RELEASE" -n "$NS" >/dev/null 2>&1; then
  log "Release not found - installing first"
  "$(dirname "$0")/deploy.sh" "$ENV_NAME"
fi

ORIGINAL="$(show_version)"
log "Current version: ${ORIGINAL}"

log "Upgrading ${RELEASE} to APP_VERSION ${NEW_VERSION} and LOG_LEVEL=INFO"
# --reuse-values is NOT used on purpose: the values file is always the source of truth.
helm upgrade "$RELEASE" "$CHART_DIR" -n "$NS" -f "$VALUES" \
  --set app.version="${NEW_VERSION}" --set config.LOG_LEVEL=INFO \
  --atomic --wait --timeout 5m
UPGRADED="$(show_version)"
echo "Version after upgrade: ${UPGRADED}"
[[ "$UPGRADED" == "$NEW_VERSION" ]] || { echo "ERROR: upgrade did not apply" >&2; exit 1; }

log "Release history"
helm history "$RELEASE" -n "$NS"

PREVIOUS="$(helm history "$RELEASE" -n "$NS" -o json | python3 -c 'import json,sys; h=json.load(sys.stdin); print(h[-2]["revision"])')"
log "Rolling back to revision ${PREVIOUS}"
helm rollback "$RELEASE" "$PREVIOUS" -n "$NS" --wait --timeout 5m
RESTORED="$(show_version)"
echo "Version after rollback: ${RESTORED}"
[[ "$RESTORED" == "$ORIGINAL" ]] || { echo "ERROR: rollback did not restore ${ORIGINAL}" >&2; exit 1; }

log "Release history after rollback"
helm history "$RELEASE" -n "$NS"
```

Code Snippet – scripts/docker-demo.sh (complete, 63 lines)

```
#!/usr/bin/env bash
# Run all four environments side by side with plain Docker (no Kubernetes).
# Each container gets the same settings Helm would put in its ConfigMap,
# read from values.yaml + values-<env>.yaml. Useful as a quick local demo
# or as a fallback when Minikube is not available.
#
# Usage: ./scripts/docker-demo.sh up     -> http://localhost:8081 (dev) ... 8084 (prod)
#        ./scripts/docker-demo.sh down
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd docker python3

ACTION="${1:-up}"

host_port_for() {
  case "$1" in dev) echo 8081 ;; test) echo 8082 ;; staging) echo 8083 ;; prod) echo 8084 ;; esac
}

# Print the merged settings of one environment as KEY=VALUE lines (env-file format).
env_file_for() {
  python3 - "$CHART_DIR" "$1" "$IMAGE_TAG" <<'PY'
import sys, yaml
chart, env, tag = sys.argv[1:]

def merge(base, over):
    out = dict(base)
    for k, v in over.items():
        out[k] = merge(out[k], v) if isinstance(v, dict) and isinstance(out.get(k), dict) else v
    return out

with open(f"{chart}/values.yaml") as f:
    base = yaml.safe_load(f)
with open(f"{chart}/values-{env}.yaml") as f:
    v = merge(base, yaml.safe_load(f))
print(f"APP_NAME={v['app']['name']}")
print(f"APP_ENV={v['environment']}")
print(f"APP_VERSION={v['app'].get('version') or v['image'].get('tag') or tag}")
print(f"REPLICA_COUNT={v['replicaCount']}")
for key, value in v["config"].items():
    print(f"{key}={value}")
PY
}

case "$ACTION" in
  up)
    for env in "${ALL_ENVS[@]}"; do
      name="multi-env-${env}"
      docker rm -f "$name" >/dev/null 2>&1 || true
      env_file="$(mktemp)"
      env_file_for "$env" > "$env_file"
      docker run -d --name "$name" -p "$(host_port_for "$env"):5000" \
        --env-file "$env_file" "${IMAGE_REPO}:${IMAGE_TAG}" >/dev/null
      rm -f "$env_file"
      echo "${env}: http://localhost:$(host_port_for "$env")"
    done
    ;;
  down)
    for env in "${ALL_ENVS[@]}"; do docker rm -f "multi-env-${env}" >/dev/null 2>&1 || true; done
    echo "stopped"
    ;;
  *)
    echo "Usage: $0 up|down" >&2; exit 1 ;;
esac
```

Code Snippet – scripts/cleanup.sh (complete, 12 lines)

```
#!/usr/bin/env bash
# Uninstall every release and delete the environment namespaces.
# Usage: ./scripts/cleanup.sh
set -euo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm

for env in "${ALL_ENVS[@]}"; do
  ns="$(namespace_for "$env")"; release="$(release_for "$env")"
  helm uninstall "$release" -n "$ns" 2>/dev/null && echo "uninstalled ${release}" || true
  kubectl delete namespace "$ns" --ignore-not-found
done
```

Code Snippet – scripts/capture-evidence.sh (complete, 238 lines)

```
#!/usr/bin/env bash
# Records the evidence quoted in the project report from a running cluster:
# command output (docs/evidence/*.txt, *.json) and dashboard screenshots (*.png).
#
# It deploys all four environments, takes a screenshot of each dashboard,
# upgrades and rolls back the dev release, forces two failures on purpose
# (a bad image tag with --atomic, and production without its Secret) and
# measures how many requests fail while a release is being updated.
#
# Prerequisites: a cluster in the current kubectl context whose nodes already
# have the images multi-env-app:<appVersion> and busybox:1.36, plus helm,
# kubectl, curl, python3 and a headless Chrome or Chromium.
#
# Usage: ./scripts/capture-evidence.sh [output-dir]      (default: docs/evidence)
set -uo pipefail
source "$(dirname "$0")/common.sh"
require_cmd kubectl helm curl python3

OUT="${1:-${PROJECT_ROOT}/docs/evidence}"
BROWSER="${BROWSER:-$(command -v google-chrome || command -v chromium || command -v chromium-browser || true)}"
PROBE_SECONDS="${PROBE_SECONDS:-45}"
CHART="./helm/multi-env-app"

mkdir -p "$OUT"
rm -f "$OUT"/*.txt "$OUT"/*.png "$OUT"/*.json "$OUT"/*.csv 2>/dev/null
cd "$PROJECT_ROOT"

strip_ansi() { sed 's/\x1b\[[0-9;]*m//g'; }

# capture <file> <command>: append "$ command" and its output to a text file.
capture() {
  local file="$1"; shift
  { printf '$ %s\n' "$*"; bash -c "$*" 2>&1 | strip_ansi; echo; } >> "${OUT}/${file}"
}

# shoot <name> <namespace> <release> <local-port>: screenshot the dashboard and
# save /api/config next to it. A fresh port-forward is used every time.
shoot() {
  local name="$1" ns="$2" release="$3" port="$4" svc pf
  svc="$(kubectl get svc -n "$ns" -l "app.kubernetes.io/instance=${release}" -o jsonpath='{.items[0].metadata.name}')"
  kubectl port-forward -n "$ns" "svc/${svc}" "${port}:80" >/dev/null 2>&1 &
  pf=$!
  for _ in $(seq 1 40); do curl -fs "http://127.0.0.1:${port}/health" >/dev/null 2>&1 && break; sleep 0.5; done
  curl -s "http://127.0.0.1:${port}/api/config" > "${OUT}/${name}-api.json"
  if [[ -n "$BROWSER" ]]; then
    "$BROWSER" --headless=new --no-sandbox --disable-gpu --hide-scrollbars \
      --force-device-scale-factor=1.5 --window-size=1440,1010 --virtual-time-budget=10000 \
      --screenshot="${OUT}/${name}.png" "http://127.0.0.1:${port}/" >/dev/null 2>&1
  else
    echo "no headless browser found: screenshot ${name} skipped" >&2
  fi
  kill "$pf" 2>/dev/null; wait "$pf" 2>/dev/null
}
```

```
54
55  section() { printf '\n\033[1;34m==> %s\033[0m\n' "$*"; }
56
57  # ------------------------------------------------------------------------
58  section "Environment"
59  capture 00-environment.txt 'uname -srm'
60  capture 00-environment.txt "lscpu | grep 'Model name'"
61  capture 00-environment.txt 'nproc'
62  capture 00-environment.txt 'free -h | sed -n 1,2p'
63  capture 00-environment.txt 'docker --version'
64  capture 00-environment.txt 'kind version'
65  capture 00-environment.txt 'kubectl version'
66  capture 00-environment.txt 'helm version --short'
67  capture 00-environment.txt 'kubectl get nodes -o wide'
68  [[ -n "$BROWSER" ]] && capture 00-environment.txt "\"$BROWSER\" --version"
69  capture 00-environment.txt 'docker image ls multi-env-app'
70  capture 00-environment.txt 'docker history multi-env-app:1.0.0'
71
72  # ------------------------------------------------------------------------
73  section "Deploy all environments"
74  start=$(date +%s)
75  ./scripts/deploy.sh all 2>&1 | tee "${OUT}/.deploy.raw"
76  end=$(date +%s)
77  { strip_ansi < "${OUT}/.deploy.raw"; echo "deploy.sh all finished in $((end -
      start)) s"; } > "${OUT}/01-deploy-all.txt"
78  rm -f "${OUT}/.deploy.raw"
79
80  section "Cluster state"
81  capture 02-cluster-state.txt 'kubectl get namespaces'
82  capture 02-cluster-state.txt 'helm list -A'
83  for env in "${ALL_ENVS[@]}"; do
84    capture "02-resources-${env}.txt" "kubectl get deploy,svc,cm,pods -n ${env} -o
      wide"
85    kubectl get pods -n "$env" -o json > "${OUT}/pods-${env}.json"
86  done
87  capture 03-prod-details.txt 'kubectl get secret -n prod'
88  capture 03-prod-details.txt 'kubectl get configmap myapp-prod-multi-env-app-
      config -n prod -o yaml'
89  capture 03-prod-details.txt 'kubectl describe deployment myapp-prod-multi-env-app
      -n prod'
90  capture 03-prod-details.txt 'helm get values myapp-prod -n prod'
91  capture 03-dev-values.txt 'helm get values myapp-dev -n dev --all'
92  capture 04-logs.txt 'kubectl logs -n dev deploy/myapp-dev-multi-env-app --
      tail=12'
93  capture 04-logs.txt 'kubectl logs -n prod deploy/myapp-prod-multi-env-app --
      tail=12'
94
95  section "Live environment tests"
96  ./scripts/test-environments.sh all 2>&1 | strip_ansi > "${OUT}/05-live-tests.txt"
97
98  capture 05-helm-test.txt 'helm test myapp-prod -n prod'
99
100 section "Dashboard screenshots"
101 shoot dashboard-dev     dev     myapp-dev     18081
102 shoot dashboard-test    test    myapp-test    18082
103 shoot dashboard-staging staging myapp-staging 18083
104 shoot dashboard-prod    prod    myapp-prod    18084
105
106 # ------------------------------------------------------------------------
107 section "Upgrade and rollback of the dev release"
108 capture 06-upgrade-rollback.txt 'helm history myapp-dev -n dev'
109 capture 06-upgrade-rollback.txt "helm upgrade myapp-dev ${CHART} -n dev -f
      helm/multi-env-app/values-dev.yaml --set app.version=1.1.0 --set
      config.LOG_LEVEL=INFO --atomic --wait --timeout 5m | sed -n 1,8p"
110 capture 06-upgrade-rollback.txt 'helm history myapp-dev -n dev'
111 capture 06-upgrade-rollback.txt 'kubectl get pods -n dev'
112 capture 06-upgrade-rollback.txt 'helm get values myapp-dev -n dev'
113 shoot dashboard-dev-upgraded dev myapp-dev 18085
114 capture 06-upgrade-rollback.txt 'helm rollback myapp-dev 1 -n dev --wait --
      timeout 5m'
115 capture 06-upgrade-rollback.txt 'helm history myapp-dev -n dev'
116 shoot dashboard-dev-rolledback dev myapp-dev 18086
117
118 # ------------------------------------------------------------------------
119 section "Failure 1: bad image tag with --atomic"
120 capture 07-atomic-failure.txt 'helm history myapp-test -n test'
121 {
```

```
  printf '$ helm upgrade myapp-test %s -n test -f helm/multi-env-app/values-test.yaml --set image.tag=9.9.9 --atomic --wait --timeout 60s\n' "$CHART"
  helm upgrade myapp-test "$CHART" -n test -f helm/multi-env-app/values-test.yaml \
    --set image.tag=9.9.9 --atomic --wait --timeout 60s 2>&1 | strip_ansi
  echo "exit status: ${PIPESTATUS[0]}"; echo
} > "${OUT}/.atomic.raw" &
atomic_pid=$!
sleep 35
capture 07-atomic-failure.txt 'kubectl get pods -n test'
capture 07-atomic-failure.txt "kubectl get events -n test --sort-by=.lastTimestamp | tail -6"
wait "$atomic_pid"
cat "${OUT}/.atomic.raw" >> "${OUT}/07-atomic-failure.txt"; rm -f "${OUT}/.atomic.raw"
capture 07-atomic-failure.txt 'helm history myapp-test -n test'
capture 07-atomic-failure.txt 'kubectl get pods -n test'

section "Failure 2: production without its Secret"
kubectl create namespace demo >/dev/null 2>&1
{
  # NodePorts are unique per cluster, so the demo release must not reuse 30084.
  printf '$ helm install demo-prod %s -n demo -f helm/multi-env-app/values-prod.yaml --set service.nodePort=30089 --wait --timeout 40s\n' "$CHART"
  helm install demo-prod "$CHART" -n demo -f helm/multi-env-app/values-prod.yaml --set service.nodePort=30089 \
    --wait --timeout 40s 2>&1 | strip_ansi
  echo
} > "${OUT}/09-missing-secret.txt"
capture 09-missing-secret.txt 'kubectl get pods -n demo'
capture 09-missing-secret.txt "kubectl get events -n demo --sort-by=.lastTimestamp | grep -i secret | tail -3"
helm uninstall demo-prod -n demo >/dev/null 2>&1
kubectl delete namespace demo --wait=false >/dev/null 2>&1

# ---------------------------------------------------------------------------
# Availability during rolling updates: a busybox pod requests /ready about eight
# times a second while the release is changed. Every failed request is logged and
# the totals are appended to 08-availability.csv (scenario,run,requests,failed,rollout_seconds).
read -r -d '' PROBE_SCRIPT <<'EOF'
start=$(date +%s); n=0; f=0
while [ $(( $(date +%s) - start )) -lt __DURATION__ ]; do
  n=$((n+1))
  if ! wget -q -T 2 -O /dev/null http://__SERVICE__/ready; then
    f=$((f+1)); echo "t=$(( $(date +%s) - start ))s request $n failed"
  fi
  sleep 0.1
done
echo "requests=$n failed=$f"
EOF

echo "scenario,run,requests,failed,rollout_seconds" > "${OUT}/08-availability.csv"

# measure_rollout <scenario> <run> <title> <namespace> <release> [helm upgrade options...]
# With no helm options the probe simply runs for the same time (control: no rollout).
measure_rollout() {
  local scenario="$1" run="$2" title="$3" ns="$4" release="$5"; shift 5
  local svc="${release}-multi-env-app" script t0 t1 result totals req failed
  script="${PROBE_SCRIPT//__DURATION__/$PROBE_SECONDS}"; script="${script//__SERVICE__/$svc}"
  kubectl delete pod probe -n "$ns" --ignore-not-found >/dev/null 2>&1
  kubectl run probe -n "$ns" --image=busybox:1.36 --image-pull-policy=IfNotPresent --restart=Never \
    --command -- sh -c "$script" >/dev/null
  kubectl wait --for=condition=Ready pod/probe -n "$ns" --timeout=60s >/dev/null 2>&1
  sleep 8
  t0=$(date +%s)
  if [[ $# -gt 0 ]]; then
    helm upgrade "$release" "$CHART" -n "$ns" -f "helm/multi-env-app/values-${ns}.yaml" "$@" --wait --timeout 3m >/dev/null 2>&1
  fi
  t1=$(date +%s)
```

```
  kubectl wait --for=jsonpath='{.status.phase}'=Succeeded pod/probe -n "$ns" --timeout="$((PROBE_SECONDS + 90))s" >/dev/null 2>&1
  result="$(kubectl logs probe -n "$ns" 2>&1)"
  {
    echo "### ${scenario}${run}: ${title}"
    if [[ $# -gt 0 ]]; then
      printf '$ helm upgrade %s %s -n %s -f helm/multi-env-app/values-%s.yaml %s --wait\n' "$release" "$CHART" "$ns" "$ns" "$*"
    fi
    echo "rollout finished in $((t1 - t0)) s"
    echo "$result"
    echo
  } >> "${OUT}/08-availability.txt"
  totals="$(echo "$result" | grep '^requests=' | tail -1)"
  req="${totals#requests=}"; req="${req%% *}"; failed="${totals##*failed=}"
  echo "${scenario},${run},${req:-NA},${failed:-NA},$((t1 - t0))" >> "${OUT}/08-availability.csv"
  kubectl delete pod probe -n "$ns" --ignore-not-found >/dev/null 2>&1
  sleep 5
}

section "Availability during rolling updates"
measure_rollout Z 1 "control: test (1 replica), no change of the release" test myapp-test
for i in 1 2 3; do
  measure_rollout A "$i" "staging (2 replicas), chart strategy maxSurge=1 maxUnavailable=0" staging myapp-staging \
    --set-string "config.CACHE_TTL_SECONDS=13${i}"
done
for i in 1 2 3; do
  measure_rollout B "$i" "test (1 replica), chart strategy maxSurge=1 maxUnavailable=0" test myapp-test \
    --set-string "config.CACHE_TTL_SECONDS=3${i}"
done
for i in 1 2 3; do
  measure_rollout C "$i" "test (1 replica), alternative strategy maxSurge=0 maxUnavailable=1" test myapp-test \
    --set-string "config.CACHE_TTL_SECONDS=4${i}" --set strategy.rollingUpdate.maxSurge=0 --set strategy.rollingUpdate.maxUnavailable=1
done

# Mitigation experiment (not part of the chart): a 5 s preStop delay keeps the old Pod serving
# while the endpoints are updated. It is added to the live Deployment with kubectl patch.
section "Mitigation experiment: preStop delay"
kubectl patch deployment myapp-test-multi-env-app -n test --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/lifecycle","value":{"preStop":{"exec":{"command":["sleep","5"]}}}}]' >/dev/null
kubectl rollout status deployment/myapp-test-multi-env-app -n test --timeout=120s >/dev/null 2>&1
{
  echo "### D: preStop hook added with kubectl patch (not part of the chart)"
  echo '$ kubectl get deployment myapp-test-multi-env-app -n test -o jsonpath={.spec.template.spec.containers[0].lifecycle}'
  kubectl get deployment myapp-test-multi-env-app -n test -o jsonpath='{.spec.template.spec.containers[0].lifecycle}'
  echo; echo
} >> "${OUT}/08-availability.txt"
for i in 1 2 3; do
  measure_rollout D "$i" "test (1 replica), chart strategy plus preStop sleep 5 (experiment)" test myapp-test \
    --set-string "config.CACHE_TTL_SECONDS=5${i}"
done

# ------------------------------------------------------------------------------
section "Final state"
capture 10-final-state.txt 'helm list -A'
capture 10-final-state.txt 'kubectl get pods -A -l app.kubernetes.io/name=multi-env-app'
echo "evidence written to ${OUT}"
```

Appendix F – Tests

Code Snippet – tests/conftest.py (complete, 9 lines)

```
import os
import sys

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
APP_DIR = os.path.join(ROOT, "app")
CHART_DIR = os.path.join(ROOT, "helm", "multi-env-app")

if APP_DIR not in sys.path:
    sys.path.insert(0, APP_DIR)
```

Code Snippet – tests/test_app.py (complete, 93 lines)

```
"""Unit tests for the Flask application and its configuration loading."""

import pytest

from app import create_app
from config import VALID_ENVIRONMENTS, AppConfig


def make_client(**env):
    app = create_app(env)
    app.config["TESTING"] = True
    return app.test_client()


@pytest.mark.parametrize("environment", VALID_ENVIRONMENTS)
def test_dashboard_shows_name_environment_and_version(environment):
    client = make_client(APP_NAME="Demo App", APP_ENV=environment,
   APP_VERSION="2.3.4")
    resp = client.get("/")
    assert resp.status_code == 200
    html = resp.get_data(as_text=True)
    assert "Demo App" in html
    assert environment.upper() in html
    assert "v2.3.4" in html


def test_health_and_ready_endpoints():
    client = make_client(APP_ENV="staging")
    assert client.get("/health").get_json() == {"status": "ok"}
    ready = client.get("/ready").get_json()
    assert ready == {"status": "ready", "environment": "staging"}


def test_api_config_reflects_environment_variables():
    client = make_client(
        APP_ENV="production",
        APP_VERSION="1.0.0",
        LOG_LEVEL="warning",
        DEBUG_MODE="false",
        MAX_CONNECTIONS="200",
        DATABASE_HOST="postgres.prod.svc.cluster.local",
    )
    data = client.get("/api/config").get_json()
    assert data["environment"] == "production"
    assert data["version"] == "1.0.0"
    settings = data["settings"]
    assert settings["LOG_LEVEL"] == "WARNING"
    assert settings["DEBUG_MODE"] is False
    assert settings["MAX_CONNECTIONS"] == 200
    assert settings["DATABASE_HOST"] == "postgres.prod.svc.cluster.local"


def test_secret_key_is_never_exposed():
    client = make_client(APP_ENV="production", SECRET_KEY="super-secret-value-
   123")
    api_body = client.get("/api/config").get_data(as_text=True)
    page_body = client.get("/").get_data(as_text=True)
    assert "super-secret-value-123" not in api_body
    assert "super-secret-value-123" not in page_body
    assert "kubernetes-secret" in api_body

```

```

def test_secret_key_is_generated_when_not_provided():
    first = AppConfig({})
    second = AppConfig({})
    assert first.secret_key_source == "auto-generated"
    assert len(first.secret_key) == 64
    assert first.secret_key != second.secret_key


def test_invalid_environment_falls_back_to_development():
    cfg = AppConfig({"APP_ENV": "qa-sandbox"})
    assert cfg.environment == "development"
    assert cfg.environment_is_valid is False


@pytest.mark.parametrize(
    "environment,theme",
    [("development", "success"), ("testing", "info"), ("staging", "warning"),
   ("production", "danger")],
)
def test_each_environment_has_distinct_theme(environment, theme):
    assert AppConfig({"APP_ENV": environment}).theme == theme


def test_beta_banner_toggle():
    on = make_client(APP_ENV="development",
   FEATURE_BETA_BANNER="true").get("/").get_data(as_text=True)
    off = make_client(APP_ENV="production",
   FEATURE_BETA_BANNER="false").get("/").get_data(as_text=True)
    assert "Beta features are enabled" in on
    assert "Beta features are enabled" not in off


def test_bad_numeric_values_use_defaults():
    cfg = AppConfig({"MAX_CONNECTIONS": "lots", "CACHE_TTL_SECONDS": ""})
    assert cfg.max_connections == 10
    assert cfg.cache_ttl_seconds == 60
```

Code Snippet – tests/test_helm_config.py (complete, 230 lines)

```
"""Tests that verify the environment-specific Helm configuration.

Part 1 (always runs): static checks of the values-*.yaml files with PyYAML.
Part 2 (runs when the `helm` binary is on PATH): renders the chart for each
environment with `helm template` and checks the generated Kubernetes objects.
"""

import os
import re
import shutil
import subprocess

import pytest
import yaml

from conftest import CHART_DIR

ENVIRONMENTS = {
    # file suffix: (expected environment, expected nodePort, minimum replicas)
    "dev": ("development", 30081, 1),
    "test": ("testing", 30082, 1),
    "staging": ("staging", 30083, 2),
    "prod": ("production", 30084, 3),
}

REQUIRED_CHART_FILES = [
    "Chart.yaml",
    "values.yaml",
    "values-dev.yaml",
    "values-test.yaml",
    "values-staging.yaml",
    "values-prod.yaml",
    "values.schema.json",
    "templates/deployment.yaml",
    "templates/service.yaml",
    "templates/configmap.yaml",
]

```

```
SECRET_KEY_PATTERN = re.compile(r"(password|passwd|secret_key|token|api_key|private_key)", re.IGNORECASE)


def load_yaml(name):
    with open(os.path.join(CHART_DIR, name), encoding="utf-8") as fh:
        return yaml.safe_load(fh)


def merged_values(suffix):
    """Shallow+nested merge of values.yaml and values-<suffix>.yaml (like Helm)."""

    def merge(base, override):
        out = dict(base)
        for key, val in override.items():
            if isinstance(val, dict) and isinstance(out.get(key), dict):
                out[key] = merge(out[key], val)
            else:
                out[key] = val
        return out

    return merge(load_yaml("values.yaml"), load_yaml("values-{}.yaml".format(suffix)))


# ---------------------------------------------------------------------------
# Part 1: static checks
# ---------------------------------------------------------------------------

@pytest.mark.parametrize("path", REQUIRED_CHART_FILES)
def test_required_chart_files_exist(path):
    assert os.path.isfile(os.path.join(CHART_DIR, path)), path


def test_chart_metadata():
    chart = load_yaml("Chart.yaml")
    assert chart["apiVersion"] == "v2"
    assert chart["name"] == "multi-env-app"
    assert re.match(r"^\d+\.\d+\.\d+$", chart["version"])
    assert re.match(r"^\d+\.\d+\.\d+$", str(chart["appVersion"]))


@pytest.mark.parametrize("suffix", ENVIRONMENTS)
def test_environment_name_matches_file(suffix):
    expected_env, _, _ = ENVIRONMENTS[suffix]
    assert merged_values(suffix)["environment"] == expected_env


@pytest.mark.parametrize("suffix", ENVIRONMENTS)
def test_replica_policy(suffix):
    _, _, min_replicas = ENVIRONMENTS[suffix]
    assert merged_values(suffix)["replicaCount"] >= min_replicas


@pytest.mark.parametrize("suffix", ENVIRONMENTS)
def test_every_environment_defines_the_same_config_keys(suffix):
    base_keys = set(load_yaml("values.yaml")["config"])
    env_keys = set(load_yaml("values-{}.yaml".format(suffix))["config"])
    assert env_keys == base_keys, "config keys differ from values.yaml: {}".format(env_keys ^ base_keys)


def test_node_ports_are_unique_and_valid():
    ports = [merged_values(s)["service"]["nodePort"] for s in ENVIRONMENTS]
    assert len(set(ports)) == len(ports)
    for suffix, port in zip(ENVIRONMENTS, ports):
        assert port == ENVIRONMENTS[suffix][1]
        assert 30000 <= port <= 32767


def test_production_hardening():
    prod = merged_values("prod")
    assert prod["config"]["DEBUG_MODE"] == "false"
    assert prod["config"]["LOG_LEVEL"] in ("WARNING", "ERROR")
    assert prod["config"]["FEATURE_BETA_BANNER"] == "false"
    assert prod["existingSecret"], "production must load SECRET_KEY from a Kubernetes Secret"
    assert prod["strategy"]["rollingUpdate"]["maxUnavailable"] == 0
    assert prod["resources"]["limits"]["memory"]
```

```
def test_staging_matches_production_feature_flags():
    staging, prod = merged_values("staging")["config"], merged_values("prod")["config"]
    assert staging["DEBUG_MODE"] == prod["DEBUG_MODE"]
    assert staging["FEATURE_BETA_BANNER"] == prod["FEATURE_BETA_BANNER"]


def test_development_is_verbose():
    dev = merged_values("dev")["config"]
    assert dev["LOG_LEVEL"] == "DEBUG"
    assert dev["DEBUG_MODE"] == "true"


@pytest.mark.parametrize("name", ["values.yaml"] + ["values-{}.yaml".format(s) for s in ENVIRONMENTS])
def test_no_hardcoded_secrets(name):
    """No secret-looking key may carry a literal value in any values file."""

    def walk(node, path=""):
        if isinstance(node, dict):
            for key, val in node.items():
                yield from walk(val, "{}.{}".format(path, key) if path else str(key))
        else:
            yield path, node

    for path, value in walk(load_yaml(name)):
        leaf = path.split(".")[-1]
        if SECRET_KEY_PATTERN.search(leaf) and value not in ("", None):
            pytest.fail("{}: '{}' looks like a hardcoded secret".format(name, path))


# ---------------------------------------------------------------------------
# Part 2: render the chart with Helm
# ---------------------------------------------------------------------------

HELM = shutil.which("helm")
needs_helm = pytest.mark.skipif(HELM is None, reason="helm binary not installed")


def helm_template(suffix):
    out = subprocess.run(
        [HELM, "template", "myapp-" + suffix, CHART_DIR, "-f",
         os.path.join(CHART_DIR, "values-{}.yaml".format(suffix)), "--namespace", suffix],
        check=True, capture_output=True, text=True,
    ).stdout
    return {doc["kind"]: doc for doc in yaml.safe_load_all(out) if doc}


@needs_helm
@pytest.mark.parametrize("suffix", ENVIRONMENTS)
def test_helm_lint_strict(suffix):
    result = subprocess.run(
        [HELM, "lint", CHART_DIR, "--strict", "-f", os.path.join(CHART_DIR, "values-{}.yaml".format(suffix))],
        capture_output=True, text=True,
    )
    assert result.returncode == 0, result.stdout + result.stderr


@needs_helm
@pytest.mark.parametrize("suffix", ENVIRONMENTS)
def test_rendered_manifests_match_environment(suffix):
    expected_env, node_port, _ = ENVIRONMENTS[suffix]
    values = merged_values(suffix)
    docs = helm_template(suffix)

    assert {"ConfigMap", "Deployment", "Service"} <= set(docs)

    cm = docs["ConfigMap"]["data"]
    assert cm["APP_ENV"] == expected_env
    assert cm["APP_VERSION"] == str(load_yaml("Chart.yaml")["appVersion"])
    for key, val in values["config"].items():
        assert cm[key] == str(val)

    dep = docs["Deployment"]
    assert dep["spec"]["replicas"] == values["replicaCount"]
```

```
    assert dep["metadata"]["labels"]["app.kubernetes.io/environment"] == expected_env
    pod = dep["spec"]["template"]["spec"]
    container = pod["containers"][0]
    assert container["envFrom"][0]["configMapRef"]["name"] == docs["ConfigMap"]["metadata"]["name"]
    assert container["resources"] == values["resources"]
    assert pod["securityContext"]["runAsNonRoot"] is True
    assert "checksum/config" in dep["spec"]["template"]["metadata"]["annotations"]

    svc = docs["Service"]
    assert svc["spec"]["ports"][0]["nodePort"] == node_port


@needs_helm
def test_production_mounts_existing_secret():
    container = helm_template("prod")["Deployment"]["spec"]["template"]["spec"]["containers"][0]
    refs = [ref for ref in container["envFrom"] if "secretRef" in ref]
    assert refs and refs[0]["secretRef"]["name"] == merged_values("prod")["existingSecret"]


@needs_helm
def test_config_change_changes_checksum():
    """Changing a config value must roll the pods (new checksum annotation)."""

    def checksum(*extra):
        out = subprocess.run(
            [HELM, "template", "r", CHART_DIR, "-f", os.path.join(CHART_DIR, "values-dev.yaml"), *extra],
            check=True, capture_output=True, text=True,
        ).stdout
        dep = next(d for d in yaml.safe_load_all(out) if d and d["kind"] == "Deployment")
        return dep["spec"]["template"]["metadata"]["annotations"]["checksum/config"]

    assert checksum() != checksum("--set", "config.LOG_LEVEL=INFO")


@needs_helm
@pytest.mark.parametrize(
    "override",
    ["environment=qa", "replicaCount=0", "config.LOG_LEVEL=VERBOSE", "config.SECRET_KEY=abc", "service.nodePort=80"],
)
def test_schema_rejects_invalid_values(override):
    result = subprocess.run([HELM, "lint", CHART_DIR, "--set", override], capture_output=True, text=True)
    assert result.returncode != 0, "schema accepted invalid value: " + override
```

Code Snippet – requirements-dev.txt (complete, 3 lines)

```
-r app/requirements.txt
pytest==8.3.3
PyYAML==6.0.2
```

Appendix G – GitHub Actions workflows

Code Snippet – .github/workflows/helm-validation.yml (complete, 117 lines)

```
name: Helm Chart Validation

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

env:
  HELM_VERSION: v3.16.2
```

```
  KUBECONFORM_VERSION: v0.6.7
  KUBERNETES_VERSION: 1.31.0
  IMAGE: multi-env-app:1.0.0

jobs:
  python-tests:
    name: Unit & configuration tests (pytest)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-python@v6
        with:
          python-version: "3.12"

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Install dependencies
        run: pip install -r requirements-dev.txt

      - name: Run pytest (app + per-environment Helm config tests)
        run: python -m pytest -v

  helm-validate:
    name: Helm lint / template / kubeconform
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Install kubeconform
        run: |
          curl -sSL "https://github.com/yannh/kubeconform/releases/download/${KUBECONFORM_VERSION}/kubeconform-linux-amd64.tar.gz" \
            | tar -xz -C /usr/local/bin kubeconform
          kubeconform -v

      - name: Validate chart for dev, test, staging and prod
        run: ./scripts/validate-helm.sh

      - name: Upload rendered manifests
        uses: actions/upload-artifact@v4
        with:
          name: rendered-manifests
          path: dist/rendered/

  docker-build:
    name: Docker build & smoke test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Build image
        run: docker build -t "$IMAGE" .

      - name: Run container and check endpoints
        run: |
          docker run -d --name smoke -p 5000:5000 -e APP_ENV=testing "$IMAGE"
          for i in $(seq 1 20); do curl -fs http://127.0.0.1:5000/health && break; sleep 1; done
          curl -fs http://127.0.0.1:5000/api/config | tee /tmp/config.json
          python3 -c "import json; d=json.load(open('/tmp/config.json')); assert d['environment']=='testing', d"
          curl -fs http://127.0.0.1:5000/ | grep -q 'id="app-env"'
          docker rm -f smoke

  deploy-kind:
    name: Deploy all environments to a kind cluster
    runs-on: ubuntu-latest
    needs: [python-tests, helm-validate, docker-build]
    steps:
      - uses: actions/checkout@v5

      - uses: azure/setup-helm@v4
```

```
          with:
            version: ${{ env.HELM_VERSION }}

        - name: Create kind cluster
          uses: helm/kind-action@v1
          with:
            cluster_name: chart-testing

        - name: Build image and load it into kind
          run: |
            docker build -t "$IMAGE" .
            kind load docker-image "$IMAGE" --name chart-testing
            docker pull busybox:1.36
            kind load docker-image busybox:1.36 --name chart-testing

        - name: Deploy dev, test, staging and prod
          run: ./scripts/deploy.sh all

        - name: Verify environment-specific configuration (live)
          run: ./scripts/test-environments.sh all

        - name: Upgrade and rollback (dev)
          run: ./scripts/upgrade-rollback-demo.sh dev

        - name: Show cluster state
          if: always()
          run: |
            helm list -A
            kubectl get pods -A -l app.kubernetes.io/name=multi-env-app -o wide
```

Code Snippet – .github/workflows/capture-evidence.yml (complete, 56 lines)

```
name: Capture report evidence

# Deploys all four environments to a throw-away kind cluster, records command
# output and dashboard screenshots with scripts/capture-evidence.sh and commits
# them to docs/evidence/ on the same branch. Run it from the Actions tab, or by
# changing one of the two files listed under "paths".
on:
  workflow_dispatch:
  push:
    paths:
      - ".github/workflows/capture-evidence.yml"
      - "scripts/capture-evidence.sh"

permissions:
  contents: write

env:
  HELM_VERSION: v3.16.2
  IMAGE: multi-env-app:1.0.0

jobs:
  capture:
    name: Capture evidence on kind
    runs-on: ubuntu-latest
    timeout-minutes: 30
    steps:
      - uses: actions/checkout@v5

      - uses: azure/setup-helm@v4
        with:
          version: ${{ env.HELM_VERSION }}

      - name: Create kind cluster
        uses: helm/kind-action@v1
        with:
          cluster_name: evidence

      - name: Build the image and load it into kind
        run: |
          docker build -t "$IMAGE" .
          kind load docker-image "$IMAGE" --name evidence
          docker pull busybox:1.36
          kind load docker-image busybox:1.36 --name evidence

      - name: Capture evidence
        run: ./scripts/capture-evidence.sh
```

```
47
48        - name: Commit evidence
49          if: always()
50          run: |
51            git config user.name "github-actions[bot]"
52            git config user.email "41898282+github-
      actions[bot]@users.noreply.github.com"
53            git add docs/evidence
54            git commit -m "ci: record report evidence [skip ci]" || echo "nothing to
      commit"
55            git pull --rebase origin "${GITHUB_REF_NAME}" || true
56            git push origin "HEAD:${GITHUB_REF_NAME}"
```

Appendix H – Evidence files

The files below are in `docs/evidence/` of the repository. They were written by `scripts/capture-evidence.sh` on the kind cluster of the workflow `capture-evidence.yml` (run 2) and committed by the workflow.

Table A.1 – Evidence files

File	Content
`00-environment.txt`	Runner, tool versions, node, image size and layers
`01-deploy-all.txt`	Output of `scripts/deploy.sh all`
`02-*.txt`, `pods-*.json`	Objects of every environment; `helm list -A`; the Pods as JSON
`03-dev-values.txt`, `03-prod-details.txt`	Computed values of development; ConfigMap, Deployment description and Secrets of production
`04-logs.txt`	Logs of a development and a production Pod
`05-helm-test.txt`, `05-live-tests.txt`	`helm test` of production; output of `test-environments.sh all`
`06-upgrade-rollback.txt`	Upgrade, history, rollback and history of the development release
`07-atomic-failure.txt`	Upgrade to a non-existent image with `--atomic`
`08-availability.txt`, `08-availability.csv`	Failed requests during rolling updates; totals per run
`09-missing-secret.txt`	Production installed without its Secret
`10-final-state.txt`	Releases and Pods at the end of the run
`dashboard-*.png`, `dashboard-*-api.json`	Screenshots and `/api/config` of the dashboards served by the Pods

Appendix I – Reproducing the results

The commands below reproduce the results on a machine with Python, Docker, Helm, kubectl and either Minikube or kind. The first two need no cluster.

Code Snippet – Reproduction of the results

```
pip install -r requirements-dev.txt
python -m pytest -q                          # 62 cases (some need helm)
./scripts/validate-helm.sh                   # lint, render, kubeconform

./scripts/minikube-setup.sh                  # or: a kind cluster + image
./scripts/deploy.sh all                      # four environments
./scripts/test-environments.sh all           # 26 live checks
./scripts/upgrade-rollback-demo.sh dev       # upgrade, history, rollback
./scripts/capture-evidence.sh                # evidence (needs Chrome)
./scripts/cleanup.sh
```

REFERENCES

[1] A. Wiggins, “III. Config,” *The Twelve-Factor App*. https://12factor.net/config [Accessed: 20 Oct. 2026.]

[2] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, “An empirical study on configuration errors in commercial and open source systems,” in *Proc. 23rd ACM Symp. Operating Systems Principles (SOSP '11)*, Cascais, Portugal, 2011, pp. 159–172, doi: 10.1145/2043556.2043572.

[3] T. Xu and Y. Zhou, “Systems approaches to tackling configuration errors: A survey,” *ACM Comput. Surv.*, vol. 47, no. 4, Art. 70, Jul. 2015, doi: 10.1145/2791577.

[4] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[5] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations*. Portland, OR, USA: IT Revolution Press, 2018.

[6] D. Merkel, “Docker: Lightweight Linux containers for consistent development and deployment,” *Linux J.*, vol. 2014, no. 239, Mar. 2014.

[7] A. Verma, L. Pedrosa, M. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in *Proc. 10th European Conf. Computer Systems (EuroSys '15)*, Bordeaux, France, 2015.

[8] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, Omega, and Kubernetes,” *ACM Queue*, vol. 14, no. 1, pp. 70–93, 2016. https://queue.acm.org/detail.cfm?id=2898444

[9] The Kubernetes Authors, “Deployments,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ [Accessed: 20 Oct. 2026.]

[10] The Helm Authors, “Charts,” *Helm Documentation*. https://helm.sh/docs/topics/charts/ [Accessed: 20 Oct. 2026.]

[11] The Helm Authors, “helm rollback,” *Helm Documentation*. https://helm.sh/docs/helm/helm_rollback/ [Accessed: 20 Oct. 2026.]

[12] The Helm Authors, “Values files,” *Helm Documentation*. https://helm.sh/docs/chart_template_guide/values_files/ [Accessed: 20 Oct. 2026.]

[13] A. Zerouali, R. Opdebeeck, and C. De Roover, “Helm charts for Kubernetes applications: Evolution, outdatedness and security risks,” in *Proc. 20th IEEE/ACM Int. Conf. Mining Software Repositories (MSR)*, Melbourne, Australia, 2023.

[14] A. Rahman and L. Williams, “Characterizing defective configuration scripts used for continuous deployment,” in *Proc. 11th IEEE Int. Conf. Software Testing, Verification and Validation (ICST)*, 2018.

[15] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, “A systematic mapping study of infrastructure as code research,” *Inf. Softw. Technol.*, vol. 108, pp. 65–77, 2019.

[16] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, “Security misconfigurations in open source Kubernetes manifests: An empirical study,” *ACM Trans. Softw. Eng. Methodol.*, 2023.

[17] The Kubernetes Authors, “kind,” Kubernetes SIGs. https://kind.sigs.k8s.io/ [Accessed: 20 Oct. 2026.]

[18] The Kubernetes Authors, “minikube start,” *Minikube Documentation*. https://minikube.sigs.k8s.io/docs/start/ [Accessed: 20 Oct. 2026.]

[19] T. A. Limoncelli, “GitOps: A path to more self-service IT,” *ACM Queue*, vol. 16, no. 3, pp. 13–26, 2018, doi: 10.1145/3236386.3237207.

[20] The Helm Authors, “Chart tests,” *Helm Documentation*. https://helm.sh/docs/topics/chart_tests/ [Accessed: 20 Oct. 2026.]

[21] The Helm Authors, “helm lint,” *Helm Documentation*. https://helm.sh/docs/helm/helm_lint/ [Accessed: 20 Oct. 2026.]

[22] The Helm Authors, “helm upgrade,” *Helm Documentation*. https://helm.sh/docs/helm/helm_upgrade/ [Accessed: 20 Oct. 2026.]

[23] The Kubernetes Authors, “ConfigMaps,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/configuration/configmap/ [Accessed: 20 Oct. 2026.]

[24] The Kubernetes Authors, “Secrets,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/configuration/secret/ [Accessed: 20 Oct. 2026.]

[25] The Kubernetes Authors, “Configure liveness, readiness and startup probes,” *Kubernetes Documentation*. https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/ [Accessed: 20 Oct. 2026.]

[26] The Kubernetes Authors, “Service,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/services-networking/service/ [Accessed: 20 Oct. 2026.]

[27] GitHub, Inc., “Workflow syntax for GitHub Actions,” *GitHub Docs*. https://docs.github.com/en/actions/writing-workflows/workflow-syntax-for-github-actions [Accessed: 20 Oct. 2026.]

[28] The Kubernetes Authors, “Declarative management of Kubernetes objects using Kustomize,” *Kubernetes Documentation*. https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/ [Accessed: 20 Oct. 2026.]

[29] Pallets Projects, “Flask Documentation.” https://flask.palletsprojects.com/ [Accessed: 20 Oct. 2026.]

[30] The Kubernetes Authors, “Expose Pod information to containers through environment variables,” *Kubernetes Documentation*. https://kubernetes.io/docs/tasks/inject-data-application/environment-variable-expose-pod-information/ [Accessed: 20 Oct. 2026.]

[31] Bootstrap contributors, “Bootstrap 5.3 Documentation.” https://getbootstrap.com/docs/5.3/ [Accessed: 20 Oct. 2026.]

[32] Docker Inc., “Dockerfile reference,” *Docker Documentation*. https://docs.docker.com/reference/dockerfile/ [Accessed: 20 Oct. 2026.]

[33] B. Chesneau and contributors, “Gunicorn – Python WSGI HTTP Server for UNIX.” https://docs.gunicorn.org/ [Accessed: 20 Oct. 2026.]

[34] The Helm Authors, “Values” (Chart Best Practices), *Helm Documentation*. https://helm.sh/docs/chart_best_practices/values/ [Accessed: 20 Oct. 2026.]

[35] JSON Schema organization, “JSON Schema.” https://json-schema.org/ [Accessed: 20 Oct. 2026.]

[36] The Kubernetes Authors, “Namespaces,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/overview/working-with-objects/namespaces/ [Accessed: 20 Oct. 2026.]

[37] pytest-dev, “pytest documentation.” https://docs.pytest.org/ [Accessed: 20 Oct. 2026.]

[38] The kubeconform contributors, “kubeconform,” GitHub repository. https://github.com/yannh/kubeconform [Accessed: 20 Oct. 2026.]

[39] The Kubernetes Authors, “Pod lifecycle: termination of Pods,” *Kubernetes Documentation*. https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/#pod-termination [Accessed: 20 Oct. 2026.]

[40] The Helm Authors, “pkg/kube/ready.go,” Helm source code, tag v3.16.2. https://github.com/helm/helm/blob/v3.16.2/pkg/kube/ready.go [Accessed: 20 Oct. 2026.]